

CVE漏洞数据库管理系统

第一章 作品概述

本系统是一个面向大规模 CVE (Common Vulnerabilities & Exposures) 数据的综合管理平台。它集数据采集、清洗、规范化存储、全文检索、统计分析以及可视化展示于一体，既服务安全研究人员对单条漏洞的深度溯源，也支撑运维、开发、合规与供应链团队对海量漏洞的宏观趋势分析与风险评估。系统核心采用 TiDB + Go(Gin) + Bootstrap + JWT 技术栈，并在数据层遵循高度规范化的关系模式，以保证一致性、可扩展性与高并发读写性能。

1.1 背景分析

在当前信息系统快速发展的背景下，漏洞管理正逐步从被动响应向主动识别与智能分析转变。本项目的驱动因素主要来自以下几方面的现实业务需求与挑战：

一、漏洞处置流程零散，无法形成闭环

在许多中小型企业或政企单位中，漏洞响应流程通常由开发、运维、安全等多个团队各自为战，缺乏统一的信息记录与评估机制。漏洞数据往往依赖临时手动收集，处理结果通过邮件、聊天记录等方式分发，极易出现漏报、误报、重复修复等问题，难以支撑审计与合规要求。

本系统提供集中化的 CVE 数据管理能力，支持通过统一的数据模型（如 `cveentry`、`metric`、`description` 等表）记录漏洞的来源、影响范围、CVSS 评分、时间信息、修复状态等，显著提升漏洞闭环管理能力。

二、缺乏漏洞风险等级量化与优先级排序依据

现实场景中，项目数量多、资产复杂，面对海量 CVE 数据时，安全人员常常无法判断“该修哪个”、“哪个最危险”。部分公司甚至根据发布日期排序处理，缺乏量化指标。

系统通过 `metric` 表中存储的标准 CVSS v2/v3 各项评分字段（如 `baseScore`、`attackVector`、`impactScore` 等），结合 `parsedvector` 表中进一步结构化的指标（如 AV/AC/PR/UI），可实现自动化漏洞风险评估与排序，辅助制定修复优先级策略。

三、组件版本分布混乱，人工比对成本高

在实际开发场景中，开源组件和中间件广泛使用，如 log4j、Spring Framework、Apache Commons 等。随着版本更新频繁，团队难以手动跟踪所有依赖项的漏洞信息。运维、开发常常因版本不清导致“修错”或“未修”。

本系统中的 `cpe_entries` 表可记录漏洞与具体组件（CPE）之间的映射关系，结合 `configuration` 表可精确描述受影响版本区间，支持后续与资产系统、依赖分析平台对接，自动比对组件版本与 CVE 匹配关系。

四、缺乏历史数据积累，难以生成趋势报表与支撑决策

安全工作缺乏量化和可视化展示往往导致管理层投入不足。项目在过去处理了多少漏洞、何种类型、是否减少了高危事件，往往无从得知。

通过 `vuln_stats_per_year`、`vuln_impact_types_per_year`、`vuln_type_summary_view` 等汇总视图，可以支撑对历史漏洞数量、漏洞类型分布、年度风险趋势等的统计分析，为管理层提供可量化的决策依据，支撑“以数据驱动的安全治理”。

五、安全团队资源有限，急需提升响应效率与自动化能力

现实中，许多企业的安全团队人数少，责任广，除漏洞处理外还需负责审计、攻防演练、终端加固等。面对每天新增的 CVE、内部资产变更与外部威胁，手动筛选处理不可持续。

该系统支持结构化 CVE 数据聚合与关联分析，可通过 API 或定制视图供扫描平台、代码管理平台调用，或为 DevSecOps 流水线提供自动化安全决策支撑，实现从“人盯人”向“平台辅助”转型。

六、满足合规审计要求，提升安全治理能力

虽然不是所有公司都直接受《等级保护 2.0》《关键信息基础设施保护条例》约束，但在安全检查、外包管理、合作方对接等过程中，统一、可查询、可追踪的漏洞处置记录已成为安全管理的“标配”。

该平台可完整记录每个漏洞的生命周期（包含发布时间、修复状态、影响资产等），通过 `cveentry`、`vendorcomment`、`cve_category_match` 等表，实现从来源追溯到修复反馈的全过程审计，满足监管与审计要求。

1.2 应用场景

1.2.1 用户角色与核心诉求

一、安全运营人员（SOC Analyst / 安全工程师）

安全运营人员是该系统的核心用户，日常负责漏洞监控、预警响应和修复跟踪工作。

➡ 诉求 1：快速获取各类漏洞统计概览

能查看系统中已收录的 CVE 总数、高风险（CVSS ≥ 7 ）的数量、已知被利用的漏洞（Exploited）、以及已修复状态的漏洞，形成实时运营概况。

➡ 诉求 2：具备图表化分析能力

通过 CVSS 分数分布直方图，迅速掌握当前高、中、低风险漏洞的占比；按年和类型查看漏洞趋势和增长模式，支撑年度规划。

➡ 诉求 3：快速定位某类漏洞，追踪修复状态

支持根据 CVE 编号、发布时间、厂商、产品、漏洞类型等高级搜索条件快速定位漏洞，实现“查找-研判-响应”闭环。

二、安全架构师 / DevSecOps 工程师

该类角色注重漏洞数据如何与代码库、资产清单、CI/CD 流水线集成，用于早期风险检测与自动修复建议。

➡ 诉求 4：细粒度厂商/产品维度漏洞建模

系统能支持多级查询：厂商 → 产品 → 版本 → 关联的 CVE，便于与依赖分析、资产扫描平台对接。

➡ 诉求 5：分类精细、可聚合分析的漏洞视图

能获取不同年份、产品线、厂商类别下的漏洞分布与高发类型，支撑补丁策略、代码依赖更新计划。

三、开发人员（Developer / 运维）

开发人员或运维工程师主要关注自己所负责系统的组件是否存在高危漏洞，并希望快速修复或规避。

➡ 诉求 6：精准定位影响自身系统的 CVE

通过产品、版本、厂商、组件名等字段精准搜索漏洞，并获取详细修复建议、利用方式等上下文信息。

➡ 诉求 7：根据 CVSS 分数或攻击向量判断漏洞优先级

通过 CVSS 各项指标（如攻击复杂度、需要权限等），判断漏洞能否远程利用、影响范围，以此决定是否紧急处理。

四、安全管理者 / 风控经理（Security Manager / GRC）

该类用户关注宏观统计结果，评估整体风险水平、漏洞响应能力及团队工作量。

➡ 诉求 8：获取可审计、可报告的漏洞视图

提供按照年份、类型、厂商的聚合统计报表（如饼图、趋势图），支撑合规检查、审计报告与管理汇报。

➡ 诉求 9：查看某年漏洞响应总量与风险趋势

可用于评估某年高风险漏洞数量是否下降，是否存在重复出现的问题组件。

五、系统管理员 / 普通注册用户

系统管理员维护后台运行状态，普通用户可注册登录，执行基础漏洞查询操作。

➡ 诉求 10：权限管理与后台控制

系统支持用户注册、登录、登出，并提供后台总览界面供管理员管理数据和查看系统运行情况。

➡ 诉求 11：无需培训即可使用的便捷界面

界面清晰、交互友好，支持多种筛选方式（关键词、时间、类型、产品），方便普通用户快速查找。

1.2.2 功能性需求

为了提升系统的安全性、可扩展性与部署灵活性，平台应采用现代 Web 安全架构与运维工具链，构建统一的身份认证机制、权限控制体系以及高可用的服务代理方案。如图1-1，以下是对该模块的详细阐述。

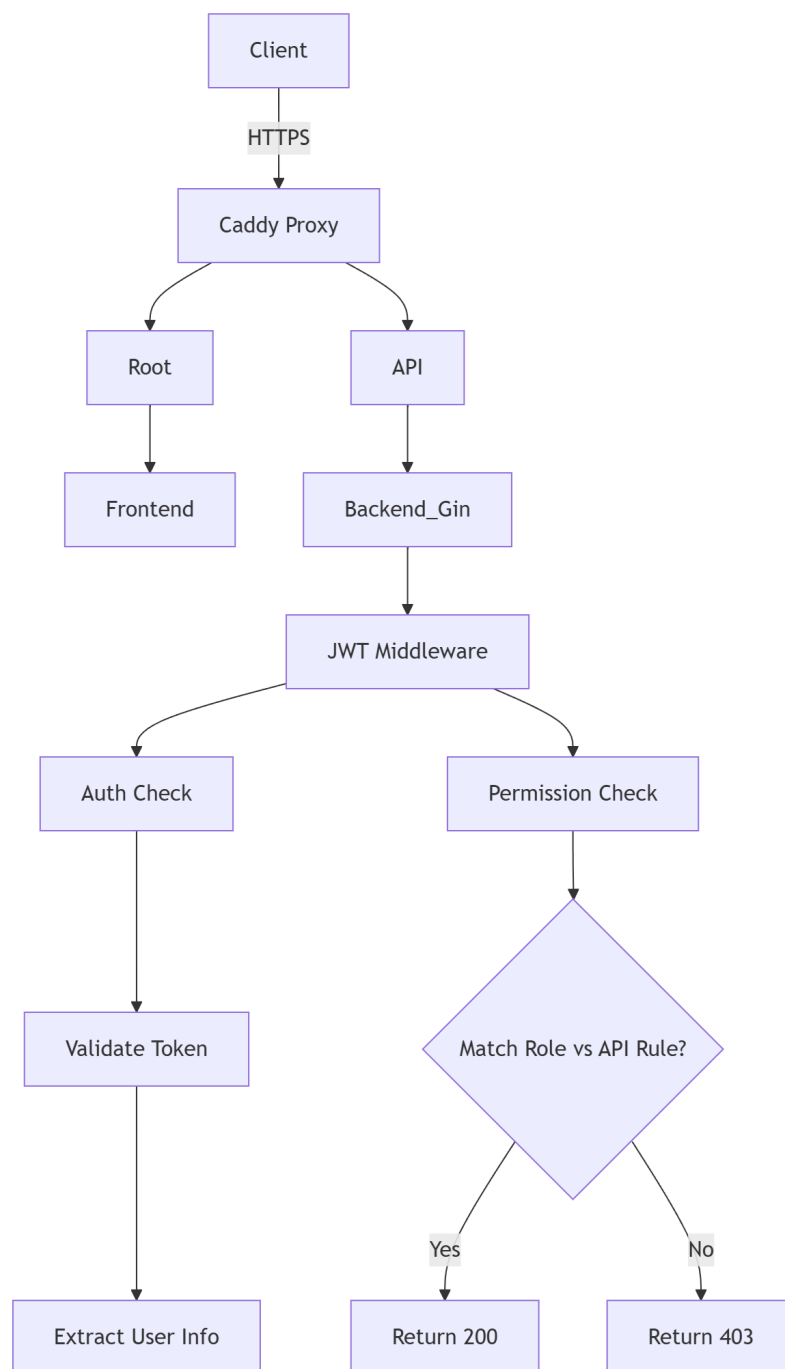


图 1-1 系统架构图

一、支持基于 JWT 的无状态身份认证机制

用户登录成功后，后端会生成一个带有签名的 JWT token，其中包含用户基本信息（如 user_id、username 等）和过期时间。该 token 通过 HTTPS 返回给前端，并由前端在后续请求中携带于 **Authorization: Bearer <token>** 请求头中。后端通过中间件对每个需鉴权的接口自动校验 token 的合法性（包括签名是否有效、是否过期），并从中提取用户信息供业务逻辑使用。

这种实现方式具有显著的技术优势。首先，由于采用无状态设计，不依赖 Session 存储，系统更易于水平扩展。其次，JWT 使用签名机制防止篡改，配合 HTTPS 可确保传输安全，安全性较强。此外，JWT 的 payload 可以灵活嵌入角色、权限等信息，便于实现细粒度的访问控制。

二、提供统一代理入口，实现前后端解耦部署

Caddy Server 作为反向代理服务器，负责将所有客户端请求统一路由到对应的服务。前端静态页面部署在 Caddy 的根路径 `/` 下，而后端 REST API 基于 Gin（或其他 Go 框架）实现，Caddy 会将 `/api/` 路径下的请求代理至该服务。同时，Caddy 自动申请并续签 Let's Encrypt TLS 证书，确保 HTTPS 加密通信。所有外部访问均通过统一的域名（如 <https://tidb-webapp.local>）完成，简化了调用流程和 DNS 配置。

这一架构具备多项技术优势。首先，前后端完全解耦，便于独立开发、测试和部署。其次，统一的入口设计降低了外部调用的复杂度，并简化了 DNS 管理。此外，Caddy 内置 Let's Encrypt 支持，可自动化证书的申请和更新，保障生产环境的安全通信。同时，Caddy 还提供缓存、压缩和负载均衡能力，进一步提升系统的响应效率和整体性能。

三、支持接口级别权限控制

在 JWT token 的 payload 中嵌入用户角色信息（如 `user_id = 1`），后端中间件会根据请求路径匹配对应的权限策略。对于敏感接口（例如 `/api/auth/protected`），中间件会校验当前用户的角色权限，如果权限不足则返回 `403 Forbidden`。权限策略可以通过配置文件或数据库进行动态管理，便于后续灵活调整和扩展。

这种权限控制方式具有多重优势。首先，它实现了精细化的访问控制，能够针对不同接口路径设置权限，避免粗粒度授权带来的安全风险。其次，权限规则集中管理，不仅方便日常维护和调整，也便于后续的安全审计。此外，由于权限判断统一在中间件层完成，业务逻辑层无需重复处理权限校验，有效减少了冗余代码，提升了开发效率。

四、支持安全的跨域访问策略（CORS）

后端服务通过启用 CORS 中间件，对跨域请求进行精细化管控，包括限制允许的来源（Origin）、HTTP 方法（Methods）以及请求头（Headers）。在配置中明确设置是否允许携带凭证

（withCredentials）以及是否暴露特定的响应头字段，同时重点确保对 **Authorization** 头的支持，使 JWT token 能在跨域场景下正常传递。此外，通过白名单机制严格约束跨域请求的发起方，仅允许受信任的前端域名访问后端接口。

这一方案在安全性和灵活性上具有显著优势。首先，通过严格限制请求来源和头部字段，有效防范 CSRF/XSS 等攻击，避免用户凭据被恶意窃取。其次，白名单机制可灵活适配多前端接入需求，例如支持不同子域名或端口的应用访问同一后端服务。同时，CORS 策略在开发阶段可临时放宽以方便调试，而在生产环境收紧配置，兼顾开发效率与线上安全，实现安全策略的动态平衡。

1.3 特色描述

本系统在漏洞管理、安全运营和权限控制方面具备多项显著特色，结合数据库结构设计与功能模块实现，形成了以下几大核心优势：

一、多维度漏洞数据聚合能力

本系统具备**全面的数据覆盖能力**，能够对 CVE 条目进行多维度信息的存储与查询，包括基本描述信息、参考链接、CVSS 安全评分指标、厂商评论、配置规则等，全面支持漏洞分析与风险评估工作。

同时，系统支持**多语言描述功能**，特别是在 CVE 描述字段中引入多语言版本，方便来自不同国家和地区的用户理解与使用，满足国际化安全团队的实际需求。

此外，系统设计了**灵活扩展的标签机制**。通过独立的 CVETag 表，每个 CVE 条目都可以关联任意数量的标签（如 disputed、confirmed、under investigation 等），实现对漏洞的高效分类、筛选与个性化检索，显著提升数据管理的灵活性与实用性。

二、基于 CVSS 的风险评估体系

系统构建了以 CVSS（通用漏洞评分系统）为核心的风险评估模型，实现了多层次、细粒度的风险分析能力。通过 Metric 表，系统可完整存储 CVSS v2 与 v3 的各项评分指标；同时借助 ParsedVector 表对评分向量进行解析，支持按攻击向量（AV）、攻击复杂度（AC）、权限要求（PR）等字段进行筛选，从而便于快速定位高风险漏洞。

为辅助数据可视化与安全决策，系统还设计了多个分数分布统计视图，如 `base_score_distribution` 和 `impact_score_distribution`，帮助用户直观掌握高风险漏洞在总体中的占比情况。此外，系统可依据 `baseScore` 自动计算并更新 severity 字段，实现漏洞的自动分级（LOW / MEDIUM / HIGH），显著提升评估效率与数据展示效果。

三、智能漏洞分类与趋势分析

在漏洞分类方面，系统引入了基于正则规则驱动自动识别机制。通过 `category_lookup` 与 `cve_category_match` 表，系统可自动对 CVE 进行标签归类，如 SQL 注入（SQL Injection）、跨站脚本（XSS）、缓冲区溢出（Overflow）等典型漏洞类型。

为了支持漏洞演化趋势分析，系统提供了多维聚合视图，包括 `vuln_type_summary_view` 和 `vuln_impact_stats_view`，可按年份和漏洞类型两个维度展示不同类别漏洞的增长态势。此外，系统还具备漏洞影响类型识别功能，能够区分漏洞是否可能导致代码执行、权限提升、拒绝服务等实际安全威胁，从而为漏洞优先级排序提供依据。

四、面向资产的漏洞关联建模

系统全面集成 CPE（通用平台枚举）数据，通过 `cpe_entries` 表记录受影响的厂商、产品、版本信息，构建“厂商 → 产品 → 版本 → CVE”四级映射结构，支持针对资产的精细化漏洞查询。用户可通过具体产品名称与版本号，快速定位可能影响自身系统的漏洞条目，提升风险排查的效率和准确性。

同时，系统在 `CVEEntry` 表中设计了 `evaluatorSolution` 字段，用于整合权威修复建议，帮助开发者和运维人员快速获取官方补丁信息与应对措施，大幅缩短响应与修复周期。

五、安全增强的身份认证与访问控制

为确保系统访问的安全性与可控性，平台采用 JWT（JSON Web Token）实现无状态认证机制，避免传统 Session 存储所带来的状态依赖问题，更适用于现代分布式系统架构。

在权限管理方面，系统基于 `users` 与 `permissions` 表构建了角色驱动的权限模型，支持接口级别的权限控制。管理员可根据用户角色分配对各类 API 的 SELECT、INSERT、UPDATE、DELETE

权限，实现细粒度的操作限制。

此外，系统还支持基于 API 路径的访问策略配置，可对如 `/api/protected` 等敏感接口设定特定访问权限，从源头保障数据安全与系统完整性。

六、统一服务代理与 TLS 自动化

系统采用 Caddy 作为统一服务代理，实现前后端请求的集中调度管理。其中，前端页面统一挂载于 `/` 路径，API 接口集中于 `/api/` 路径，极大地简化了域名配置与跨域访问控制逻辑。

为保障通信安全，Caddy 内建 Let's Encrypt 支持，能够自动申请与续签 TLS 证书，无需人工干预即可实现 HTTPS 加密传输，确保生产环境中数据的保密性与完整性。

同时，系统还通过合理配置 CORS（跨域资源共享）策略，设置来源白名单、允许方法和请求头等，有效防范常见的 CSRF 和 XSS 攻击，进一步提升平台整体的安全防护能力。

1.4 应用前景分析

本系统不仅适用于企业内部的安全运营平台建设，还可广泛应用于开源社区、政府机构、云服务商等多个领域，具有广阔的推广价值和发展潜力。

一、企业安全运营平台

该系统特别适用于大型互联网公司、金融机构、电信运营商等对信息安全要求较高的组织。作为安全运营的核心支撑工具，系统可服务于多个业务场景：安全团队可每日实时监控新披露的漏洞信息；DevSecOps 工程师能够将漏洞数据无缝集成进 CI/CD 流水线，自动阻断存在风险的构建；开发者也可根据具体组件版本快速识别依赖风险，从源头管控漏洞引入；安全管理者则可通过系统生成年度漏洞态势报告与趋势图表，用于合规审查与安全分析。

平台的优势体现在：提供实时更新的漏洞统计仪表盘；支持与现有资产管理平台联动，确保漏洞与实际业务系统映射准确；并通过细粒度权限控制机制，有效保障数据访问安全与操作合规性。

二、开源漏洞情报平台

系统亦可作为开源漏洞情报平台，服务对象包括开源项目维护者、漏洞披露平台、CVE 编号分发机构等。平台具备汇总全球公开 CVE 条目的能力，能够系统性地分类展示不同开源项目中漏洞的分布情况，并通过标准化的 RESTful API 向外部系统提供结构化数据服务，便于社区、工具链和研究机构调用利用。

在优势方面，平台支持多语言界面，提升全球社区的可用性；内置的智能分类引擎显著优化漏洞检索效率；基于 CVSS 分数的风险评估机制可推荐修复优先级，帮助维护者合理安排修复节奏。

三、行业监管与合规审计平台

该系统也可广泛应用于政府监管机构、网络安全管理局、金融监管部门等行业监管场景。通过持续监测关键基础设施的漏洞暴露情况，系统可为能源、交通、医疗等重点行业提供漏洞态势分析支撑。监管者可依托平台输出标准格式的合规报表，支撑审计检查和政策制定。

平台优势体现在：支持按年度、漏洞类型、厂商等维度生成聚合统计报表，辅助宏观安全研判；同时，平台内置权限隔离机制，保障跨单位、跨系统的数据隔离与访问控制，提升数据的安全性与监管可信度。

四、教育科研与实训平台

面向高校、网络安全培训机构及 CTF 比赛主办方，系统可作为教育科研与实训平台提供服务。学生可通过平台学习漏洞类型、CVSS 评分标准与风险评估逻辑；教师则可构建教学案例库，用真实漏洞数据驱动教学场景；CTF 平台可接入真实漏洞信息，设计贴近实战的攻防题目，提升实训体验。

该平台具备一系列教育友好特性：提供结构化、高质量的漏洞数据集；支持关键词搜索与多条件筛选；界面直观简洁，使用门槛低，用户无需复杂培训即可快速上手使用。

五、云服务安全平台

对于云计算服务提供商与 SaaS 厂商而言，系统可作为云服务安全平台使用。它可持续监控云端运行的软件栈是否包含已知漏洞，帮助服务商实现对客户环境的漏洞预警。同时，平台支持将修复建议自动推送给客户，并在后台构建完整的云原生漏洞响应流程，优化事件响应效率。

平台的核心优势包括：提供 RESTful API 接口，方便集成至各类自动化工具链；输出格式为标准 JSON，可兼容各类安全系统；并支持多租户架构设计，适配不同客户、不同部门的隔离管理需求，保障数据的独立性与安全性。

第二章 作品设计与实现

2.1 系统整体功能框架

本系统包括六个主要功能模块，分别为：**CVE 数据总览与统计分析、时间维度查询与可视化、厂商与产品信息管理、CVE 具体信息查询、高级搜索功能**，以及**安全认证与用户管理系统**，全面支持对漏洞数据的可视化展示、精确查询与安全访问。如图 2-1 所示：

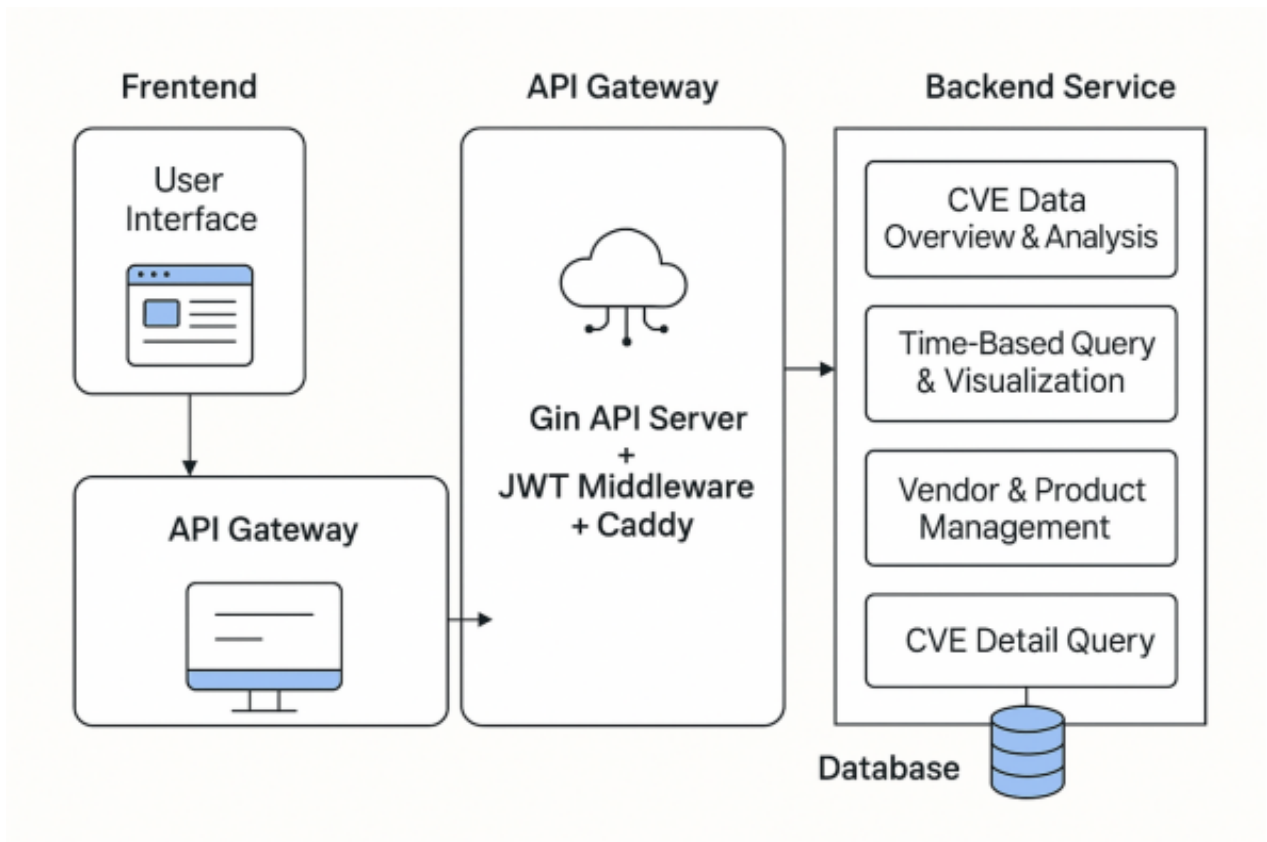


图 2-1 系统整体框架图

2.2 数据库规范论证

2.2.1 CVEEntry

描述：存储 CVE 条目的基本信息。

含义：每一个 CVE 漏洞的基本信息。

代码块

```
1 CREATE TABLE IF NOT EXISTS CVEEntry (  
2     id VARCHAR(50) PRIMARY KEY,  
3     sourceIdentifier VARCHAR(255),  
4     published DATETIME,  
5     lastModified DATETIME,  
6     vulnStatus VARCHAR(50),  
7     evaluatorComment TEXT,  
8     evaluatorImpact TEXT,  
9     evaluatorSolution TEXT,  
10    cisaActionDue DATETIME,  
11    cisaExploitAdd DATETIME,  
12    cisaRequiredAction TEXT,  
13    cisaVulnerabilityName TEXT  
14 );
```


函数依赖：id -> sourceIdentifier, published, lastModified, vulnStatus, evaluatorComment, evaluatorImpact, evaluatorSolution, cisaActionDue, cisaExploitAdd, cisaRequiredAction, cisaVulnerabilityName

所有字段都由主键id决定，没有复合主键，也没有多值依赖或传递依赖，满足BCNF。

2.2.2 Description

描述：存储 CVE 的多语言描述。

含义：每个 CVE 条目可能有多个不同语言的描述。

代码块

```
1 CREATE TABLE IF NOT EXISTS Description (  
2     cve_id VARCHAR(50),  
3     lang VARCHAR(10),  
4     value TEXT,  
5     FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE  
6 );
```

函数依赖：(cve_id, lang) -> value

所有字段都依赖主键(cve_id, lang)，无传递依赖，满足BCNF。

2.2.3 CVETag

描述：存储与 CVE 相关的标签。

含义：为 CVE 添加分类或标签信息。

代码块

```
1 CREATE TABLE IF NOT EXISTS CVETag (  
2     cve_id VARCHAR(50),  
3     sourceIdentifier VARCHAR(255),  
4     tag VARCHAR(100),  
5     FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE  
6 );
```

函数依赖：(cve_id, tag) -> sourceIdentifier

所有字段都依赖主键(cve_id, tag)，无传递依赖，满足BCNF。

2.2.4 Reference

描述：存储与 CVE 相关的参考链接。

含义：提供漏洞相关的外部资源链接。


```

代码块
1 CREATE TABLE IF NOT EXISTS Reference (
2     cve_id VARCHAR(50),
3     url TEXT,
4     source VARCHAR(255),
5     tag VARCHAR(100),
6     FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
7 );

```

函数依赖：(cve_id, source, url) -> tag

所有字段都依赖主键：(cve_id, source, url)，无传递依赖，满足BCNF。

2.2.5 Metric

描述：存储 CVE 的指标信息（如 CVSS 分数）。

含义：记录漏洞的技术影响评分。

```

代码块
1 CREATE TABLE IF NOT EXISTS Metric (
2     cve_id VARCHAR(50),
3     version VARCHAR(10),
4     vectorString TEXT,
5     baseScore FLOAT,
6     baseSeverity VARCHAR(50),
7     impactScore FLOAT,
8     exploitabilityScore FLOAT,
9     source VARCHAR(255),
10    type VARCHAR(50),
11    json_data JSON,
12    FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
13 );

```

函数依赖：(cve_id, version, baseScore) -> vectorString, baseSeverity, impactScore, exploitabilityScore, source, json_data

所有字段都依赖主键(cve_id, version, baseScore)，无传递依赖，满足BCNF。

2.2.6 Weakness

描述：存储 CVE 的弱点信息。

含义：关联 CWE（Common Weakness Enumeration）等弱点信息。

```

代码块
1 CREATE TABLE IF NOT EXISTS Weakness (
2     cve_id VARCHAR(50),

```

```

3      source VARCHAR(255),
4      type VARCHAR(50),
5      lang VARCHAR(10),
6      value TEXT,
7      FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
8  );

```

函数依赖：(cve_id, source, type, lang) -> value

所有字段都依赖主键(cve_id, source, type, lang)，无传递依赖，满足BCNF。

2.2.7 Configuration

描述：存储漏洞的配置匹配规则。

含义：用于描述受影响的产品配置。

代码块

```

1  CREATE TABLE IF NOT EXISTS Configuration (
2      cve_id VARCHAR(50),
3      operator VARCHAR(20),
4      negate BOOLEAN,
5      matchCriteriaId VARCHAR(100),
6      criteria TEXT,
7      vulnerable BOOLEAN,
8      versionStartIncluding TEXT,
9      versionEndIncluding TEXT,
10     versionStartExcluding TEXT,
11     versionEndExcluding TEXT,
12     FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
13 );

```

函数依赖：(cve_id, matchCriteriaId) -> operator, negate, criteria, vulnerable, versionStartIncluding, versionEndIncluding, versionEndExcluding

所有字段都依赖主键(cve_id, matchCriteriaId)，无传递依赖，满足BCNF。

2.2.8 VendorComment

描述：存储厂商对漏洞的评论。

含义：厂商对漏洞的反馈或声明。

代码块

```

1  CREATE TABLE IF NOT EXISTS VendorComment (
2      cve_id VARCHAR(50),
3      organization VARCHAR(255),

```

```

4      comment TEXT,
5      lastModified DATETIME,
6      FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
7  );

```

函数依赖：(cve_id, organization) -> comment, lastModified

所有字段都依赖主键(cve_id, organization)，无传递依赖，满足BCNF。

2.2.9 VectorEmbedding

描述：存储 CVE 的向量数据。

含义：用于机器学习或语义搜索的嵌入向量。

代码块

```

1  CREATE TABLE IF NOT EXISTS VectorEmbedding (
2      cve_id VARCHAR(50) PRIMARY KEY,
3      embedding BLOB,
4      FOREIGN KEY (cve_id) REFERENCES CVEEntry(id) ON DELETE CASCADE
5  );

```

函数依赖：cve_id -> embedding

所有字段都依赖主键cve_id，无传递依赖，满足BCNF。

2.2.10 Users

描述：存储用户信息。

含义：系统用户管理。

代码块

```

1  CREATE TABLE IF NOT EXISTS users (
2      id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3      email       VARCHAR(255) NOT NULL UNIQUE,
4      username    VARCHAR(100) NOT NULL UNIQUE,      -- 新增字段
5      password_hash VARBINARY(60) NOT NULL,
6      created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7      updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
      CURRENT_TIMESTAMP
8  ) ENGINE = InnoDB
9  DEFAULT CHARSET = utf8mb4;

```

函数依赖：id -> email, username, password_hash, created_at, updated_at

所有字段都依赖主键id，无传递依赖，满足BCNF。

2.2.11 permission

描述：存储用户的权限信息。

含义：用户对数据库表的操作权限。

代码块

```
1  CREATE TABLE permissions (  
2      id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3      user_id     BIGINT UNSIGNED NOT NULL,  
4      table_name  VARCHAR(64) NOT NULL,  
5      can_select  BOOLEAN DEFAULT FALSE,  
6      can_insert  BOOLEAN DEFAULT FALSE,  
7      can_update  BOOLEAN DEFAULT FALSE,  
8      can_delete  BOOLEAN DEFAULT FALSE,  
9      created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10     updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
      CURRENT_TIMESTAMP,  
11     CONSTRAINT fk_perm_user  
12         FOREIGN KEY (user_id) REFERENCES users(id)  
13         ON DELETE CASCADE  
14         ON UPDATE CASCADE,  
15     UNIQUE KEY uk_user_table (user_id, table_name)  
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

函数依赖：(user_id, table_name) -> can_select, can_insert, can_update, can_delete

所有字段都依赖主键(user_id, table_name)，无传递依赖，满足BCNF。

2.2.12 E-R图

如图2-2所示，本系统的核心实体包括 CVEEntry（漏洞条目）、Description（多语言描述）、Metric（CVSS评分）、Reference（参考链接）、Configuration（配置规则）、VendorComment（厂商评论）等。其中，CVEEntry 是主实体，其他实体均与其存在外键关联，体现了一对多或多对一的关系。权限模型部分通过 Users 和 Permissions 表实现接口级别控制。

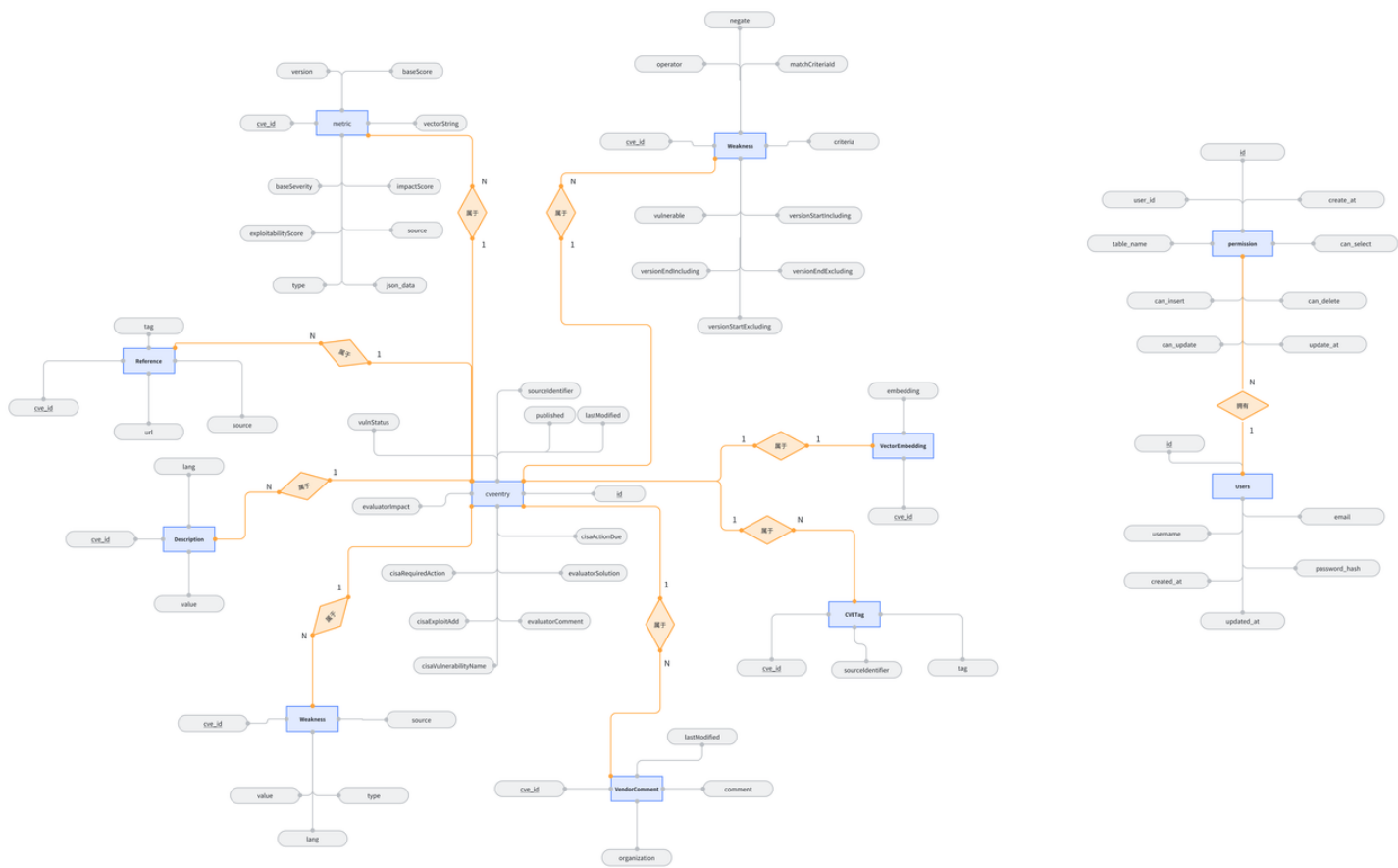


图 2-2 E-R关系图

2.3 系统架构设计

2.3.1 技术选型说明（TiDB、Gin、JWT、HTML+Bootstrap、Python）

本系统在整体架构设计上秉持“高并发、高可用、易部署、可扩展”的原则，针对漏洞数据存储、处理、呈现、安全控制等多个环节，选用了性能优异且社区活跃的技术组件。

在数据库层，采用了开源的 NewSQL 分布式数据库 **TiDB**。TiDB 兼容 MySQL 协议，天然支持水平扩展和强一致性事务，尤其适合需要存储海量结构化/半结构化漏洞数据的应用场景。同时，其原生支持 JSON 类型字段，能直接适配 NVD（国家漏洞数据库）提供的原始数据格式，无需复杂转换，极大提升数据导入与查询效率。借助 HTAP 能力，系统未来亦可在同一套数据中同时完成事务处理与趋势分析，满足合规审计与大屏可视化的需求。

后端部分采用 **Go 语言** 配合高性能的 **Gin 框架** 构建 RESTful API 服务。Go 的静态类型和并发优势使其在处理大规模并发请求时表现出色，而 Gin 提供了极简的路由定义、中间件机制与 JSON 响应支持，能快速搭建高效稳定的 Web 服务。通过 Gin 中间件机制，系统顺利集成了 JWT 认证与 CORS 控制逻辑，保证 API 调用的安全性与跨域访问灵活性。

在身份认证与权限控制方面，选用了基于开放标准的 **JWT（JSON Web Token）** 机制。JWT 支持无状态认证，适用于多节点部署场景，可在 Token 中嵌入用户角色、访问范围等信息，实现接口级别的细粒度权限管理。结合 HTTPS 加密传输，可有效防止认证信息被截获或篡改，增强整体系统的安全防护能力。

前端页面使用 **HTML 搭配 Bootstrap 框架** 进行开发。Bootstrap 提供丰富的响应式组件和布局系统，能够快速搭建界面友好、移动端兼容的页面，适合原型设计与教学展示。同时，由于其无需复杂构建工具，易于与后端模板引擎集成，后续也可以平滑过渡到更现代的前端框架（如 Vue 或 React）。

漏洞数据的预处理工作则交由 **Python** 脚本完成。Python 拥有强大的第三方库支持（如 requests、json、pandas），可高效完成漏洞数据的定时爬取、格式清洗、去重归类等流程。处理后的数据可通过 SQLAlchemy 或 pymysql 接口直接批量导入 TiDB 数据库，实现自动化的数据更新机制。

最后，为提升系统部署效率与外部访问体验，引入了现代化反向代理服务 **Caddy**。Caddy 内置 Let's Encrypt 支持，能够自动申请与续签 HTTPS 证书，保障用户通信安全。同时，其灵活的反向代理功能可将前端页面与后端 API 服务整合于统一入口，实现前后端解耦。通过配置统一的 CORS 策略与路径分发规则（如 /api/ 与 /），系统可提供一致的访问体验，并方便集成 DNS 和负载均衡策略，适应生产环境快速上线部署的需求。

综上，本系统各技术组件搭配紧密，兼顾性能、安全与开发效率，构建出一个稳定、高效、可扩展的漏洞情报平台。

2.3.2 系统架构图描述

如图2-3所示，本系统采用前后端分离 + 微服务化设计思想，整体架构分为五层：

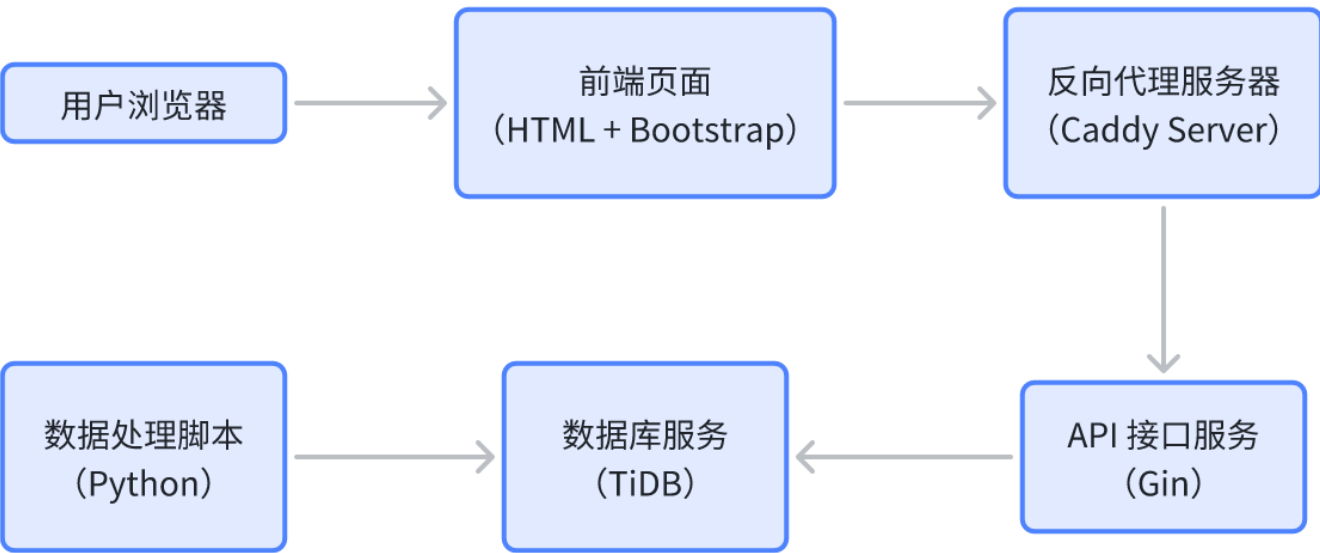


图 2-3 系统架构简图

本系统采用六层分层架构设计，覆盖用户交互、前后端通信、业务处理、数据存储与后台支撑等关键环节，构建了一个高性能、安全可靠的漏洞信息服务平台。

最外层为**用户浏览器层**，用户通过浏览器访问系统提供的可视化界面，所有访问请求首先由反向代理服务器 Caddy 接收。系统统一采用 HTTPS 协议，保障前端与服务端之间的数据传输安全，有效防止中间人攻击和数据泄露。

第二层为**前端页面层**，由 HTML 与 Bootstrap 共同构建。该层负责展示 CVE 漏洞条目、搜索筛选功能、统计图表等信息，用户操作界面清晰直观。所有动态内容均通过 AJAX 异步调用后端 API 实时加载，页面由 Gin 的模板引擎渲染生成，并可平滑切换为静态资源托管模式。为提升安全性和用户体验，前端不直接访问后端服务，而是通过 Caddy 统一入口完成代理转发。

核心通信枢纽为**反向代理服务器层**，系统选用现代化的 Caddy 作为服务网关。Caddy 具备自动化管理 HTTPS 证书的能力，并支持高效的反向代理调度机制，能够将用户请求按路径划分转发至前端或后端接口服务。同时，Caddy 提供灵活的 CORS 策略配置，用于控制跨域访问权限，有效保障接口安全。此外，Caddy 的性能优化能力也为系统稳定运行提供了坚实基础。

第四层为**API 接口服务层**，基于 Go + Gin 框架开发，负责系统所有业务逻辑处理。该层实现用户注册与登录、权限校验、漏洞数据查询与管理等功能，所有接口均遵循 RESTful 风格，便于前后端分离开发。系统采用 JWT（JSON Web Token）实现无状态用户认证，增强系统的扩展性与安全性。数据操作通过 GORM ORM 框架对接 TiDB 数据库，实现对漏洞信息与用户权限配置的精准管理。

支撑后端逻辑的数据存储部分为**数据库服务层**，使用分布式数据库 TiDB。TiDB 支持 MySQL 协议，天然具备高可用、强一致、可横向扩展等特性，适合承载海量的漏洞数据。数据库中存储了 CVE 信息、用户账号、权限配置、统计指标等关键数据，并支持复杂查询与聚合分析操作，为系统提供了稳定、高效的底层支撑。

最后一层为**数据处理脚本层**，采用 Python 实现。该模块负责完成数据解析、清洗与标准化处理后批量导入 TiDB。采集脚本支持增量更新、数据版本比对、异常记录与日志追踪，确保数据完整性和更新及时性，是整个系统数据流的起点与基础保障。

通过上述六层架构协同运行，系统实现了从数据采集、存储、处理到前端展示与用户交互的完整闭环，具备良好的可扩展性、安全性与维护性，适用于漏洞管理、态势感知、信息安全教学等多种应用场景。

2.3.3 模块划分与交互关系（可按功能模块划分：如用户管理、数据采集、搜索展示等）

为了实现 CVE 漏洞数据库管理系统的各项核心功能，系统在架构设计上划分为多个功能模块。每个模块承担不同的职责，并通过清晰的接口进行数据交换和业务协同。这种模块化的设计不仅有助于代码维护和功能扩展，也为后续的权限控制、性能优化和部署升级提供了良好的基础。

一、系统主要功能模块划分

根据系统需求分析与技术实现方案，本系统共划分为以下六大核心功能模块：

① 用户管理模块（User Management）

用户管理模块是系统的基础组成部分，负责用户的注册、登录、登出、信息修改及身份认证等功能。系统通过邮箱或用户名的唯一性校验确保注册信息的准确性，并采用基于 JWT（JSON Web Token）的无状态认证机制，提升安全性与扩展性。用户登录后系统生成令牌进行身份验证，支持用户信息的查询与更新。同时，为防止恶意攻击或暴力破解，该模块还实现了登录失败次数限制机制及相应的安全策略控制。该模块主要涉及 `users` 与 `permissions` 表结构。

② 权限控制模块（Permission Control）

权限控制模块通过角色与权限的关联关系，实现系统资源的细粒度访问控制。系统支持对接口级别（如 SELECT、INSERT、UPDATE、DELETE）的权限进行配置，并通过中间件对每次请求进行拦截与权限判断，保障系统的运行安全。此外，管理员可对用户权限进行动态的增删改查操作，所有权限变更将自动记录日志以供审计。该模块以 `permissions` 表为核心，支撑灵活高效的访问控制机制。

③ 数据采集与处理模块（Data Collection & Processing）

数据采集与处理模块用于处理从 GitHub 下载的 CVE 数据集。该模块使用 Python 编写，结合 json、glob 与 tqdm 等工具实现高效的数据解析与处理流程。系统自动解析原始 JSON 格式的漏洞数据，并将其映射至数据库表结构，支持字段标准化、去重处理、版本对比等操作。为保证数据完整性与稳定性，模块设计了异常日志记录机制与失败重试策略，同时支持全量导入与增量更新两种数据同步模式。该模块涉及的数据库表包括：`CVEEntry`、`Description`、`Metric`、`Reference`、`Configuration`、`VendorComment`、`Weakness` 和 `CVETag`。

④ 漏洞查询与展示模块（CVE Query & Display）

该模块为用户提供多条件的漏洞信息检索与详情展示功能。支持根据 CVE 编号、发布时间、厂商、产品名称、关键词等维度进行模糊或精确搜索，并实现结果的分页展示与自定义排序。用户可点击查看漏洞详情页，获取包括 CVSS 分数、影响等级、修复建议、攻击路径、参考链接等在内的完整信息内容。此外，系统还支持多语言描述切换功能及标签筛选功能，提升了用户的交互体验。该模块主要涉及表结构为：`CVEEntry`、`Description`、`Metric`、`Reference`、`Configuration`、`VendorComment`、`Weakness` 与 `CVETag`。

⑤ 高级搜索与风险评估模块（Advanced Search & Risk Assessment）

为了满足用户对漏洞数据更深入的分析需求，该模块提供了基于 CVSS 向量信息、漏洞类型、攻击方式等字段的高级组合搜索功能。用户可以按 CVSS 分数区间、攻击向量（AV）、攻击复杂度（AC）、权限要求（PR）等多维度条件筛选高危漏洞。系统自动识别并标注漏洞严重等级（如 LOW、MEDIUM、HIGH、CRITICAL），并通过可视化直方图展示评分分布情况，便于用户开展漏洞风险评估。相关表结构包括：`Metric`、`ParsedVector`、`cve_category_match` 与 `category_lookup`。

⑥ 统计分析与可视化模块（Statistics & Visualization）

该模块负责对历史漏洞数据进行多维度的统计与可视化分析，帮助用户把握全局安全趋势。系统可按年份、厂商、产品等维度聚合漏洞数量，生成统计图表；同时展示漏洞类型分布饼图、CVSS 得分柱状图以及高风险漏洞增长趋势折线图等。前端采用 ECharts 或 Chart.js 实现交互式可视化界面，提升数据可读性与使用体验。该模块主要依赖的视图结构包括：`vuln_stats_per_year`、`vuln_impact_types_per_year`、`base_score_distribution`、`impact_score_distribution` 和 `exploitability_score_distribution`。

二、模块间的依赖与交互关系

为实现上述功能模块的高效协作，系统采用松耦合、高内聚的设计原则，各模块间通过标准接口进行通信。如图2-4，具体交互流程如下：

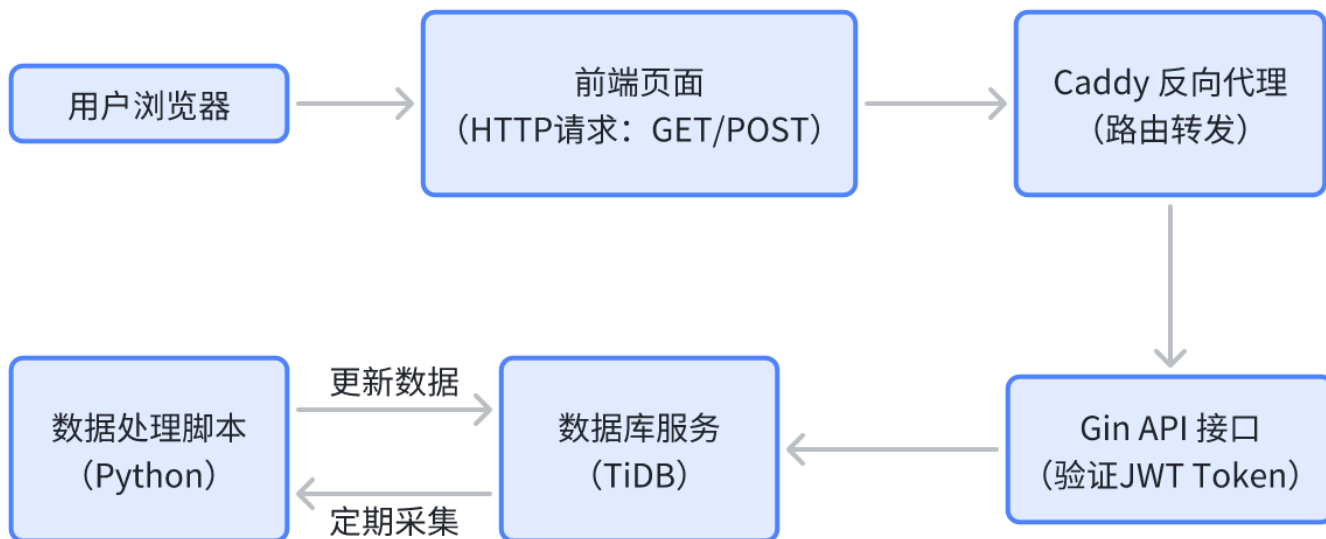


图 2-4 模块依赖关系图

本系统各功能模块间通过清晰的接口定义与逻辑协作构建起高内聚、低耦合的架构体系，确保各功能组件稳定运行与高效协同。首先，**用户管理模块与权限控制模块**紧密集成，在用户成功登录后，系统会基于其用户 ID 查询并加载对应权限配置。所有访问敏感资源的 API 接口在请求前均需通过权限模块进行鉴权校验，确保操作合法性与数据安全性。同时，系统管理员可通过权限模块动态修改用户角色与权限，实现细粒度的访问控制管理。

数据采集模块与数据库模块之间承担着数据流入的关键职能。采集模块通过定时任务抓取外部漏洞数据源，如 NVD 发布的 JSON 文件，经过清洗与结构化处理后，转换为内部数据库兼容格式，并写入 `CVEEntry` 表及相关实体表。系统支持断点续传与数据冲突检测机制，保障数据同步的连续性与一致性。

在用户交互功能方面，**漏洞查询模块与数据库模块**实现用户输入到数据输出的闭环流程。查询模块根据前端传入的参数构造动态 SQL 查询语句，并从 TiDB 中提取符合条件的 CVE 条目。查询逻辑支持多字段联合过滤、关键词模糊匹配与全文索引检索，提升用户查询效率与精准度。

为满足更复杂的检索需求，**高级搜索模块在查询模块基础上进一步封装逻辑**。用户可通过 CVSS 向量、漏洞影响范围、CWE 编号等条件进行深度筛选，系统将这些复杂查询条件解析为底层 SQL 表达式，并通过查询模块统一执行，返回高相关性、高精度的搜索结果，适应专业用户的多维度需求。

系统的**统计分析模块依托数据库层的视图与聚合函数**，周期性或按需生成关键统计指标。通过 `GROUP BY`、`COUNT()`、`SUM()` 等 SQL 聚合函数，模块可统计漏洞数量分布、影响程度分类、时间趋势变化等数据，并借助 ECharts 等图表库将结果以柱状图、饼图、折线图等形式动态展示，提升用户对漏洞态势的直观认知。

在整个系统运行过程中，**日志与监控模块作为辅助组件对所有业务模块进行行为记录与运行状态监控**。包括用户操作日志、异常错误日志、API 调用日志等内容，均被记录并可用于问题追踪与审计分析。同时，系统可接入 Prometheus + Grafana 实现对关键资源、服务响应时间、接口调用频率等指标的可视化监控与自动告警，全面提升系统稳定性与可维护性。

通过上述模块间的有机联动，系统实现了数据采集、权限控制、查询分析与运维监控的一体化闭环管理，具备良好的扩展性、安全性与实用性。

三、模块划分的优势

- ① 高内聚低耦合：每个模块职责单一，模块之间通过清晰的接口通信，便于开发与维护。
- ② 易于扩展：新增功能可通过添加新模块或扩展现有模块实现，不影响已有功能。
- ③ 支持微服务化演进：未来可将各模块拆分为独立服务，构建分布式漏洞管理系统。
- ④ 便于权限隔离与安全管理：不同模块的数据访问权限可分别控制，提升整体安全性。
- ⑤ 提升开发效率：前后端分离 + 模块化设计，使团队成员可并行开发不同模块。

模块名称	功能说明	技术实现
用户管理模块	用户注册、登录、登出、权限分配	Gin + JWT + TiDB(users, permissions)
数据采集模块	解析 JSON 漏洞数据	Python + glob + tqdm + json
漏洞查询模块	支持按关键词、时间、厂商、产品等条件检索 CVE	Gin + SQL 查询 + TiDB
高级搜索模块	支持 CVSS 分数、攻击向量、影响等级等高级筛选	Gin + SQL 查询 + TiDB
统计分析模块	展示漏洞趋势、风险分布、分类统计等	Gin + ECharts + TiDB 视图
权限控制模块	实现接口级别权限控制	JWT + Gin 中间件 + TiDB(permissions)

表 2-1 系统核心模块与功能说明表

2.4 核心功能实现

2.4.1 数据采集与清洗流程（如何从现有的Json数据里面提取数据，并存储到tidb中）

一、数据来源与格式

为构建一个结构化、可查询、具备高可用性的 CVE 漏洞信息数据库系统，首先需要从权威来源获取原始漏洞数据，并通过统一的数据处理流程进行解析、标准化和去重，最终写入 TiDB 数据库以支持后续查询与分析。本系统主要的数据来源为美国国家标准与技术研究院（NIST）提供的 **国家漏洞数据库**（NVD, *National Vulnerability Database*）。该数据库是全球通用的 CVE 数据发布平台，持续更新并公开所有 CVE 编号下的漏洞信息。

NVD 官方以 **JSON 格式**发布每条 CVE 漏洞条目的完整结构化数据，包括但不限于：漏洞编号（CVE-ID）、漏洞的发布时间与最近更新时间、CVSS 安全评分（v2/v3）、攻击向量（如本地、远程、物理）、漏洞的描述信息（支持多语言）、相关参考链接、厂商评论、CPE 匹配项、影响组件范

围等关键字段。这些数据由 NIST 根据行业标准统一维护，并通过 REST API 或压缩文件的形式批量发布，通常以年度分批组织，每个文件的命名格式为 `CVE-YYYY-NNNN.json`。

原始数据文件被打包为 ZIP 格式，便于批量下载和离线处理。系统的数据处理脚本对其中的 JSON 文件进行遍历读取，并按字段映射规则解析为数据库表结构。针对不同条目之间可能存在的重复、冗余或字段差异等情况，系统设计了相应的 **数据标准化与去重策略**，确保最终导入数据库的内容准确、完整且无冗余。通过这一处理流程，原始半结构化的 CVE 数据得以转化为可高效索引和查询的关系型数据，为后续搜索引擎、统计分析和安全预警提供了坚实的数据基础。

二、数据采集与处理流程

数据采集与处理流程主要包括以下几个步骤：

① 文件扫描与加载

程序使用 `glob` 模块扫描指定目录下的所有 `.json` 文件，并逐个打开进行解析。路径由配置项 `JSON_DIR` 控制，便于后续扩展至网络爬虫或远程 FTP 加载。

代码块

```
1 files = glob(os.path.join(JSON_DIR, "*.json"))
```

② JSON 解析与字段映射

每条 JSON 记录被解析为 Python 字典对象后，提取其中的关键字段，并根据预定义的数据模型映射到对应的数据库表中。例如：

- 主表 `CVEEntry`：记录漏洞的基本信息；
- 子表 `Description`：多语言描述；
- 子表 `Metric`：CVSS 评分信息；
- 子表 `Reference`：参考链接；
- 子表 `Configuration`：受影响组件版本区间；
- 子表 `Weakness`：关联 CWE 弱点；
- 子表 `VendorComment`：厂商反馈；
- 子表 `CVETag`：自定义标签分类；
- 子表 `VectorEmbedding`（预留）：用于语义搜索的嵌入向量（预留）。

③ 数据清洗与校验

在插入数据库前，程序对字段值进行清洗和有效性判断。例如：

- 使用 `is_empty()` 函数过滤空值或无效字段；
- 对 `CVETag`、`Reference` 等多值字段进行循环遍历插入；

- 对 `configurations` 中的嵌套结构进行展开处理；
- 对 `vectorEmbedding` 进行长度检查（必须为 384 维）；

代码块

```
1  # 主表
2  cursor.execute("""
3      INSERT IGNORE INTO CVEEntry (
4          id, sourceIdentifier, published, lastModified, vulnStatus,
5          evaluatorComment, evaluatorImpact, evaluatorSolution,
6          cisaActionDue, cisaExploitAdd, cisaRequiredAction,
7          cisaVulnerabilityName
8      ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
9  """, (
10     cve_id,
11     cve.get('sourceIdentifier'),
12     cve.get('published'),
13     cve.get('lastModified'),
14     cve.get('vulnStatus'),
15     cve.get('evaluatorComment'),
16     cve.get('evaluatorImpact'),
17     cve.get('evaluatorSolution'),
18     cve.get('cisaActionDue'),
19     cve.get('cisaExploitAdd'),
20     cve.get('cisaRequiredAction'),
21     cve.get('cisaVulnerabilityName')
22 ))
23
24 # 描述
25 for desc in cve.get('descriptions', []):
26     if not is_empty(desc.get('value')):
27         cursor.execute("""
28             INSERT INTO Description (cve_id, lang, value)
29             VALUES (%s, %s, %s)
30             """, (cve_id, desc.get('lang'), desc.get('value')))
31
32 # 标签
33 for tag_block in cve.get('cveTags', []):
34     src = tag_block.get('sourceIdentifier')
35     for tag in tag_block.get('tags', []):
36         if not is_empty(tag):
37             cursor.execute("""
38                 INSERT INTO CVETag (cve_id, sourceIdentifier, tag)
39                 VALUES (%s, %s, %s)
40                 """, (cve_id, src, tag))
41
42 # 引用
```

```

42 for ref in cve.get('references', []):
43     url = ref.get('url')
44     source = ref.get('source')
45     for tag in ref.get('tags', []):
46         if not is_empty(url):
47             cursor.execute("""
48                 INSERT INTO Reference (cve_id, url, source, tag)
49                 VALUES (%s, %s, %s, %s)
50             """, (cve_id, url, source, tag))
51
52 # 指标
53 for ver in ['cvssMetricV2', 'cvssMetricV30', 'cvssMetricV31', 'cvssMetricV40']:
54     for metric in cve.get('metrics', {}).get(ver, []):
55         data = metric.get('cvssData', {})
56         cursor.execute("""
57             INSERT INTO Metric (
58                 cve_id, version, vectorString, baseScore, baseSeverity,
59                 impactScore, exploitabilityScore, source, type, json_data
60             ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
61         """, (
62             cve_id,
63             data.get('version'),
64             data.get('vectorString'),
65             data.get('baseScore'),
66             data.get('baseSeverity'),
67             metric.get('impactScore'),
68             metric.get('exploitabilityScore'),
69             metric.get('source'),
70             metric.get('type'),
71             json.dumps(metric)
72         ))
73
74 # 弱点
75 for w in cve.get('weaknesses', []):
76     source = w.get('source')
77     wtype = w.get('type')
78     for desc in w.get('description', []):
79         cursor.execute("""
80             INSERT INTO Weakness (cve_id, source, type, lang, value)
81             VALUES (%s, %s, %s, %s, %s)
82             """, (cve_id, source, wtype, desc.get('lang'), desc.get('value')))
83
84 # 配置
85 for conf in cve.get('configurations', []):
86     operator = conf.get('operator')
87     for node in conf.get('nodes', []):
88         node operator = node.get('operator')

```


④ 批量写入与事务控制

为提升插入效率并减少数据库压力，程序采用 批处理提交机制 ，每处理完一定数量（默认 100 条）记录后执行一次 `commit()` 操作，避免单次事务过大导致性能下降或失败。

代码块

```
1  if count % BATCH_SIZE == 0:
2      conn.commit()
```

⑤ 错误日志与异常处理

程序通过 `try-except` 块捕获文件解析、字段缺失、数据库插入等环节可能出现的异常，并打印错误信息以便排查问题，同时不影响其余数据的正常处理。

代码块

```
1  except Exception as e:
2      print(f"[ERROR] File: {path} - {e}")
```

2.4.2 漏洞检索与高级查询实现

一、简单的CVE编号查询

为了满足用户快速定位特定漏洞的需求，系统提供了基于 CVE 编号（如 `CVE-2021-44228`）的精确查询功能。该功能允许用户通过输入完整的 CVE ID 直接跳转到对应的漏洞详情页面，展示其完整的技术细节、评分信息、参考链接等内容。

① 功能描述

简单 CVE 编号查询是系统中最基础也是最常用的查询方式之一。它具有以下特点：

特性	描述
输入方式	用户在前端搜索框中输入完整的 CVE ID（如 CVE-XXXX-XXXXX）
查询类型	精确匹配，不支持模糊或部分匹配
返回内容	包括漏洞基本信息、多语言描述、CVSS 评分、配置规则、厂商评论、参考链接等
展示形式	在 cve_detail.html 页面中以结构化方式呈现

表 2-2 CVE编号查询特点表

② 技术实现流程

当用户在前端页面输入指定的 CVE 编号并提交查询请求后，系统会向后端接口 `/api/cve/{id}` 发起 GET 请求。后端接收到请求后，首先在 `cveentry` 主表中根据该编号查找对应的漏洞基础信息，随后分别查询与该漏洞关联的多个子表数据，包括 `descriptions`（多语言描述）、`metrics`（CVSS 评分信息）、`references`（参考链接）、`vendorcomments`（厂商评论）、`weaknesses`（CWE 弱点信息）以及 `configurations`（受影响组件配置）。所有查询结果将被统一封装为结构体，并传递给前端模板进行渲染。最终，用户在前端页面上可以完整查看该漏洞的详细信息，包括技术细节、评分指标、参考资料和厂商反馈等内容。

③ 数据库交互方式

系统使用 GORM 执行原始 SQL 查询，分别从多个关联表中提取所需字段，避免全量 JOIN 查询带来的性能问题。以下是主要查询逻辑：

1. 主表查询 (CVEEntry)

代码块

```
1 SELECT id, sourceIdentifier, published, lastModified, vulnStatus
2 FROM cveentry
3 WHERE id = ?
```

2. 关联子表查询

- Descriptions: `SELECT value FROM description WHERE cve_id = ? AND lang = 'en'`
- Configurations: `SELECT criteria FROM configuration WHERE cve_id = ?`
- Metrics: `SELECT ... FROM metric WHERE cve_id = ?`
- References: `SELECT url, source, tag FROM reference WHERE cve_id = ?`
- VendorComments: `SELECT organization, comment, lastModified FROM vendorcomment WHERE cve_id = ?`
- Weaknesses: `SELECT source, type, lang, value FROM weakness WHERE cve_id = ?`

④ 关键代码解析

通过 Gin + GORM 实现一个安全高效的 CVE 查询接口。每张子表单独查询，避免复杂 JOIN 导致的性能瓶颈；同时使用结构体绑定返回值，保证数据格式一致性。

代码块

```
1 func GetSimpleCveByID(db *gorm.DB) gin.HandlerFunc {
2     return func(c *gin.Context) {
3         id := c.Param("id")
4     }
```

```

5      // 1. 查询主表信息
6      var entry SimpleCVEEntry
7      db.Raw(`SELECT id, sourceIdentifier, published, lastModified,
vulnStatus
8          FROM cveentry WHERE id = ?`, id).Scan(&entry)
9
10     if entry.ID == "" {
11         // 处理未找到的情况
12         c.HTML(http.StatusNotFound, "cve_detail.html", gin.H{"error": "CVE
不存在"})
13         return
14     }
15
16     // 2. 查询各个子表
17     var descriptions []SimpleDescription
18     db.Raw(`SELECT value FROM description WHERE cve_id = ? AND lang =
'en`, id).Scan(&descriptions)
19     entry.Descriptions = descriptions
20
21     // 类似地查询 metrics, references, vendor_comments 等
22
23     // 3. 渲染模板
24     c.HTML(http.StatusOK, "cve_detail.html", gin.H{
25         "cve": entry,
26     })
27 }
28 }

```

⑤ 查询结果返回结构

查询成功时，系统会返回如下 JSON 格式的数据（前端页面可直接解析并展示）：

代码块

```

1  {
2      "id": "CVE-2021-44228",
3      "source_identifier": "security@apache.org",
4      "published": "2021-12-09T20:15:00Z",
5      "last_modified": "2022-01-13T17:00:00Z",
6      "vuln_status": "Analyzed",
7      "configurations": [
8          {"criteria": "cpe:2.3:a:apache:log4j:1.2.17:*:*:*:*:*:*"}
9      ],
10     "descriptions": [
11         {"value": "Apache Log4j2 2.0-beta9 到 2.17.1 版本存在远程代码执行漏洞..."}
12     ],
13     "metrics": [

```

```

14     {
15         "version": "3.1",
16         "vector_string": "/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H",
17         "base_score": 10.0,
18         "base_severity": "CRITICAL",
19         "type": "CVSSv3"
20     }
21 ],
22 "references": [
23     {"url": "https://logging.apache.org/log4j/2.x/security.html", "tag":
    "Patch", "source": "Apache"}
24 ],
25 "vendor_comments": [],
26 "weaknesses": [
27     {"source": "MITRE", "type": "Primary", "lang": "en", "value": "CWE-434 可信
    危险类型文件上传"}
28 ]
29 }

```

⑥ 效果展示

CVE Detail - CVE-2021-44228

Basic Information

Source Identifier:

Published: 2021-12-10 10:15:09 -0500 EST

Last Modified: <nil>

Status:

Describe

Apache Log4j2 2.0-beta9 through 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1) JNDI features used in configuration, log messages, and parameters do not protect against attacker controlled LDAP and other JNDI related endpoints. An attacker who can control log messages or log message parameters can execute arbitrary code loaded from LDAP servers when message lookup substitution is enabled. From log4j 2.15.0, this behavior has been disabled by default. From version 2.16.0 (along with 2.12.2, 2.12.3, and 2.3.1), this functionality has been completely removed. Note that this vulnerability is specific to log4j-core and does not affect log4net, log4cxx, or other Apache Logging Services projects.

Configuration

- cpe:2.3:a:apache:log4j:*.***.*.*.*
- cpe:2.3:a:apache:log4j:*.***.*.*.*
- cpe:2.3:a:apache:log4j:*.***.*.*.*
- cpe:2.3:a:apache:log4j:2.0:-:*.***.*.*
- cpe:2.3:a:apache:log4j:2.0-beta9:*.***.*.*

图 2-5 CVE具体信息展示图

二、根据厂商和版本查找

为了满足开发人员与运维团队对特定组件漏洞信息的精准定位需求，系统提供了基于 厂商（Vendor）和 产品版本（Product Version）的多级查询功能。该功能支持用户通过输入厂商名称、产品名称及具体版本号，快速检索出影响该组件的所有 CVE 条目，并进一步查看其详细信息、CVSS 分数、修复建议等关键内容。

① 功能描述

本功能的核心目标是实现“厂商 → 产品 → 版本 → 漏洞”的四级关联查询能力，帮助用户快速识别所使用组件是否存在已知安全风险。例如：

- 查询 `Apache` 公司发布的 `Log4j` 组件在 `2.15.0` 版本中存在的所有漏洞；
- 查看某个产品（如 `nginx`）在不同版本中的漏洞分布情况；
- 支持模糊匹配厂商或产品名，提升搜索灵活性。

② 技术实现方式

该功能通过构建多层级的结构化漏洞数据库实现高效关联查询。首先，后端采用TiDB设计四层数据模型，通过外键建立厂商表、产品表、版本表与漏洞表之间的级联关系，并针对厂商名、产品名等字段创建全文索引以支持模糊匹配。查询时，API 接收用户输入的条件参数，动态生成多表 JOIN 查询语句，例如通过 `厂商名称 LIKE %Apache% AND 产品名称=Log4j AND 版本号=2.15.0` 的组合条件检索关联漏洞。

③ 数据库结构支持

系统通过 `cpe_entries` 表记录漏洞与 CPE（Common Platform Enumeration）之间的映射关系，其中包含以下关键字段：

字段名	类型	描述
id	BIGINT	主键，唯一标识每条记录。自增主键，便于快速检索与索引优化。
cve_id	VARCHAR(100)	关联的 CVE 编号（如 CVE-2021-44228），用于连接CVEEntry表进行联合查询。
vulnerable	BOOLEAN	标识当前 CPE 是否受该漏洞影响，可用于过滤有效/无效配置项。
part	VARCHAR(10)	组件类型（如 a=application, o=operating system, h=hardware），表示漏洞影响的系统层级。
vendor	VARCHAR(255)	厂商名称，例如apache,microsoft,openjdk，支持模糊匹配以查找特定厂商的所有漏洞。
product	VARCHAR(255)	产品名称，例如log4j,windows,nginx，配合厂商字段使用，实现“厂商 → 产品”筛选。
version	VARCHAR(100)	产品版本号，如2.15.0，支持精确版本匹配与区间判断，用于定位具体受影响版本。
update_version	VARCHAR(100)	更新版本号，某些 CPE 包含补丁级别或更新包编号，可用于更细粒度的影响范围判断。
edition	VARCHAR(100)	发行版或版本类型，例如企业版、社区版、测试版等，用于区分不同发行版本的差异。
language	VARCHAR(100)	软件语言版本，如en-US,zh-CN，用于国际化支持场景下的漏洞匹配。
sw_edition	VARCHAR(100)	软件发布形式（如 desktop, server, mobile 等），有助于判断漏洞是否适用于当前环境。
target_sw	VARCHAR(100)	目标软件平台（如 Java VM、Node.js 运行时），用于判断运行环境是否受漏洞影响。
target_hw	VARCHAR(100)	目标硬件平台（如 x86, ARM），用于嵌入式设备或特定架构下的漏洞匹配。
other	VARCHAR(100)	其他补充信息字段，用于存储未涵盖在上述字段中的附加条件或特殊限制。

表 2-3 cpe_entries字段表

代码块

```
1 CREATE TABLE IF NOT EXISTS cpe_entries (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     cve_id VARCHAR(100),  
4     vulnerable BOOLEAN,  
5     part VARCHAR(10),  
6     vendor VARCHAR(255),  
7     product VARCHAR(255),  
8     version VARCHAR(100),  
9     update_version VARCHAR(100),  
10    edition VARCHAR(100),  
11    language VARCHAR(100),  
12    sw_edition VARCHAR(100),  
13    target_sw VARCHAR(100),  
14    target_hw VARCHAR(100),
```

```
15     other VARCHAR(100)
16 );
```

1) 根据厂商查找产品和CVE

CVE database

Vulnerabilities

[By Date](#)

[By type](#)

[CVSS Scores](#)

[Search](#)

Vulnerable Software

Vendors

Products

Version Search

Account

[Login](#)

CVE id (eg: CVE-2021-44228)

Search

Vendors - Vendor names starting with "All"

a

Search

Browse vendor names starting with:

All

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

5

6

7

8

9

@

31641 vendors found

Vendor Name	Products	Vulnerabilities
*	1	1
-	2	11
.bbsoftware	1	3
.joomclan	1	1
.matteoiammarrone	1	1
01-scripts	2	2
07fly	1	1
0branch	1	2
0kims	1	1

图 2-6 Vendors查询展示图

A. 提供 `/api/vendors` 接口，接收 `vendor` 参数查询对应的厂商：

代码块

```
1  const baseURL = "/api/vendors";
2  fetch(`${baseURL}?letter=${letter}&page=${page}`)
```

代码块

```
1  func GetVendors(db *gorm.DB) gin.HandlerFunc {
2      return func(c *gin.Context) {
3          page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
4          limit := 20
5          offset := (page - 1) * limit
6
7          search := c.Query("search")
8          letter := c.Query("letter")
9
10         var rows *sql.Rows
11         var total int
12         var err error
13
14         base := `
```



```

15         SELECT vendor, COUNT(DISTINCT product) as product_count,
COUNT(DISTINCT cve_id) as vuln_count
16         FROM cpe_entries
17         WHERE %s
18         GROUP BY vendor
19         ORDER BY vendor
20         LIMIT ? OFFSET ?`
21
22     countSQL := `SELECT COUNT(DISTINCT vendor) FROM cpe_entries WHERE %s`
23
24     var condition string
25     var args []interface{}
26
27     if search != "" {
28         condition = "LOWER(vendor) LIKE ?"
29         args = append(args, "%"+strings.ToLower(search)+"%")
30     } else if letter != "" {
31         condition = "LOWER(vendor) LIKE ?"
32         args = append(args, strings.ToLower(letter)+"%")
33     } else {
34         condition = "1=1"
35     }
36
37     // 查询总数
38     err = db.Raw(fmt.Sprintf(countSQL, condition),
args...).Row().Scan(&total)
39     if err != nil {
40         c.JSON(500, gin.H{"error": "count error"})
41         return
42     }
43
44     // 查询分页数据
45     args = append(args, limit, offset)
46     rows, err = db.Raw(fmt.Sprintf(base, condition), args...).Rows()
47     if err != nil {
48         c.JSON(500, gin.H{"error": "query error"})
49         return
50     }
51     defer rows.Close()
52
53     var results []VendorEntry
54     for rows.Next() {
55         var v VendorEntry
56         if err := rows.Scan(&v.Vendor, &v.ProductCount, &v.VulnCount); err
== nil {
57             results = append(results, v)
58         }

```

```

59     }
60
61     c.JSON(200, VendorResponse{
62         Vendors:    results,
63         Total:      total,
64         TotalPages: int(math.Ceil(float64(total) / float64(limit))),
65     })
66 }
67 }

```

Vendors - Vendor names starting with "All"

Browse vendor names starting with:

All
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z
0
1
2
3
4

5
6
7
8
9
@

31641 vendors found

Vendor Name	Products	Vulnerabilities
*	1	1
-	2	11
.bbsoftware	1	3
.joomclan	1	1
.matteoiamarrone	1	1
01-scripts	2	2
07fly	1	1
0branch	1	2

图 2-7 Vendors模糊字段查询展示图

B. 根据上一步查找出来的厂商返回对应的产品

代码块

```

1  <td><a href="/vendor_product_details.html?
   vendor=${encodeURIComponent(vendor.vendor)}">${vendor.product_count}</a></td>

```

代码块

```

1  func GetVendorProducts(db *gorm.DB) gin.HandlerFunc {
2      return func(c *gin.Context) {
3          vendor := c.Param("vendor")
4
5          // 定义切片存储查询结果
6          var products []ProductWithPart

```

```

7
8      // 修改 SQL 查询, 同时获取 product 和 part 字段
9      err := db.Raw(`
10          SELECT DISTINCT product, part
11          FROM cpe_entries
12          WHERE LOWER(vendor) = ?
13          ORDER BY product
14      `, strings.ToLower(vendor)).Scan(&products).Error
15
16      if err != nil {
17          c.JSON(500, gin.H{"error": "query error"})
18          return
19      }
20
21      // 返回结果
22      c.JSON(200, gin.H{
23          "vendor": vendor,
24          "products": products,
25      })
26  }
27  }

```

Vendors - Vendor names starting with "All"

Browse vendor names starting with:

All

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

5

6

7

8

9

@

31641 vendors found

Vendor Name	Products	Vulnerabilities
*	1	1
-	2	11
.bbsoftware	1	3
.joomclan	1	1
.matteoiamarrone	1	1
01-scripts	2	2
07fly	1	1
0branch	1	2
0kime	1	1

图 2-8 Vendors对应的Products展示图

CVE id (eg: CVE-2021-44228)

Search

Products for -

	Product Name	Product Name
1	-	OS
2	wireless_ip_camera_360	Hardware

图 2-9 具体Products查询展示图

C. 根据厂商查找所有的CVE

代码块

```
1 <td><a href="/vendor_vulnerabilities_details.html?
  vendor=${encodeURIComponent(vendor.vendor)}">${vendor.vuln_count}</a></td>
```

代码块

```
1 func GetVendorVulnerabilities(db *gorm.DB) gin.HandlerFunc {
2     return func(c *gin.Context) {
3         vendor := c.Param("vendor")
4
5         var cveIds []string
6
7         err := db.Raw(`
8             SELECT DISTINCT cve_id
9             FROM cpe_entries
10            WHERE LOWER(vendor) = ?
11            ORDER BY cve_id
12        `, strings.ToLower(vendor)).Scan(&cveIds).Error
13
14         if err != nil {
15             c.JSON(500, gin.H{"error": "query error"})
16             return
17         }
18
19         c.JSON(200, gin.H{
20             "vendor": vendor,
21             "cve_ids": cveIds,
22         })
23     }
24 }
```

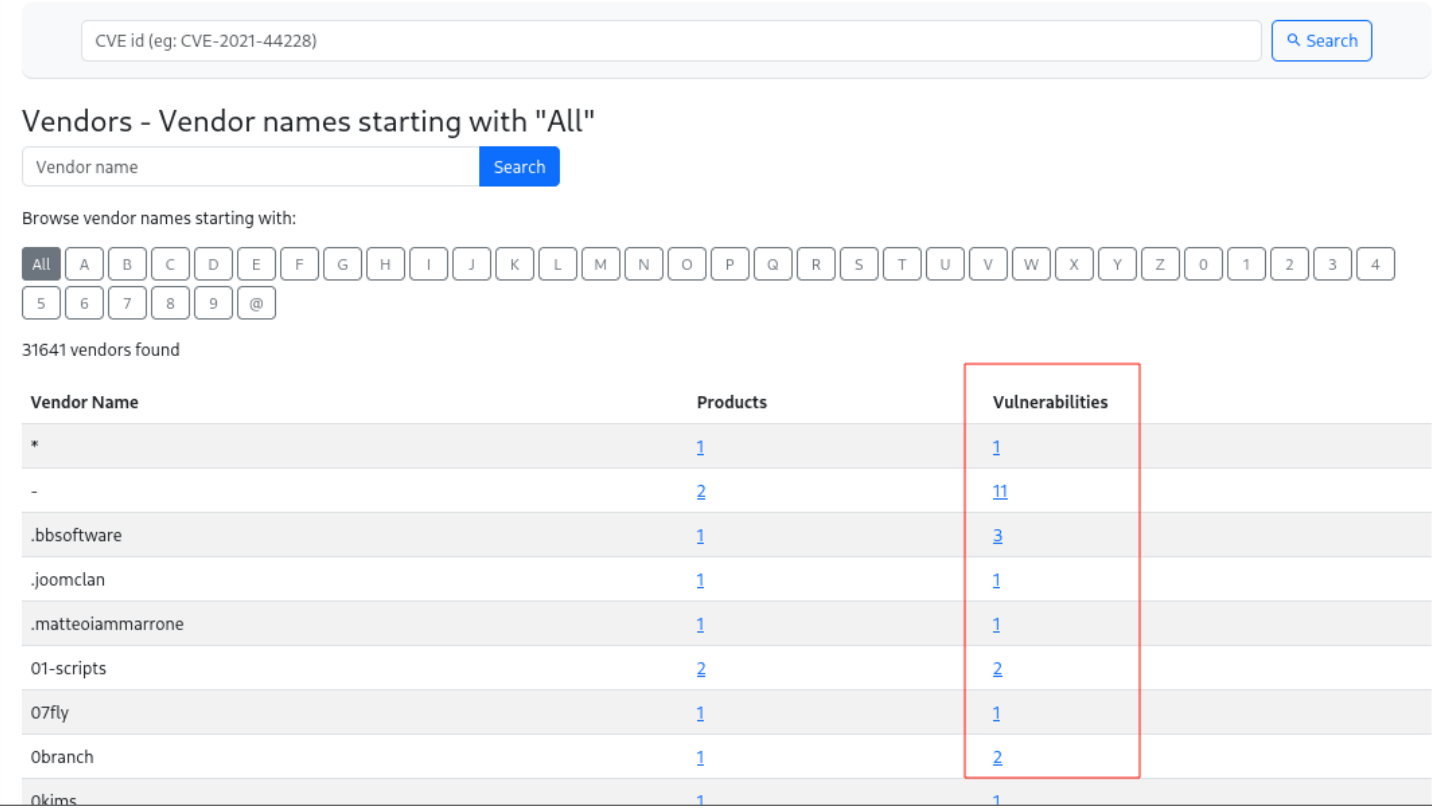


图 2-10 Vendors对应的Vulnerabilities展示图

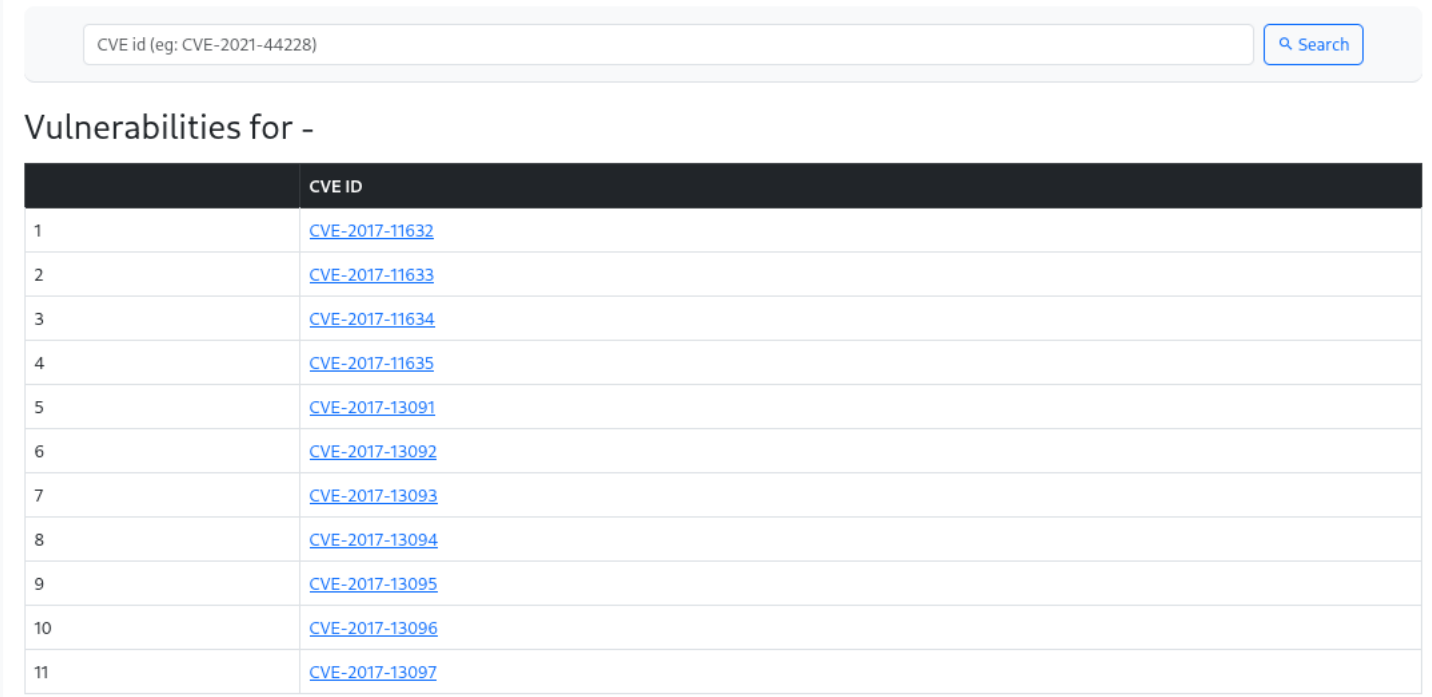


图 2-11 具体的Vulnerabilities查询展示图

D. 根据输出的部分关键词，返回出建议意见

代码块

```
1 fetch(`${baseUrl}/suggest?keyword=${encodeURIComponent(keyword)}`)
```

```
1 代码块 func VendorSuggest(db *gorm.DB) gin.HandlerFunc {
2      return func(c *gin.Context) {
3          keyword := strings.ToLower(strings.TrimSpace(c.Query("keyword")))
4          if keyword == "" {
5              c.JSON(400, gin.H{"error": "keyword is required"})
6              return
7          }
8
9          var vendors []string
10
11         err := db.Raw(`
12             SELECT DISTINCT vendor
13             FROM cpe_entries
14             WHERE LOWER(TRIM(vendor)) LIKE ?
15             ORDER BY vendor
16             LIMIT 10
17         `, keyword+"%").Scan(&vendors).Error
18
19         if err != nil {
20             c.JSON(500, gin.H{"error": "query error"})
21             return
22         }
23
24         c.JSON(200, gin.H{"suggestions": vendors})
25     }
26 }
27
28 type ProductWithPart struct {
29     Product string `json:"product"`
30     Part     string `json:"part"`
31 }
```

CVE id (eg: CVE-2021-44228)

Search

Vendors - Vendor names starting with "All"

a

Search

a-a-s_application_access_server

a-blog

a-conman

a-enterprise

a-forms_project

a-ftp

a-link

a-pdf

a-shop

a-stage-inc

01-scripts

07fly

0branch

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

	Products	Vulnerabilities
a-ftp	1	1
a-link	2	11
a-pdf	1	3
a-shop	1	1
a-stage-inc	1	1
01-scripts	2	2
07fly	1	1
0branch	1	2

图 2-12 Vendors查询建议展示图

E. 考虑到数量的限制，使用 `LIMIT ? OFFSET ?`，只需要读取部分信息

代码块

```

1  base := `
2      SELECT vendor, COUNT(DISTINCT product) as product_count, COUNT(DISTINCT
3      cve_id) as vuln_count
4      FROM cpe_entries
5      WHERE %s
6      GROUP BY vendor
7      ORDER BY vendor
8      LIMIT ? OFFSET ?`
9  countSQL := `SELECT COUNT(DISTINCT vendor) FROM cpe_entries WHERE %s`

```

2) 根据产品查看具体的厂商和查找CVE

CVE database

Vulnerabilities

[By Date](#)

[By type](#)

[CVSS Scores](#)

[Search](#)

Vulnerable Software

[Vendors](#)

[Products](#)

[Version Search](#)

Account

[Login](#)

CVE id (eg: CVE-2021-44228)

Search

Products - Product names starting with "All"

Product name

Search

Browse product names starting with:

All

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

5

6

7

8

9

@

147526 products found

Product Name	Vendor Name	Vulnerabilities	Products Type
*	amd	1	Hardware
*	intel	1	Hardware
-	-	2	Operating System
-	intel	1	Hardware
-	azure-iot-cli-extension	1	Application
-	qualcomm	1	Hardware
-ma5200g	huawei	1	Hardware
.net	microsoft	49	Application
net agent	newrelic	1	Application

图 2-13 Products查询展示图

A. 提供 `/api/products` 接口，接收 `product` 参数查询对应的厂商：

代码块

```
1 const url = new URL("/api/products", window.location.origin);
```

代码块

```
1 func GetProducts(db *gorm.DB) gin.HandlerFunc {
2     return func(c *gin.Context) {
3         page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
4         limit := 20
5         offset := (page - 1) * limit
6
7         search := c.Query("search")
8         letter := c.Query("letter")
9
10        var condition string
11        var args []interface{}
12
13        if search != "" {
14            condition = "product LIKE ?"
15            args = append(args, "%"+search+"%")
16        } else if letter != "" {
17            condition = "product LIKE ?"
18            args = append(args, letter+"%")
19        } else {
20            condition = "1=1"
```

```
21     }
22
23     countSQL := fmt.Sprintf(`
24         SELECT COUNT(*) FROM (
25             SELECT DISTINCT vendor, product
26             FROM cpe_entries
27             WHERE %s
28         ) AS sub
29     `, condition)
30
31     var total int
32     if err := db.Raw(countSQL, args...).Row().Scan(&total); err != nil {
33         c.JSON(500, gin.H{"error": "count error"})
34         return
35     }
36
37     dataSQL := fmt.Sprintf(`
38         SELECT
39             CASE part
40                 WHEN 'a' THEN 'Application'
41                 WHEN 'o' THEN 'Operating System'
42                 WHEN 'h' THEN 'Hardware'
43                 ELSE 'Unknown'
44             END AS part_type,
45             vendor, product,
46             COUNT(DISTINCT cve_id) AS vulnerability_count
47         FROM cpe_entries
48         WHERE %s
49         GROUP BY part_type, vendor, product
50         ORDER BY product
51         LIMIT ? OFFSET ?`, condition)
52
53     argsWithPage := append(args, limit, offset)
54     rows, err := db.Raw(dataSQL, argsWithPage...).Rows()
55     if err != nil {
56         c.JSON(500, gin.H{"error": "query error"})
57         return
58     }
59     defer rows.Close()
60
61     var products []ProductEntry
62     for rows.Next() {
63         var p ProductEntry
64         if err := rows.Scan(&p.ProductType, &p.VendorName, &p.ProductName,
65             &p.VulnCount); err == nil {
66             products = append(products, p)
```

```

67     }
68
69     c.JSON(200, ProductResponse{
70         Products:    products,
71         Total:       total,
72         TotalPages:  int(math.Ceil(float64(total) / float64(limit))),
73     })
74 }
75 }

```

CVE database

Vulnerabilities

[By Date](#)
[By type](#)
[CVSS Scores](#)
[Search](#)

Vulnerable Software

[Vendors](#)
[Products](#)
[Version Search](#)

Account

[Login](#)

CVE id (eg: CVE-2021-44228)

Search

Products - Product names starting with "All"

Product name

Search

Browse product names starting with:

All

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

5

6

7

8

9

@

147526 products found

Product Name	Vendor Name	Vulnerabilities	Products Type
*	amd	1	Hardware
*	intel	1	Hardware
-	-	2	Operating System
-	intel	1	Hardware
-	azure-iot-cli-extension	1	Application
-	qualcomm	1	Hardware
-ma5200g	huawei	1	Hardware
.net	microsoft	49	Application

图 2-14 Products对应的Vendor展示图

B. 接受 vendor、product 参数查抄对应的CVE

代码块

```

1  data.products.forEach(p => {
2      const cveLink = `/product_vulnerabilities_details.html?
  vendor=${encodeURIComponent(p.vendor)}&product=${encodeURIComponent(p.product)}
  `;
3      const row = `<tr>
4          <td>${p.product}</td>
5          <td>${p.vendor}</td>
6          <td><a href="${cveLink}"
  target="_blank">${p.vulnerability_count}</a></td>
7          <td>${p.part_type}</td>
8      </tr>`;
9      tbody.insertAdjacentHTML("beforeend", row);
10 }

```

```
1 代码块 const response = await fetch(`/api/products/cveid?
    vendor=${encodeURIComponent(vendor)}&product=${encodeURIComponent(product)}`);
```

代码块

```
1 // GET /api/vendors/:vendor/products?page=1&search=http
2 func GetProductsByVendor(db *gorm.DB) gin.HandlerFunc {
3     return func(c *gin.Context) {
4         vendor := c.Param("vendor") // URL 中的原始字符串 (已
// URLDecode)
5         search := c.Query("search") // 可选搜索
6         page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
7         limit := 20
8         offset := (page-1)*limit
9
10        condition := "vendor = ?"
11        args := []interface{}{vendor}
12
13        if search != "" {
14            condition += " AND LOWER(product) LIKE ?"
15            args = append(args, "%"+strings.ToLower(search)+"%")
16        }
17
18        // 计算总数
19        var total int
20        countSQL := fmt.Sprintf(`SELECT COUNT(DISTINCT product) FROM
cpe_entries WHERE %s`, condition)
21        if err := db.Raw(countSQL, args...).Row().Scan(&total); err != nil {
22            c.JSON(500, gin.H{"error": "count error"})
23            return
24        }
25
26        // 分页查询
27        argsWithPage := append(args, limit, offset)
28        rows, err := db.Raw(fmt.Sprintf(`
29            SELECT vendor, product, COUNT(DISTINCT cve_id) AS vuln_count
30            FROM cpe_entries
31            WHERE %s
32            GROUP BY product, vendor
33            ORDER BY product
34            LIMIT ? OFFSET ?`, condition), argsWithPage...).Rows()
35        if err != nil {
36            c.JSON(500, gin.H{"error": "query error"})
37            return
38        }
39        defer rows.Close()
```

```

40
41     var list []ProductEntry
42     for rows.Next() {
43         var p ProductEntry
44         if err := rows.Scan(&p.VendorName, &p.ProductName, &p.VulnCount);
err == nil {
45             list = append(list, p)
46         }
47     }
48
49     c.JSON(200, ProductResponse{
50         Products: list,
51         Total:     total,
52         TotalPages: int(math.Ceil(float64(total)/float64(limit))),
53     })
54 }
55 }
56

```

CVE database

Vulnerabilities

[By Date](#)
[By type](#)
[CVSS Scores](#)
[Search](#)

Vulnerable Software

[Vendors](#)
[Products](#)
[Version Search](#)

Account

[Login](#)

Products - Product names starting with "All"

Browse product names starting with:

All

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

0

1

2

3

4

5

6

7

8

9

@

147526 products found

Product Name	Vendor Name	Vulnerabilities	Products Type
*	amd	1	Hardware
*	intel	1	Hardware
-	-	2	Operating System
-	intel	1	Hardware
-	azure-iot-cli-extension	1	Application
-	qualcomm	1	Hardware
-ma5200g	huawei	1	Hardware
.net	microsoft	49	Application

图 2-15 Products对应的Vulnerabilities展示图

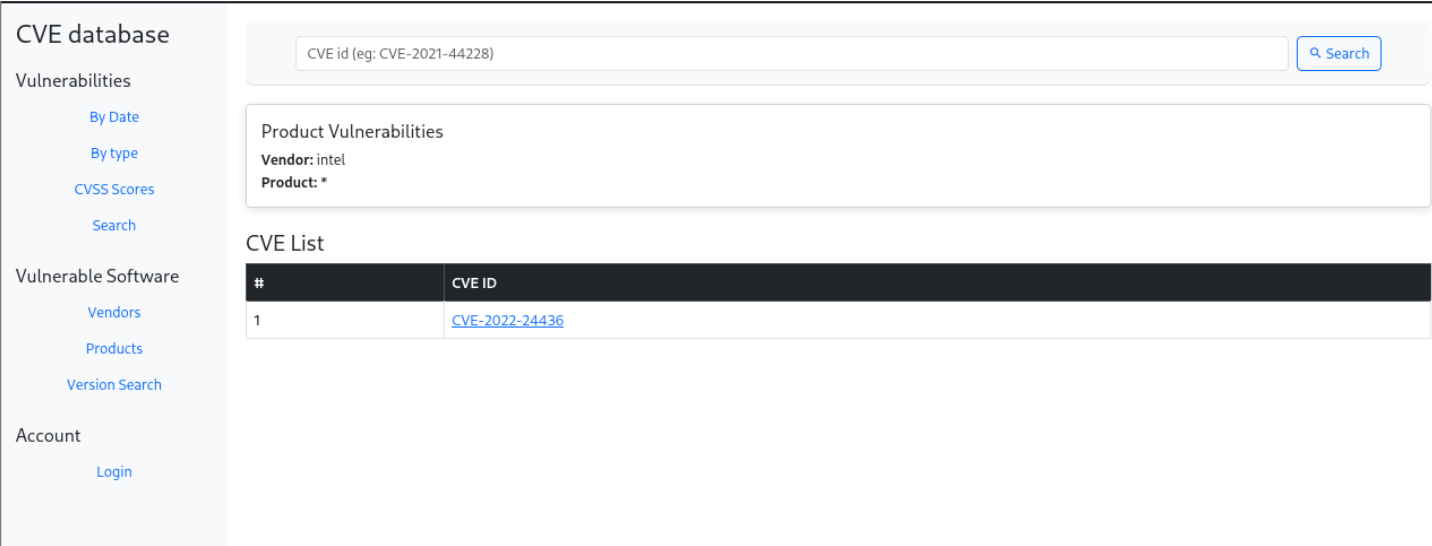


图 2-16 具体的Vulnerabilities查询展示图

C. 提供建议功能

代码块

```
1  fetch("/api/products/suggest?keyword=" + encodeURIComponent(keyword))
```

代码块

```
1  func ProductSuggest(db *gorm.DB) gin.HandlerFunc {
2      return func(c *gin.Context) {
3          keyword := c.Query("keyword")
4          if keyword == "" {
5              c.JSON(http.StatusOK, gin.H{"suggestions": []string{}})
6              return
7          }
8
9          var suggestions []string
10         err := db.Raw(`
11             SELECT DISTINCT product
12             FROM cpe_entries
13             WHERE LOWER(TRIM(product)) LIKE LOWER(?)
14             ORDER BY product
15             LIMIT 10
16         `, keyword+"%").Scan(&suggestions).Error
17
18         if err != nil {
19             c.JSON(500, gin.H{"error": "suggest query error"})
20             return
21         }
22
23         c.JSON(200, gin.H{"suggestions": suggestions})
24     }
```

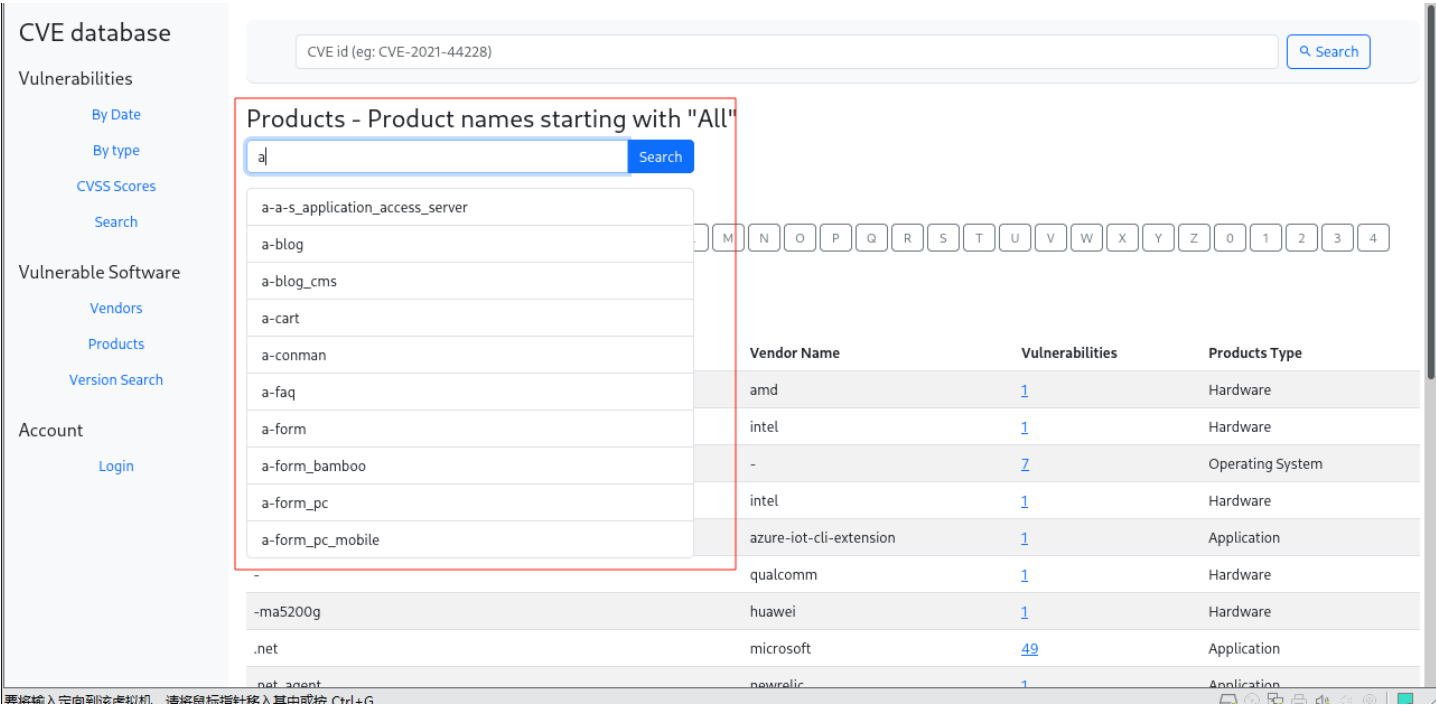


图 2-17 Vendors查询建议展示图

三、高级查询功能实现

为了满足安全研究人员和运维人员对漏洞数据的复杂筛选需求，系统提供了基于多种维度的 高级搜索功能。该功能允许用户根据 CVSS 分数、攻击向量、厂商、产品、发布时间、更新时间、漏洞类型等多个条件组合查询 CVE 条目，并返回匹配的 CVE ID 列表供进一步查看详情。

① 功能描述

功能项	描述
查询方式	支持 GET 请求传参，参数为可选组合
返回内容	匹配的 CVE ID 列表（字符串数组）
排序方式	按 CVE 发布时间倒序排列
查询粒度	支持模糊匹配、区间查询、枚举筛选等
使用场景	安全分析人员按漏洞特征批量筛选；开发人员查找特定组件相关的高危漏洞

表 2-4 高级查询特点表

② 技术实现流程

为了实现多维度的高级漏洞搜索功能，系统设计了一整套基于 RESTful API 的查询流程，前后端协作完成复杂搜索条件的解析、数据过滤及结果展示。整体技术流程如下：

当用户在前端界面中填写多个筛选条件（如 CVSS 分数区间、攻击向量、CWE 类型、影响组件关键词等）后，前端会将这些条件通过 AJAX 请求方式发送至后端定义的 `/api/cve/search` 等接口。后端接收到请求后，首先对查询参数进行解析，并动态构建 SQL 查询中的 `WHERE` 子句。

在构造查询逻辑时，系统会联合多张核心数据表进行过滤操作，涉及的主要表包括 `CVEEntry`（基本信息）、`Metric`（CVSS 评分）、`CPE`（影响组件）、`Weakness`（漏洞类型）、`ParsedVector`（攻击路径解析）和 `Category`（分类标签）。通过这些子表的关联查询，系统能够实现对复杂条件的精确筛选。

查询结果经过去重处理，并按照漏洞的发布日期进行排序，最终返回符合条件的 CVE ID 列表。前端根据该结果动态生成表格或卡片式展示，用户可点击任一条目跳转至漏洞详情页，查看该 CVE 的完整结构化信息。

该流程不仅支持多字段联合过滤，还具备良好的扩展性，可后续支持分页加载、关键词联想、模糊搜索等高级功能，提升用户检索效率和交互体验。

③ 数据库交互方式

系统使用 GORM 构建原始 SQL 查询语句，动态拼接 `WHERE` 子句，以支持灵活的组合查询。主要涉及以下几张表：

- `cveEntry`：主表，记录基本漏洞信息
- `metric`：CVSS 评分信息
- `cpe_entries`：受影响组件信息
- `weakness`：关联 CWE 弱点
- `parsedVector`：CVSS 向量拆解字段
- `category_lookup` / `cve_category_match`：漏洞分类信息

代码块

```
1  SELECT DISTINCT ce.id, ce.published
2  FROM cveEntry ce
3  LEFT JOIN metric m ON ce.id = m.cve_id
4  LEFT JOIN cpe_entries cpe ON ce.id = cpe.cve_id
5  LEFT JOIN weakness w ON ce.id = w.cve_id
6  LEFT JOIN parsedVector v ON ce.id = v.cve_id
7  LEFT JOIN cve_category_match ccm ON ce.id = ccm.cve_id
8  WHERE 1=1
9      AND cpe.vendor LIKE ?
10     AND m.baseScore >= ?
11     AND v.AV = ?
12  ORDER BY ce.published DESC
```


实际语句由 `buildConditionSQL()` 函数动态生成，确保安全性与灵活性。

代码块

```
1 func buildMultiWildcardSQL(col, raw string, args *[]interface{}) string {
2     parts := strings.Split(raw, ",")
3     var conds []string
4     for _, p := range parts {
5         p = strings.TrimSpace(p)
6         if p == "" {
7             continue
8         }
9         lowerP := strings.ToLower(p)
10        if strings.ContainsRune(p, '*') {
11            conds = append(conds, fmt.Sprintf("LOWER(%s) LIKE ?", col))
12            *args = append(*args, strings.ReplaceAll(lowerP, "*", "%"))
13        } else {
14            conds = append(conds, fmt.Sprintf("LOWER(%s) = ?", col))
15            *args = append(*args, lowerP)
16        }
17    }
18    return "(" + strings.Join(conds, " OR ") + ")"
19 }
```

④ 关键代码解析

参数绑定与校验

代码块

```
1 var params AdvancedSearchParams
2 if err := c.ShouldBindQuery(&params); err != nil {
3     c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
4 }
```

使用 Gin 自动绑定 URL 查询参数到结构体中，并进行类型转换。

1. 构建 WHERE 子句

代码块

```
1 conditionSQL, args, err := buildConditionSQL(&params)
```

2. 执行查询并返回结果

```
1 var rows []idWithPublished
2 db.Raw(dataSQL, args...).Scan(&rows)
```

3. 多值通配符处理函数

代码块

```
1 func buildMultiWildcardSQL(col, raw string, args *[]interface{}) string
```

支持逗号分隔的多值输入（如 `apache,nginx`），自动转换为多个 `OR` 条件，并支持通配符（`*`）。

⑤ 请求示例

请求地址：

代码块

```
1 https://tidb-webapp.local/search_detail.html?
  vendor=apache&product=&cwe_id=&cvss_min=8.0&cvss_max=9.0&publish_date_start=&pu
  blish_date_end=&update_date_start=&update_date_end=&attack_vector=N&attack_comp
  lexity=&authentication=S&confidentiality_impact=&integrity_impact=&availability
  _impact=
```

返回结构：

代码块

```
1 0 "CVE-2022-33140"
2 1 "CVE-2022-24289"
3 2 "CVE-2022-24288"
4 3 "CVE-2021-44832"
5 4 "CVE-2021-44521"
6 5 "CVE-2021-43999"
7 6 "CVE-2021-43083"
8 7 "CVE-2021-41971"
9 8 "CVE-2021-39236"
10 9 "CVE-2021-39232"
11 10 "CVE-2021-33036"
```

⑥ 效果展示

查询页面：

CVE id (eg: CVE-2021-44228)

Search

Advanced Vulnerability Search

Basic Information

Vendor

e.g. Google or Goog* or Google, Microsoft

Product

e.g. Android or Office* or Android, Http Server

CWE Id

e.g. 89. Exact match

CVSS Score Range

Min

to

Max

Dates

Publish Date Range

yyyy/mm/dd

to

yyyy/mm/dd

LastModify Date Range

yyyy/mm/dd

to

yyyy/mm/dd

Vulnerability Characteristics

Vulnerability Categories

图 2-18 高级查询展示图

查询结果：

CVE id (eg: CVE-2021-44228)

Search

Advanced Vulnerability Search

CVE ID	Describe	CVSS Scores	Release Date	Operate
CVE-2016-0714	The session-persistence implementation in Apache Tomcat 6.x befor...	8.8	2016/2/25	详情
CVE-2021-27644	In Apache DolphinScheduler before 1.3.6 versions, authorized users ...	8.8	2021/11/1	详情
CVE-2021-43083	Apache PLC4X - PLC4C (Only the C language implementation was ef...	8.8	2021/12/19	详情
CVE-2021-33036	In Apache Hadoop 2.2.0 to 2.10.1, 3.0.0-alpha1 to 3.1.4, 3.2.0 to 3.2.2,...	9	2022/6/15	详情
CVE-2016-3090	The TextParseUtil.translateVariables method in Apache Struts 2.x be...	8.8	2017/10/30	详情
CVE-2017-12636	CouchDB administrative users can configure the database server via ...	9	2017/11/14	详情
CVE-2018-11778	UnixAuthenticationService in Apache Ranger 1.2.0 was updated to co...	8.8	2018/10/5	详情
CVE-2016-5397	The Apache Thrift Go client library exposed the potential during code...	9	2018/2/12	详情
CVE-2020-11977	In Apache Syncope 2.1.X releases prior to 2.1.7, when the Flowable ex...	8.5	2020/9/15	详情
CVE-2021-39236	In Apache Ozone before 1.2.0, Authenticated users with valid Ozone ...	8.8	2021/11/19	详情
CVE-2016-4461	Apache Struts 2.x before 2.3.29 allows remote attackers to execute a...	9	2017/10/16	详情
CVE-2018-8028	An authenticated user can execute ALTER TABLE EXCHANGE PARTIT...	8.8	2018/8/23	详情
CVE-2018-11769	CouchDB administrative users before 2.2.0 can configure the databa...	9	2018/8/8	详情

图 2-19 高级查询结果图

2.4.3 权限控制模块实现（基于 JWT 的鉴权机制）

为了保障系统的数据安全性与接口访问可控性，本系统实现了基于 JWT（JSON Web Token）的无状态身份认证与接口级权限控制机制。该机制不仅支持用户登录、登出等基本功能，还通过中间件方式对敏感 API 进行统一鉴权管理，确保不同角色用户仅能访问其授权范围内的资源。

一、JWT 鉴权机制概述

本系统采用 JWT（JSON Web Token）作为用户身份验证与授权的核心机制。JWT 是一种开放标准（RFC 7519），用于在网络应用之间安全、简洁地传输用户身份信息。系统通过 JWT 实现用户身份验证（Authentication），确保登录用户的合法性；同时用于接口访问授权（Authorization），根据令牌中的权限信息控制资源访问；此外，JWT 支持无状态的会话管理（Session Management），无需在服务器端存储用户会话数据，提升系统扩展性；该机制还具备良好的多端兼容性，适用于 Web 前端、移动端应用及第三方 API 调用等多种使用场景。

核心流程如下：

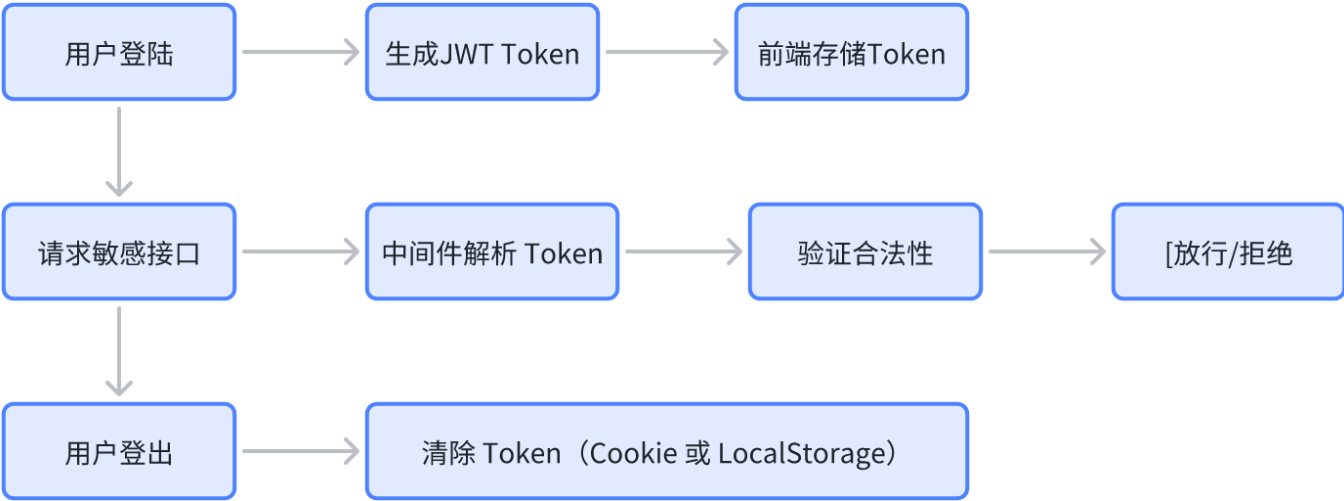


图 2-20 权限控制流程图

二、技术实现说明

1. JWT Token 生成

在用户成功登录后，系统使用 `github.com/golang-jwt/jwt/v5` 库生成签名 Token，并包含以下关键字段：

字段名	类型	含义
user_id	float64	用户唯一标识（uint 转换为 float64）
email	string	用户邮箱地址
username	string	用户名
exp	int64	Token 过期时间（Unix 时间戳）

表 2-5 JWT Token 字段表

使用 HMAC-SHA256 签名算法，密钥由环境变量 `JWT_SECRET` 控制，默认值 `"replace-with-32-random-bytes"` 仅用于开发测试，生产环境需替换。

代码块

```
1 func GenerateJWT(userID uint, email, username string) (string, error) {
2     token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
3         "user_id": float64(userID),
4         "email":    email,
5         "username": username,
6         "exp":     time.Now().Add(24 * time.Hour).Unix(), // 默认有效期 24 小时
```

```
7     })
8     return token.SignedString(config.JwtSecret)
9 }
```

2. Token 解析与校验

系统通过 `ParseJWT()` 函数对传入的 Token 进行解析与合法性判断，包括：

- 签名是否有效；
- Token 是否过期；
- Claims 中是否包含必要字段（如 `user_id`, `email`, `username`）；

代码块

```
1 func ParseJWT(tokenStr string) (jwt.MapClaims, error) {
2     token, err := jwt.Parse(tokenStr, func(token *jwt.Token) (interface{},
3         error) {
4         // 只接受 HMAC 签名方法
5         if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
6             return nil, fmt.Errorf("unexpected signing method: %v",
7                 token.Header["alg"])
8         }
9         return config.JwtSecret, nil
10    })
11
12    if err != nil || !token.Valid {
13        return nil, fmt.Errorf("invalid token: %v", err)
14    }
15
16    claims, ok := token.Claims.(jwt.MapClaims)
17    if !ok {
18        return nil, fmt.Errorf("invalid claims type")
19    }
20
21    // 验证必要字段
22    if _, ok := claims["user_id"]; !ok {
23        return nil, fmt.Errorf("missing user_id in token")
24    }
25
26    if _, ok := claims["username"]; !ok {
27        return nil, fmt.Errorf("missing username in token")
28    }
29
30    if _, ok := claims["email"]; !ok {
31        return nil, fmt.Errorf("missing email in token")
32    }
33 }
```

```
31
32     return claims, nil
33 }
```

3. Gin 中间件实现

为了确保系统中受保护的 API 接口具备统一且安全的访问控制机制，本系统基于 JWT（JSON Web Token）认证机制，在 Gin 框架中自定义了 `JWTMiddleware()` 中间件，用于全局接入所有敏感路由。

该中间件支持从两种常见来源中获取令牌：一是 HTTP 请求头中的 `Authorization: Bearer <token>` 字段，二是浏览器自动携带的 Cookie 值。中间件会自动判断 Token 的存在与位置，并提取出有效载荷。

接收到 Token 后，`JWTMiddleware()` 会对其进行格式校验、签名验证及过期时间检查，确保 Token 的合法性与时效性。一旦验证通过，系统会从 Token 中解析出用户 ID、角色权限等关键信息，并将其注入到 Gin 的上下文对象中（`c *gin.Context`），以供后续的业务处理逻辑读取和判断。

如果 Token 校验失败（例如签名错误、Token 过期或格式不符），中间件将立即拦截请求并返回 HTTP 403 Unauthorized 响应，提示用户认证未通过，从而有效防止未授权访问或伪造身份请求。通过这一中间件机制，系统实现了 API 层面的统一鉴权，不仅提高了安全性，还增强了代码模块的复用性和可维护性。

代码块

```
1 func JWTMiddleware() gin.HandlerFunc {
2     return func(c *gin.Context) {
3         // 尝试从 Authorization Header 获取
4         authHeader := c.GetHeader("Authorization")
5         var tokenStr string
6
7         if strings.HasPrefix(authHeader, "Bearer ") {
8             tokenStr = strings.TrimPrefix(authHeader, "Bearer ")
9             log.Printf("Token from Header: %s", tokenStr)
10        } else {
11            cookieToken, err := c.Cookie("token")
12            if err != nil || cookieToken == "" {
13                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
14                    "Missing or invalid token"})
15                return
16            }
17            tokenStr = cookieToken
18            log.Printf("Token from Cookie: %s", tokenStr)
19        }
20        claims, err := ParseJWT(tokenStr)
```

```

21         if err != nil {
22             log.Printf("JWT Parse Error: %v", err)
23             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"Invalid token"})
24             return
25         }
26
27         // 类型断言
28         userIDFloat, ok := claims["user_id"].(float64)
29         if !ok {
30             log.Printf("Invalid user_id type in token")
31             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"invalid user_id in token"})
32             return
33         }
34         userID := uint(userIDFloat)
35
36         email, ok := claims["email"].(string)
37         if !ok {
38             log.Printf("Invalid email type in token")
39             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"invalid email in token"})
40             return
41         }
42         username, ok := claims["username"].(string)
43         if !ok {
44             log.Printf("Invalid username type in token")
45             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"invalid username in token"})
46             return
47         }
48
49         // 设置上下文变量
50         c.Set("username", username)
51         c.Set("user_id", userID)
52         c.Set("email", email)
53
54         c.Next()
55     }
56 }

```

4. 受保护接口示例

通过中间件保护的接口可直接从上下文中读取用户信息，例如：

代码块


```

1 func ProtectedHandler(c *gin.Context) {
2     userIDVal, _ := c.Get("user_id")
3     emailVal, _ := c.Get("email")
4     usernameVal, _ := c.Get("username")
5
6     if userIDVal == nil || emailVal == nil {
7         c.JSON(401, gin.H{"error": "unauthorized"})
8         return
9     }
10
11     c.JSON(200, gin.H{
12         "user_id": userIDVal,
13         "email": emailVal,
14         "username": usernameVal,
15         "status": "token valid",
16     })
17 }

```

5. 登出功能实现

由于 JWT 是无状态的，无法像 Session 一样主动销毁。因此，系统通过设置 Cookie 过期时间为过去时间来实现登出逻辑：

代码块

```

1 func LogoutHandler(c *gin.Context) {
2     domain := ""
3     if strings.Contains(c.Request.Host, "tidb-webapp.local") {
4         domain = "tidb-webapp.local"
5     }
6     secure := domain != ""
7
8     // 清除 token cookie
9     c.SetCookie("token", "", -1, "/", domain, secure, true)
10
11     c.JSON(http.StatusOK, gin.H{
12         "message": "已成功登出",
13     })
14 }

```

三、优势与特点总结

特点	描述
无状态鉴权	不依赖服务器端 Session 存储，适合分布式部署
安全性强	使用签名机制防止篡改，配合 HTTPS 加密通信
易于扩展	可在 Token 中嵌入角色、权限等信息，用于细粒度控制
多种 Token 获取方式	支持 Header 和 Cookie，适配前后端分离架构
可审计性强	所有用户操作均可记录 user_id，便于日志追踪
前端友好	支持多种客户端调用方式，如 Axios、Fetch、Postman 等

表 2-6 JWT鉴权优势表

2.4.4 统计图表与可视化实现（如使用 ECharts 或 Chart.js）

为了帮助用户更直观地理解漏洞数据的趋势与分布情况，系统集成了基于 ECharts 的可视化模块，支持对 CVSS 评分（基础分、影响分、利用分）等进行多维度统计展示，在这里只是举其中的一个例子。该模块通过后端聚合查询生成 JSON 数据，并由前端异步请求渲染为交互式图表，提升了用户体验和数据分析效率。

一、功能描述

本模块主要提供以下的可视化图表：

1	图表名称	图表/可视化数据实现的功能	api接口	调用的函数	显示的页面	描述
2	Total	获取 CVE 总数	/api/stats	getStats	/index.html	获取 CVE 总数
3	High risk script	获取 CVE 中严重（CVSS 分数 >= 9）数量	/api/stats	getStats	/index.html	获取 CVE 中严重（CVSS 分数 >= 9）数量
4	Known to exploit	获取有 CISA 行动到期的 CVE 数量	/api/stats	getStats	/index.html	获取有 CISA 行动到期的 CVE 数量
5	Bug fixed	获取有 CISA 行动到期的 CVE 数量	/api/stats	getStats	/index.html	获取有 CISA 行动到期的 CVE 数量
6	Vulnerabilities distributed by CVSS	Base Score 分	/api/base-score-	getBaseScoreDistr	/index.html	衡量漏洞整体严重性，

6	Vulnerabilities distributed by CVSS score	Impact Score 分布	/api/impact-score-distribution	getimpactScoreDistribution	/index.html	反映其潜在危害程度
7	Vulnerabilities distributed by CVSS score	Impact Score 分布	/api/impact-score-distribution	getimpactScoreDistribution	/index.html	反映漏洞对机密性、完整性、可用性的破坏程度
8	Vulnerabilities distributed by CVSS score	Exploitability Score 分布	/api/exploitability-score-distribution	getexploitabilityScoreDistribution	/index.html	衡量攻击者成功利用该漏洞的难易程度
9	Vulnerabilities by type and year	查看漏洞类型与年份之间关系	/api/type-year-stats	getTypeYearStats	/index.html	展现不同年份中各类漏洞数量变化趋势
10	Vulnerabilities by type and number	查看不同漏洞之间的比例	/api/vulnerabilities-by-type	getvulnerabilitiesbytype	/index.html	体现不同漏洞类型的占比情况
11	Vulnerabilities By Types/Categories (1999-2024)	查看年份和漏洞类别的数量关系	/vulnerabilities-type	vulnerabilityTypeHandler(db)	/vulnerabilities.html	揭示年份与漏洞类别之间的关联性
12	Vulnerabilities by Impact Types (1999-2024)	查看年份和漏洞影响类型的数量关系	/vulnerabilities	vulnerabilitiesHandler(db)	/vulnerabilities.html	揭示年份与漏洞影响类型之间的关系
13	CVSS Score Range - Bar Chart	查看特定时间段 CVSS 的分布情况	/api/cvss_cves?range=\${range}&start=\${start}&end=\${end}	getCVEsBySeverityRange	/cvss_score.html	查看特定时间段 CVSS 的分布情况
14	CVSS Distribution Bubble Chart	查看特定时间段 CVSS 的分布情况	/api/cvss_cves?range=\${range}&start=\${start}&end=\${end}	getCVEsBySeverityRange	/cvss_score.html	查看特定时间段 CVSS 的分布情况

表 2-7 统计与可视化实现的api接口对应表

二、技术实现流程

1. Dashboard

前端展示：

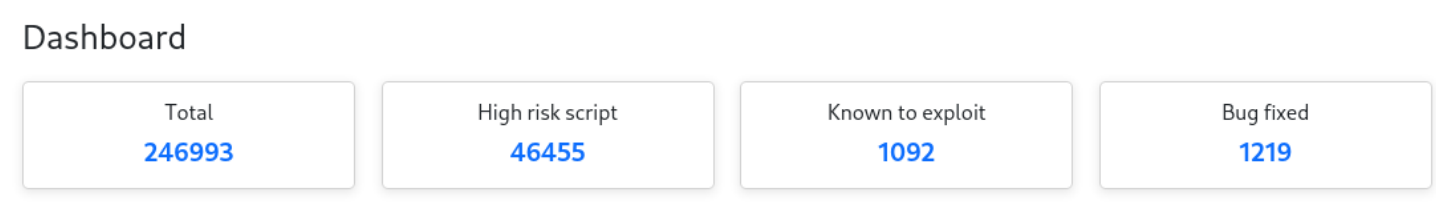


图 2.20 Dashboard展示图

前端源码：

代码块

```
1  <div class="row mt-4">
2    <div class="col-md-3">
3      <div class="card stat-card">
4        <h5>Total</h5>
5        <a href="/cve_total_details.html" class="text-decoration-none">
6          <div id="total" class="stat-value" style="cursor:
loading...">loading...</div>
7        </a>
8      </div>
9    </div>
10   <div class="col-md-3">
11     <div class="card stat-card">
12       <h5>High risk script</h5>
13       <a href="/cve_high_risk_details.html" class="text-decoration-none">
14         <div id="critical" class="stat-value">loading...</div>
15       </a>
16     </div>
17   </div>
18   <div class="col-md-3">
19     <div class="card stat-card">
20       <h5>Known to exploit</h5>
21       <a href="/cve_known_to_exploit_details.html" class="text-
decoration-none">
22         <div id="exploitable" class="stat-value">loading...</div>
23       </a>
24     </div>
25   </div>
26   <div class="col-md-3">
27     <div class="card stat-card">
28       <h5>Bug fixed</h5>
29       <a href="/cve_bug_fixed_details.html" class="text-decoration-none">
30         <div id="patched" class="stat-value">loading...</div>
31       </a>
32     </div>
33   </div>
34 </div>
```

代码块

```
1  <script>
2    // 获取统计数据
3    fetch('/api/stats')
4      .then(response => response.json())
```

```

5      .then(data => {
6          document.getElementById('total').innerText = data.total;
7          document.getElementById('critical').innerText = data.critical;
8          document.getElementById('exploitable').innerText =
data.exploitable;
9          document.getElementById('patched').innerText = data.patched;
10     })
11     .catch(error => console.error('Failed to load statistics', error));
12 </script>

```

后端源码：

代码块

```

1  // API 处理函数
2  func getStats(c *gin.Context) {
3      var stats struct {
4          Total      int64 `json:"total"`
5          Critical   int64 `json:"critical"`
6          Exploitable int64 `json:"exploitable"`
7          Patched    int64 `json:"patched"`
8      }
9
10     // 获取 CVE 总数
11     db.Table("CVEEntry").Count(&stats.Total)
12     // 获取 CVE 中严重 (CVSS 分数 >= 9) 数量
13     db.Table("Metric").Where("baseScore >= 9.0").Count(&stats.Critical)
14     // 获取有 CISA 行动到期的 CVE 数量
15     db.Table("CVEEntry").Where("cisaActionDue IS NOT
NULL").Count(&stats.Exploitable)
16     // 获取已修复的 CVE 数量
17     db.Table("CVEEntry").Where("evaluatorSolution IS NOT
NULL").Count(&stats.Patched)
18
19     c.JSON(http.StatusOK, stats)
20 }

```

2. Vulnerabilities distributed by CVSS score

前端展示：

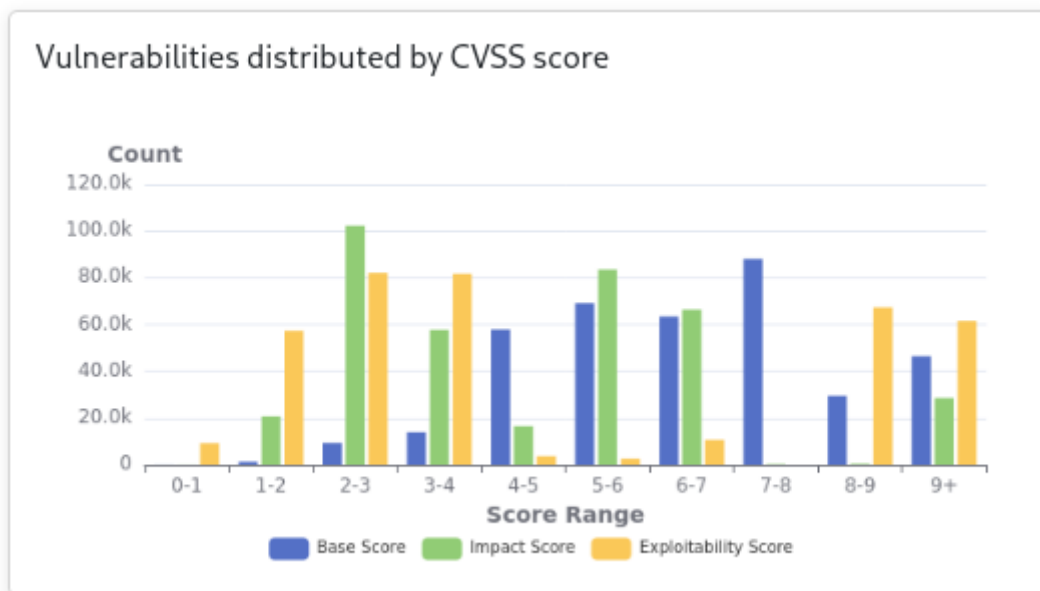


图 2-21 CVSS score直方图

前端源码:

代码块

```
1 <div class="col-md-6">
2   <div class="card p-3">
3     <h5>Vulnerabilities distributed by CVSS score</h5>
4     <div id="score-chart" style="height: 300px;"></div>
5   </div>
6 </div>
```

代码块

```
1 <script>
2   document.addEventListener('DOMContentLoaded', async function () {
3     const scoreChart = echarts.init(document.getElementById('score-
4 chart'));
5     // 并发请求三个 API
6     const [baseRes, impactRes, exploitRes] = await Promise.all([
7       fetch('/api/base-score-distribution'),
8       fetch('/api/impact-score-distribution'),
9       fetch('/api/exploitability-score-distribution'),
10    ]);
11    const [baseData, impactData, exploitData] = await Promise.all([
12      baseRes.json(),
13      impactRes.json(),
14      exploitRes.json(),
15    ]);
16    // 提取 Y 轴数据 (按顺序)
```

```

16         const xLabels = ['0-1', '1-2', '2-3', '3-4', '4-5', '5-6', '6-7', '7-
17         8', '8-9', '9+'];
18         const getCounts = (data) => xLabels.map(label => {
19             const item = data.find(d => d.score_range === label);
20             return item ? item.score_count : 0;
21         });
22         const baseCounts = getCounts(baseData);
23         const impactCounts = getCounts(impactData);
24         const exploitCounts = getCounts(exploitData);
25     });
26 });
27 </script>

```

后端源码：

代码块

```

1  func getBaseScoreDistribution(c *gin.Context) {
2      // 定义用于接收查询结果的数据结构
3      var results []struct {
4          ScoreRange string `json:"score_range"`
5          ScoreCount int64  `json:"score_count"`
6      }
7
8      // 执行 SQL 查询
9      err := db.Raw(`
10          SELECT '0-1' AS score_range, COUNT(*) AS score_count FROM metric WHERE
11          baseScore >= 0 AND baseScore < 1
12          UNION ALL
13          SELECT '1-2', COUNT(*) FROM metric WHERE baseScore >= 1 AND baseScore
14          < 2
15          UNION ALL
16          SELECT '2-3', COUNT(*) FROM metric WHERE baseScore >= 2 AND baseScore
17          < 3
18          UNION ALL
19          SELECT '3-4', COUNT(*) FROM metric WHERE baseScore >= 3 AND baseScore
20          < 4
21          UNION ALL
22          SELECT '4-5', COUNT(*) FROM metric WHERE baseScore >= 4 AND baseScore
23          < 5
24          UNION ALL
25          SELECT '5-6', COUNT(*) FROM metric WHERE baseScore >= 5 AND baseScore
26          < 6
27          UNION ALL
28          SELECT '6-7', COUNT(*) FROM metric WHERE baseScore >= 6 AND baseScore
29          < 7
30      `).Scan(&results)
31      if err != nil {
32          c.JSON(500, gin.H{"error": err.Error()})
33          return
34      }
35      c.JSON(200, results)
36  }

```

```

23         UNION ALL
24         SELECT '7-8', COUNT(*) FROM metric WHERE baseScore >= 7 AND baseScore
    < 8
25         UNION ALL
26         SELECT '8-9', COUNT(*) FROM metric WHERE baseScore >= 8 AND baseScore
    < 9
27         UNION ALL
28         SELECT '9+', COUNT(*) FROM metric WHERE baseScore >= 9
29     `).Scan(&results).Error
30
31     // 错误处理
32     if err != nil {
33         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
34         return
35     }
36
37     // 返回 JSON 响应
38     c.JSON(http.StatusOK, results)
39 }

```

代码块

```

1  func getImpactScoreDistribution(c *gin.Context) {
2      // 定义用于接收查询结果的数据结构
3      var results []struct {
4          ScoreRange string `json:"score_range"`
5          ScoreCount int64  `json:"score_count"`
6      }
7
8      // 执行 SQL 查询
9      err := db.Raw(`
10         SELECT '0-1' AS score_range, COUNT(*) AS score_count FROM metric WHERE
    impactScore >= 0 AND impactScore < 1
11         UNION ALL
12         SELECT '1-2', COUNT(*) FROM metric WHERE impactScore >= 1 AND
    impactScore < 2
13         UNION ALL
14         SELECT '2-3', COUNT(*) FROM metric WHERE impactScore >= 2 AND
    impactScore < 3
15         UNION ALL
16         SELECT '3-4', COUNT(*) FROM metric WHERE impactScore >= 3 AND
    impactScore < 4
17         UNION ALL
18         SELECT '4-5', COUNT(*) FROM metric WHERE impactScore >= 4 AND
    impactScore < 5
19         UNION ALL

```



```

20         SELECT '5-6', COUNT(*) FROM metric WHERE impactScore >= 5 AND
impactScore < 6
21         UNION ALL
22         SELECT '6-7', COUNT(*) FROM metric WHERE impactScore >= 6 AND
impactScore < 7
23         UNION ALL
24         SELECT '7-8', COUNT(*) FROM metric WHERE impactScore >= 7 AND
impactScore < 8
25         UNION ALL
26         SELECT '8-9', COUNT(*) FROM metric WHERE impactScore >= 8 AND
impactScore < 9
27         UNION ALL
28         SELECT '9+', COUNT(*) FROM metric WHERE impactScore >= 9
29     `).Scan(&results).Error
30
31     // 错误处理
32     if err != nil {
33         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
34         return
35     }
36
37     // 返回 JSON 响应
38     c.JSON(http.StatusOK, results)
39 }

```

代码块

```

1  func getexploitabilityScoreDistribution(c *gin.Context) {
2      // 定义用于接收查询结果的数据结构
3      var results []struct {
4          ScoreRange string `json:"score_range"`
5          ScoreCount int64  `json:"score_count"`
6      }
7
8      // 执行 SQL 查询
9      err := db.Raw(`
10         SELECT '0-1' AS score_range, COUNT(*) AS score_count FROM metric WHERE
exploitabilityScore >= 0 AND exploitabilityScore < 1
11         UNION ALL
12         SELECT '1-2', COUNT(*) FROM metric WHERE exploitabilityScore >= 1 AND
exploitabilityScore < 2
13         UNION ALL
14         SELECT '2-3', COUNT(*) FROM metric WHERE exploitabilityScore >= 2 AND
exploitabilityScore < 3
15         UNION ALL

```

```

16         SELECT '3-4', COUNT(*) FROM metric WHERE exploitabilityScore >= 3 AND
exploitabilityScore < 4
17         UNION ALL
18         SELECT '4-5', COUNT(*) FROM metric WHERE exploitabilityScore >= 4 AND
exploitabilityScore < 5
19         UNION ALL
20         SELECT '5-6', COUNT(*) FROM metric WHERE exploitabilityScore >= 5 AND
exploitabilityScore < 6
21         UNION ALL
22         SELECT '6-7', COUNT(*) FROM metric WHERE exploitabilityScore >= 6 AND
exploitabilityScore < 7
23         UNION ALL
24         SELECT '7-8', COUNT(*) FROM metric WHERE exploitabilityScore >= 7 AND
exploitabilityScore < 8
25         UNION ALL
26         SELECT '8-9', COUNT(*) FROM metric WHERE exploitabilityScore >= 8 AND
exploitabilityScore < 9
27         UNION ALL
28         SELECT '9+', COUNT(*) FROM metric WHERE exploitabilityScore >= 9
29     `).Scan(&results).Error
30
31     // 错误处理
32     if err != nil {
33         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
34         return
35     }
36
37     // 返回 JSON 响应
38     c.JSON(http.StatusOK, results)
39 }

```

3. Vulnerabilities by type and year

前端展示：

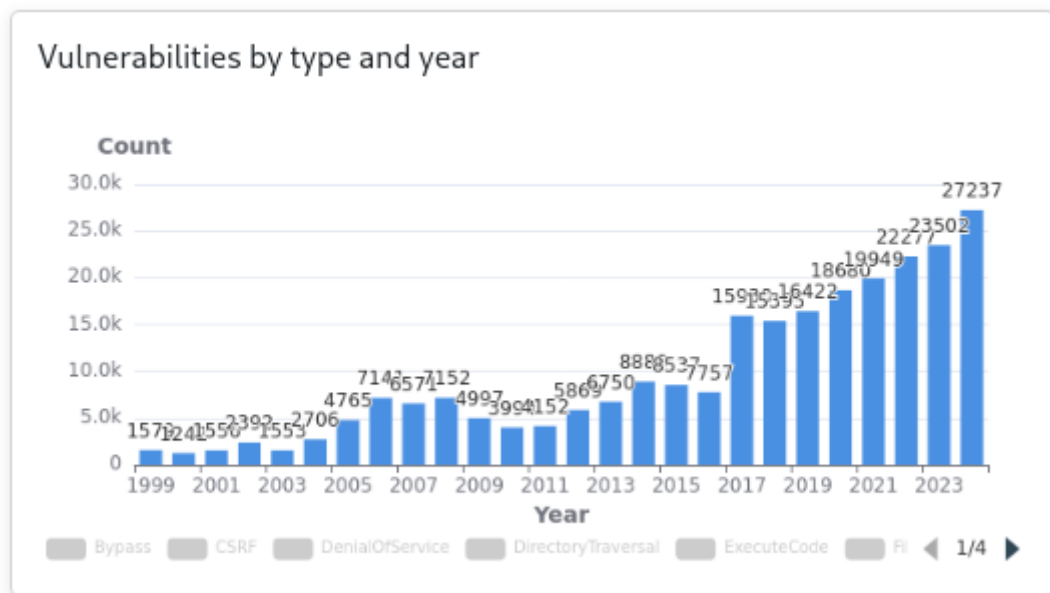


图 2-22 漏洞类型年度直方图

前端源码：

代码块

```
1 <div class="col-md-6">
2   <div class="card p-3">
3     <h5>Vulnerabilities by type and year</h5>
4     <div id="type-year-chart" style="height: 300px;"></div>
5   </div>
6 </div>
```

代码块

```
1 <script>
2   document.addEventListener('DOMContentLoaded', async function () {
3     const typeYearChart = echarts.init(document.getElementById('type-year-
4     chart'));
5     // 1. 获取真实数据
6     const response = await fetch('/api/type-year-stats');
7     const rawData = await response.json();
8
9     // 2. 预处理：构建年份列表和漏洞类型列表（去重 + 排序）
10    const yearsSet = new Set();
11    const typeSet = new Set();
12    const dataMap = {};
13
14    rawData.forEach(item => {
15      const year = String(item.year);
16      const type = item.vuln_type;
```

```

17
18     yearsSet.add(year);
19     typeSet.add(type);
20
21     // 构造映射 key: `${type}_${year}` => count
22     dataMap[`${type}_${year}`] = item.vuln_count;
23 });
24
25     const years = Array.from(yearsSet).sort();
26     const vulnerabilityTypes = Array.from(typeSet).sort();
27
28     // 3. 构建 seriesData 数组
29     const seriesData = vulnerabilityTypes.map(type => {
30         return {
31             name: type,
32             type: 'bar',
33             stack: 'total',
34             emphasis: {
35                 focus: 'series'
36             },
37             data: years.map(year => dataMap[`${type}_${year}`] || 0)
38         };
39     });
40
41     // 4. 添加 Total 系列
42     const totalData = years.map((year, index) => {
43         return seriesData.reduce((sum, series) => sum +
series.data[index], 0);
44     });
45
46     // 默认只显示 Total, 其它系列先隐藏
47     const option = typeYearChart.getOption();
48     option.legend[0].selected = { 'Total': true };
49     vulnerabilityTypes.forEach(type => {
50         option.legend[0].selected[type] = false;
51     });
52     typeYearChart.setOption(option);
53 });
54 </script>

```

后端源码:

代码块

```

1 func getTypeYearStats(c *gin.Context) {
2     var results []struct {
3         Year      int      `json:"year"`

```

```

4      VulnType  string `json:"vuln_type"`
5      VulnCount int64  `json:"vuln_count"`
6  }
7
8      err := db.Raw(`SELECT year, vuln_type, vuln_count FROM vuln_stats_view
ORDER BY year, vuln_type`).Scan(&results).Error
9      if err != nil {
10         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
11         return
12     }
13
14     c.JSON(http.StatusOK, results)
15 }

```

4. Vulnerabilities by type and number

前端展示：

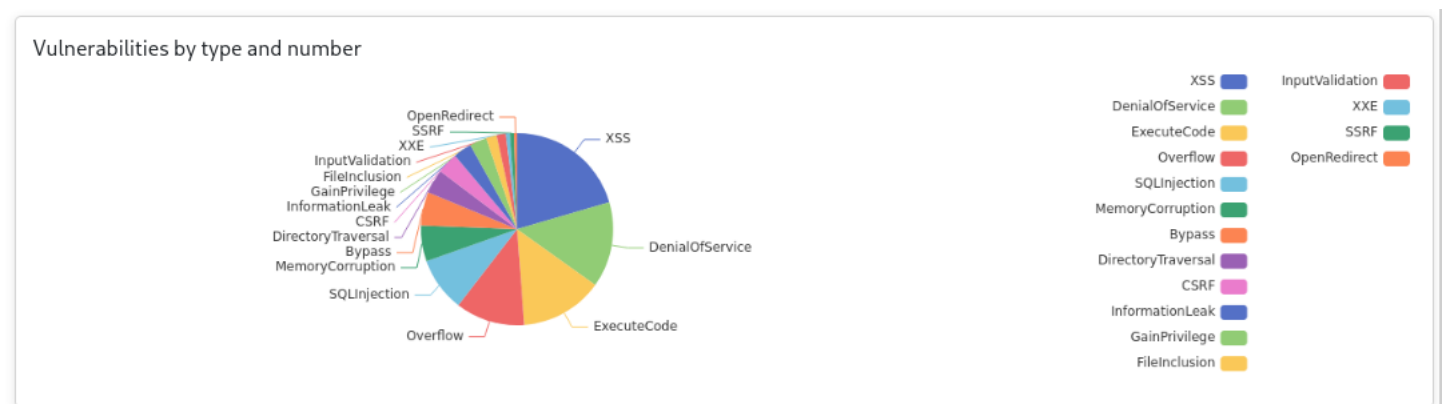


图 2-23 漏洞类型分布圆饼图

前端源码：

代码块

```

1  <div class="row mt-4">
2      <div class="col-md-12">
3          <div class="card p-3">
4              <h5>Vulnerabilities by type and number</h5>
5              <div id="vuln-type-pie" style="height: 300px;"></div>
6          </div>
7      </div>
8  </div>

```

代码块

```

1  <script>
2      // 等待文档加载完成
3      document.addEventListener('DOMContentLoaded', function () {
4          // 获取饼图容器
5          var chartDom = document.getElementById('vuln-type-pie');
6          var myChart = echarts.init(chartDom);
7
8          // 请求后端接口获取数据
9          fetch('/api/vulnerabilites-by-type')
10             .then(response => response.json())
11             .then(data => {
12                 // 处理返回的数据，转换为饼图所需的格式
13                 const pieData = data.map(item => ({
14                     name: item.vuln_type,
15                     value: item.total_count
16                 }));
17
18                 // 使用配置项生成图表
19                 myChart.setOption(option);
20             })
21             .catch(error => {
22                 console.error('Failed to load pie chart data:', error);
23             });
24     });
25 </script>

```

后端源码：

代码块

```

1  func getvulnerabilitiesbytype(c *gin.Context) {
2      // 用于接收每种类型的总数
3      var results []struct {
4          VulnType      string `json:"vuln_type"`
5          VulnTotalCount int64  `json:"total_count"`
6          Percentage     float64 `json:"percentage"`
7      }
8
9      // 先获取总漏洞数量（排除 other）
10     var total int64
11     if err := db.Raw(`
12         SELECT SUM(vuln_count)
13         FROM vuln_stats_view
14         WHERE vuln_type != 'Other'
15     `).Scan(&total).Error; err != nil {
16         c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to count
total": " + err.Error()})

```

```

17         return
18     }
19
20     // 查询每种漏洞类型的数量 (排除 other)
21     var rawResults []struct {
22         VulnType      string
23         VulnTotalCount int64
24     }
25
26     if err := db.Raw(`
27         SELECT vuln_type, SUM(vuln_count) AS vuln_total_count
28         FROM vuln_stats_view
29         WHERE vuln_type != 'Other'
30         GROUP BY vuln_type
31         ORDER BY vuln_total_count DESC
32     `).Scan(&rawResults).Error; err != nil {
33         c.JSON(http.StatusInternalServerError, gin.H{"error": "Query failed: "
+ err.Error()})
34         return
35     }
36
37     // 计算百分比并填充最终结构
38     for _, r := range rawResults {
39         percentage := float64(r.VulnTotalCount) * 100.0 / float64(total)
40         results = append(results, struct {
41             VulnType      string `json:"vuln_type"`
42             VulnTotalCount int64  `json:"total_count"`
43             Percentage     float64 `json:"percentage"`
44         }{
45             VulnType:      r.VulnType,
46             VulnTotalCount: r.VulnTotalCount,
47             Percentage:     percentage,
48         })
49     }
50
51     c.JSON(http.StatusOK, results)
52 }

```

5. Vulnerabilities By Types/Categories (1999-2024)

前端展示：

Vulnerabilities By Types/Categories (1999-2024)

Vulnerability Statistics

Show10entries

Search...

Year	Overflow	Memory Corruption	SQL Injection	XSS	Directory Traversal	File Inclusion	CSRF	XXE	SSRF	Open Redirect	Input Validation
1999	284	1	0	2	10	0	0	0	0	0	0
2000	225	0	2	5	26	0	0	0	0	0	0
2001	234	0	7	31	123	2	0	0	0	0	0
2002	402	2	46	216	115	14	2	1	0	0	2
2003	290	7	46	128	55	17	0	0	0	0	0
2004	395	25	149	302	112	38	5	0	0	0	0
2005	448	29	601	764	199	94	9	5	0	1	2
2006	582	95	998	1323	334	902	18	0	0	1	3
2007	735	127	689	848	338	637	68	0	1	2	9
2008	554	141	1490	998	459	200	114	1	0	21	7
Total	17375	17810	13881	31752	6292	2453	5508	1016	1049	739	2243

Showing 1 to 10 of 26 entries

<

1

2

3

>

表 2-8 漏洞类型年度数量表

前端源码：

代码块

```
1 <div class="card-header py-3 d-flex flex-row align-items-center justify-content-between">
2   <h6 class="m-0 font-weight-bold text-primary">
3     <i class="mdi mdi-table-large impact-icon"></i>
4     Vulnerability Statistics
5   </h6>
6   <div class="dropdown no-arrow">
7     <a class="dropdown-toggle" href="#" role="button" id="dropdownMenuLink"
8       data-bs-toggle="dropdown" aria-expanded="false">
9       <i class="mdi mdi-dots-vertical text-gray-400"></i>
10    </a>
11    <ul class="dropdown-menu dropdown-menu-end shadow animated--fade-in"
12      aria-labelledby="dropdownMenuLink">
13      <li><a class="dropdown-item" href="#"><i
14        class="mdi mdi-refresh me-2"></i>Refresh</a>
15      </li>
16      <li><a class="dropdown-item" href="#"><i
17        class="mdi mdi-printer me-2"></i>Print</a></li>
18      <div class="dropdown-divider"></div>
19      <li><a class="dropdown-item" href="#"><i
```



```

20         class="mdi mdi-cog me-2"></i>Settings</a></li>
21     </ul>
22 </div>
23 </div>
24 <div class="card-body">
25     <div class="table-responsive">
26         <table id="vulnTable1" class="table table-bordered table-hover"
width="100%"
27             cellpadding="0">
28             <thead class="thead-light">
29                 <tr>
30                     <th class="text-center">
31                         <i class="mdi mdi-calendar impact-icon text-muted">
</i>Year
32                     </th>
33                     <th class="text-center">
34                         <i class="mdi mdi-alert impact-icon text-danger">
</i>Overflow
35                     </th>
36                     <th class="text-center">
37                         <i class="mdi mdi-memory impact-icon text-danger">
</i>Memory
38                         Corruption
39                     </th>
40                     <th class="text-center">
41                         <i class="mdi mdi-database-search impact-icon text-
warning"></i>SQL
42                         Injection
43                     </th>
44                     <th class="text-center">
45                         <i
46                             class="mdi mdi-script-text-outline impact-icon
text-warning"></i>XSS
47                     </th>
48                     <th class="text-center">
49                         <i
50                             class="mdi mdi-folder-search-outline impact-icon
text-info"></i>Directory
51                         Traversal
52                     </th>
53                     <th class="text-center">
54                         <i
55                             class="mdi mdi-file-remove-outline impact-icon
text-secondary"></i>File
56                         Inclusion
57                     </th>
58                     <th class="text-center">

```

```

59         <i class="mdi mdi-shield-off impact-icon text-warning">
        </i>CSRF
60     </th>
61     <th class="text-center">
62         <i class="mdi mdi-xml impact-icon text-info"></i>XXE
63     </th>
64     <th class="text-center">
65         <i
66             class="mdi mdi-cloud-search-outline impact-icon
text-info"></i>SSRF
67     </th>
68     <th class="text-center">
69         <i
70             class="mdi mdi-link-variant-remove impact-icon
text-warning"></i>Open
71         Redirect
72     </th>
73     <th class="text-center">
74         <i
75             class="mdi mdi-form-textbox-password impact-icon
text-success"></i>Input

```

代码块

```

1 <script>
2     $(document).ready(function () {
3         // 获取数据并渲染表格
4         $.ajax({
5             url: '/vulnerabilities-type',
6             dataType: 'json',
7             success: function (data) {
8                 const tableBody1 = $('#tableBody1');
9
10                // 清空当前表格内容
11                tableBody1.empty();
12
13                // 添加每年的数据行
14                data.years.forEach(year => {
15                    const row = `<tr>
16                        <td class="text-center fw-bold">${year}</td>
17                        <td class="text-center">
18                            <span class="badge bg-danger bg-opacity-10 text-
19danger badge-impact clickable" data-year="${year}" data-type="Overflow">
20                                ${data.data[year]['Overflow'] || 0}
21                            </span>
22                        </td>
23                    </tr>
24                `;
25                tableBody1.append(row);
26            }
27        });
28    });
29 </script>

```

```

23         <span class="badge bg-warning bg-opacity-10 text-
warning badge-impact clickable" data-year="{year}" data-
type="MemoryCorruption">
24             ${data.data[year]['MemoryCorruption']} || 0}
25         </span>
26     </td>
27     <td class="text-center">
28         <span class="badge bg-info bg-opacity-10 text-info
badge-impact clickable" data-year="{year}" data-type="SQLInjection">
29             ${data.data[year]['SQLInjection']} || 0}
30         </span>
31     </td>
32     <td class="text-center">
33         <span class="badge bg-secondary bg-opacity-10 text-
secondary badge-impact clickable" data-year="{year}" data-type="XSS">
34             ${data.data[year]['XSS']} || 0}
35         </span>
36     </td>
37     <td class="text-center">
38         <span class="badge bg-success bg-opacity-10 text-
success badge-impact clickable" data-year="{year}" data-
type="DirectoryTraversal">
39             ${data.data[year]['DirectoryTraversal']} || 0}
40         </span>
41     </td>
42     <td class="text-center">
43         <span class="badge bg-danger bg-opacity-10 text-
danger badge-impact clickable" data-year="{year}" data-type="FileInclusion">
44             ${data.data[year]['FileInclusion']} || 0}
45         </span>
46     </td>
47     <td class="text-center">
48         <span class="badge bg-warning bg-opacity-10 text-
warning badge-impact clickable" data-year="{year}" data-type="CSRF">
49             ${data.data[year]['CSRF']} || 0}
50         </span>
51     </td>
52     <td class="text-center">
53         <span class="badge bg-info bg-opacity-10 text-info
badge-impact clickable" data-year="{year}" data-type="XXE">
54             ${data.data[year]['XXE']} || 0}
55         </span>
56     </td>
57     <td class="text-center">
58         <span class="badge bg-secondary bg-opacity-10 text-
secondary badge-impact clickable" data-year="{year}" data-type="SSRF">
59             ${data.data[year]['SSRF']} || 0}

```

```

60         </span>
61     </td>
62     <td class="text-center">
63         <span class="badge bg-success bg-opacity-10 text-
success badge-impact clickable" data-year="{year}" data-type="OpenRedirect">
64             {data.data[year]['OpenRedirect']} || 0}
65         </span>
66     </td>
67     <td class="text-center">
68         <span class="badge bg-dark bg-opacity-10 text-dark
badge-impact clickable" data-year="{year}" data-type="InputValidation">
69             {data.data[year]['InputValidation']} || 0}
70         </span>
71     </td>
72
73 </tr>`;
74 tableBody1.append(row);

```

代码块

[illegible]

```

26         </span>
27     </td>
28     <td class="text-center">
29         <span class="badge bg-info bg-opacity-10 text-info
badge-impact clickable" data-year="${year}" data-type="PrivilegeEscalation">
30             ${data.data[year]['PrivilegeEscalation']} || 0}
31         </span>
32     </td>
33     <td class="text-center">
34         <span class="badge bg-secondary bg-opacity-10 text-
secondary badge-impact clickable" data-year="${year}" data-
type="DenialOfService">
35             ${data.data[year]['DenialOfService']} || 0}
36         </span>
37     </td>
38     <td class="text-center">
39         <span class="badge bg-success bg-opacity-10 text-
success badge-impact clickable" data-year="${year}" data-
type="InformationLeak">
40             ${data.data[year]['InformationLeak']} || 0}
41         </span>
42     </td>
43 </tr>`;
44     tableBody.append(row);
45 });
46
47     // // 更新总计
48     $('#total-code-execution').html(`<a href="/details?
type=CodeExecution" class="text-decoration-none"><span class="badge bg-danger
text-white">${data.totals['CodeExecution']}</a>`);
49     $('#total-bypass').html(`<span class="badge bg-warning text-
white">${data.totals['Bypass']}</span>`);
50     $('#total-privilege').html(`<span class="badge bg-info text-
white">${data.totals['PrivilegeEscalation']}</span>`);
51     $('#total-dos').html(`<span class="badge bg-secondary text-
white">${data.totals['DenialOfService']}</span>`);
52     $('#total-leak').html(`<span class="badge bg-success text-
white">${data.totals['InformationLeak']}</span>`);
53     // 更新总计
54     $('#total-code-execution').html(`
55         <a href="/vulnerabilities_total_details.html?
type=CodeExecution" class="text-decoration-none">
56             <span class="badge bg-danger text-
white">${data.totals['CodeExecution']}</span>
57         </a>`);
58
59     $('#total-bypass').html(`

```

```

60         <a href="/vulnerabilities_total_details.html?type=Bypass"
class="text-decoration-none">
61         <span class="badge bg-warning text-
white">${data.totals['Bypass']}

```

后端源码:

代码块

```

1  func vulnerabilityTypeHandler(db *gorm.DB) func(c *gin.Context) {
2      return func(c *gin.Context) {
3          type ImpactCount struct {
4              Year                string `json:"year"`
5              Overflow              int    `json:"Overflow"`
6              MemoryCorruption        int    `json:"MemoryCorruption"`
7              SQLInjection            int    `json:"SQLInjection"`
8              XSS                    int    `json:"XSS"`
9              DirectoryTraversal      int    `json:"DirectoryTraversal"`
10             FileInclusion             int    `json:"FileInclusion"`
11             CSRF                     int    `json:"CSRF"`
12             XXE                      int    `json:"XXE"`
13             SSRF                     int    `json:"SSRF"`
14             OpenRedirect              int    `json:"OpenRedirect"`
15             InputValidation           int    `json:"InputValidation"`
16         }
17
18         var results []ImpactCount
19         db.Raw("SELECT * FROM vuln_type_stats_view ORDER BY year
ASC").Scan(&results)
20
21         years := []string{}
22         data := make(map[string]map[string]int)
23         totals := map[string]int{
24             "Overflow":          0,
25             "MemoryCorruption":  0,
26             "SQLInjection":      0,
27             "XSS":               0,
28             "DirectoryTraversal": 0,

```

```

29         "FileInclusion":      0,
30         "CSRF":               0,
31         "XXE":                0,
32         "SSRF":               0,
33         "OpenRedirect":       0,
34         "InputValidation":    0,
35     }
36
37     for _, row := range results {
38         year := row.Year
39         years = append(years, year)
40
41         // 每年数据
42         data[year] = map[string]int{
43             "Overflow":      row.Overflow,
44             "MemoryCorruption": row.MemoryCorruption,
45             "SQLInjection":    row.SQLInjection,
46             "XSS":              row.XSS,
47             "DirectoryTraversal": row.DirectoryTraversal,
48             "FileInclusion":     row.FileInclusion,
49             "CSRF":             row.CSRF,
50             "XXE":              row.XXE,
51             "SSRF":             row.SSRF,
52             "OpenRedirect":     row.OpenRedirect,
53             "InputValidation":  row.InputValidation,
54         }
55
56         // 累加总数
57         totals["Overflow"] += row.Overflow
58         totals["MemoryCorruption"] += row.MemoryCorruption
59         totals["SQLInjection"] += row.SQLInjection
60         totals["XSS"] += row.XSS
61         totals["DirectoryTraversal"] += row.DirectoryTraversal
62         totals["FileInclusion"] += row.FileInclusion
63         totals["CSRF"] += row.CSRF
64         totals["XXE"] += row.XXE
65         totals["SSRF"] += row.SSRF
66         totals["OpenRedirect"] += row.OpenRedirect
67         totals["InputValidation"] += row.InputValidation
68     }
69
70     c.JSON(http.StatusOK, gin.H{
71         "years": years,
72         "data": data,
73         "totals": totals,
74     })
75 }

```

6. Vulnerabilities by Impact Types (1999-2024)

前端展示：

Vulnerabilities by Impact Types (1999-2024)

Vulnerability Statistics

Show 10 entries

Search...

Year	Code Execution	Bypass	Privilege Escalation	Denial of Service	Information Leak
1999	0	39	179	298	0
2000	0	53	79	322	0
2001	0	79	51	359	2
2002	0	132	49	548	6
2003	1	72	40	386	4
2004	1	139	9	634	7
2005	5	253	8	761	7
2006	5	260	15	969	8
2007	9	229	16	1103	10
2008	10	393	14	988	10
Total	13393	10060	2831	34596	6150

Showing 1 to 10 of 26 entries

<

1

2

3

>

表 2-9 漏洞影响类型年度数量表

前端源码：

代码块

```
1 <div class="card-header py-3 d-flex flex-row align-items-center justify-
  content-between">
2   <h6 class="m-0 font-weight-bold text-primary">
3     <i class="mdi mdi-table-large impact-icon"></i>
4     Vulnerability Statistics
5   </h6>
6   <div class="dropdown no-arrow">
7     <a class="dropdown-toggle" href="#" role="button" id="dropdownMenuLink"
8       data-bs-toggle="dropdown" aria-expanded="false">
9       <i class="mdi mdi-dots-vertical text-gray-400"></i>
10    </a>
11    <ul class="dropdown-menu dropdown-menu-end shadow animated--fade-in"
12      aria-labelledby="dropdownMenuLink">
13      <li><a class="dropdown-item" href="#"><i
```



```

14         class="mdi mdi-refresh me-2"></i>Refresh</a>
15     </li>
16     <li><a class="dropdown-item" href="#"><i
17         class="mdi mdi-printer me-2"></i>Print</a></li>
18     <div class="dropdown-divider"></div>
19     <li><a class="dropdown-item" href="#"><i
20         class="mdi mdi-cog me-2"></i>Settings</a></li>
21 </ul>
22 </div>
23 </div>
24 <div class="card-body">
25     <div class="table-responsive">
26         <table id="vulnTable" class="table table-bordered table-hover"
width="100%"
27         cellpadding="0">
28         <thead class="thead-light">
29             <tr>
30                 <th class="text-center">
31                     <i class="mdi mdi-calendar impact-icon"></i>Year
32                 </th>
33                 <th class="text-center">
34                     <i class="mdi mdi-code-braces impact-icon text-danger">
</i>Code
35                     Execution
36                 </th>
37                 <th class="text-center">
38                     <i
39                         class="mdi mdi-lock-open-variant impact-icon text-
warning"></i>Bypass
40                 </th>
41                 <th class="text-center">
42                     <i
43                         class="mdi mdi-account-arrow-up impact-icon text-
info"></i>Privilege
44                     Escalation
45                 </th>
46                 <th class="text-center">
47                     <i class="mdi mdi-server-off impact-icon text-
secondary"></i>Denial
48                     of
49                     Service
50                 </th>
51                 <th class="text-center">
52                     <i
53                         class="mdi mdi-information-outline impact-icon
text-success"></i>Information
54                     Leak

```

```

55         </th>
56     </tr>
57 </thead>
58 <tbody id="tableBody">
59     <!-- Data will be inserted here by JavaScript -->
60 </tbody>
61 <tfoot class="total-row">
62     <tr>
63         <th class="text-center">Total</th>
64         <th id="total-code-execution" class="text-center">0</th>
65         <th id="total-bypass" class="text-center">0</th>
66         <th id="total-privilege" class="text-center">0</th>
67         <th id="total-dos" class="text-center">0</th>
68         <th id="total-leak" class="text-center">0</th>
69     </tr>
70 </tfoot>
71 </table>
72 </div>
73 </div>

```

后端源码:

代码块

```

1  func vulnerabilitiesHandler(db *gorm.DB) func(c *gin.Context) {
2      return func(c *gin.Context) {
3          type ImpactCount struct {
4              Year                string
5              CodeExecution        int
6              Bypass                int
7              PrivilegeEscalation    int
8              DenialOfService         int
9              InformationLeak        int
10         }
11
12         var results []ImpactCount
13         db.Raw("SELECT * FROM vuln_impact_stats_view ORDER BY year
14             ASC").Scan(&results)
15
16         years := []string{}
17         data := make(map[string]map[string]int)
18         totals := map[string]int{
19             "CodeExecution":        0,
20             "Bypass":                0,
21             "PrivilegeEscalation":  0,
22             "DenialOfService":      0,
23             "InformationLeak":      0,

```

```

23     }
24
25     for _, row := range results {
26         year := row.Year
27         years = append(years, year)
28         data[year] = map[string]int{
29             "CodeExecution":      row.CodeExecution,
30             "Bypass":              row.Bypass,
31             "PrivilegeEscalation":  row.PrivilegeEscalation,
32             "DenialOfService":      row.DenialOfService,
33             "InformationLeak":      row.InformationLeak,
34         }
35
36         totals["CodeExecution"] += row.CodeExecution
37         totals["Bypass"] += row.Bypass
38         totals["PrivilegeEscalation"] += row.PrivilegeEscalation
39         totals["DenialOfService"] += row.DenialOfService
40         totals["InformationLeak"] += row.InformationLeak
41     }
42
43     c.JSON(http.StatusOK, gin.H{
44         "years":  years,
45         "data":   data,
46         "totals": totals,
47     })
48 }
49 }

```

7. CVSS Score Range - Bar Chart

前端展示：

CVSS Score Range - Bar Chart

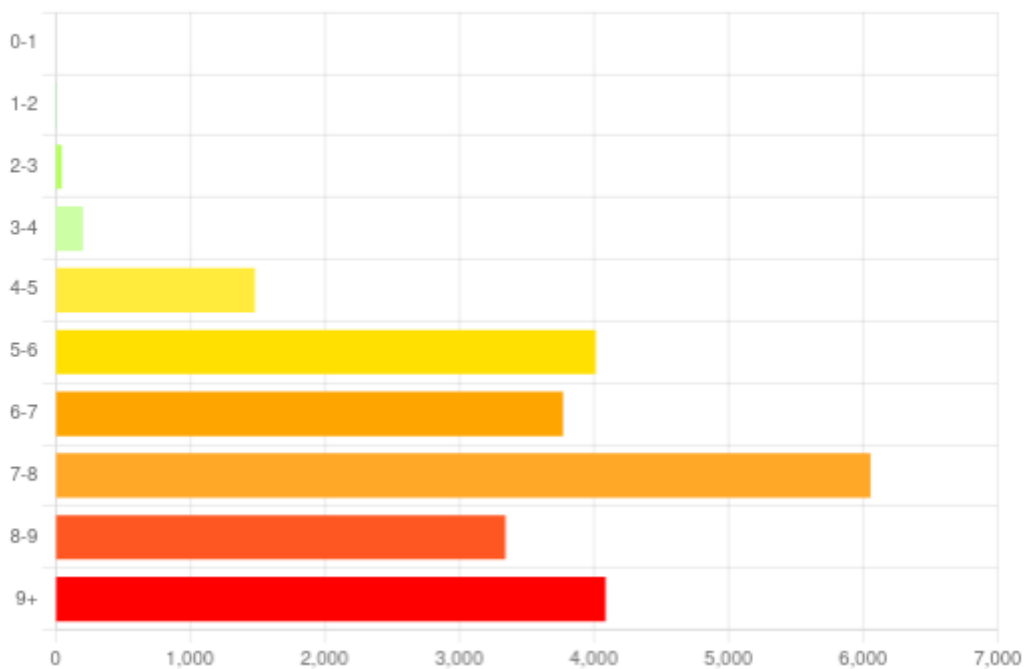


图 2-24 CVSS score 分布直方图

前端源码：

代码块

```
1 <div class="col-md-6 mb-4">
2     <h6 class="mb-3">CVSS Score Range - Bar Chart</h6>
3     <canvas id="barChart" height="200"></canvas>
4 </div>
```

后端展示：

代码块

```
1 func getCVEsBySeverityRange(c *gin.Context) {
2     severity := c.Query("range")
3     start := c.Query("start")
4     end := c.Query("end")
5
6     scoreMin := 0.0
7     scoreMax := 10.0
8
9     switch severity {
10    case "0-1":
11        scoreMax = 1.0
12    case "1-2":
13        scoreMin = 1.0; scoreMax = 2.0
14    case "2-3":
```

```

15     scoreMin = 2.0; scoreMax = 3.0
16     case "3-4":
17         scoreMin = 3.0; scoreMax = 4.0
18     case "4-5":
19         scoreMin = 4.0; scoreMax = 5.0
20     case "5-6":
21         scoreMin = 5.0; scoreMax = 6.0
22     case "6-7":
23         scoreMin = 6.0; scoreMax = 7.0
24     case "7-8":
25         scoreMin = 7.0; scoreMax = 8.0
26     case "8-9":
27         scoreMin = 8.0; scoreMax = 9.0
28     case "9+":
29         scoreMin = 9.0; scoreMax = 10.0
30 }
31
32 var results []CVEIDResult
33
34 err := db.Raw(`
35     SELECT DISTINCT c.id AS id
36     FROM CVEEntry c
37     JOIN Metric m ON c.id = m.cve_id
38     WHERE c.published BETWEEN ? AND ?
39     GROUP BY c.id
40     HAVING MAX(m.baseScore) >= ? AND MAX(m.baseScore) < ?
41 `, start, end, scoreMin, scoreMax).Scan(&results).Error
42
43 if err != nil {
44     c.JSON(500, gin.H{"error": err.Error()})
45     return
46 }
47
48 var cveIDs []string
49 for _, r := range results {
50     cveIDs = append(cveIDs, r.ID)
51 }
52
53 c.JSON(200, cveIDs)
54 }

```

8. CVSS Distribution Bubble Chart

前端展示：

CVSS Distribution Bubble Chart

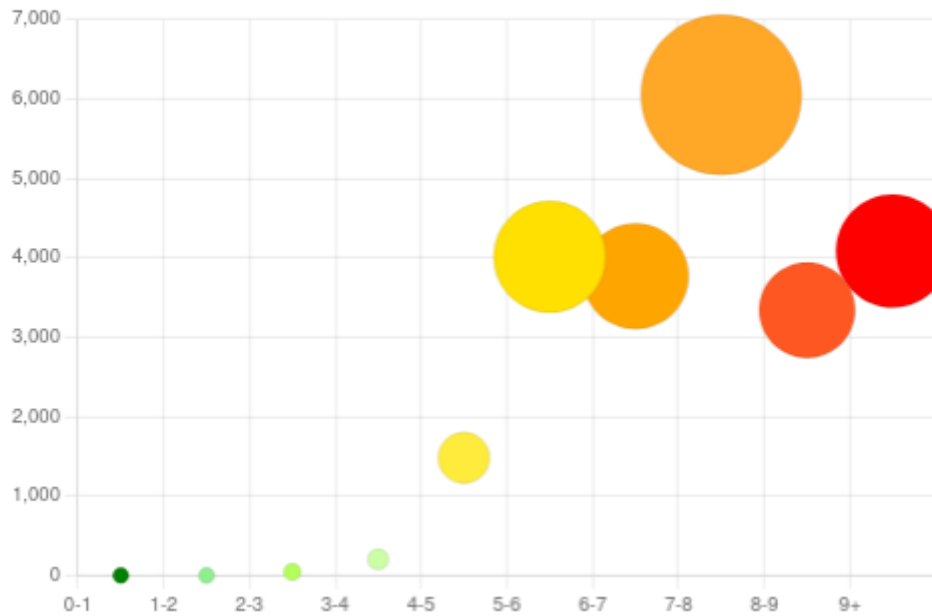


图 2-25 CVSS score 分布气泡图

前端源码：

代码块

```
1 <div class="col-md-6 mb-4">
2   <h6 class="mb-3">CVSS Score Range - Bar Chart</h6>
3   <canvas id="barChart" height="200"></canvas>
4 </div>
```

后端源码：

代码块

```
1 func getCVEsBySeverityRange(c *gin.Context) {
2     severity := c.Query("range")
3     start := c.Query("start")
4     end := c.Query("end")
5
6     scoreMin := 0.0
7     scoreMax := 10.0
8
9     switch severity {
10    case "0-1":
11        scoreMax = 1.0
12    case "1-2":
13        scoreMin = 1.0; scoreMax = 2.0
14    case "2-3":
```

```

15     scoreMin = 2.0; scoreMax = 3.0
16     case "3-4":
17         scoreMin = 3.0; scoreMax = 4.0
18     case "4-5":
19         scoreMin = 4.0; scoreMax = 5.0
20     case "5-6":
21         scoreMin = 5.0; scoreMax = 6.0
22     case "6-7":
23         scoreMin = 6.0; scoreMax = 7.0
24     case "7-8":
25         scoreMin = 7.0; scoreMax = 8.0
26     case "8-9":
27         scoreMin = 8.0; scoreMax = 9.0
28     case "9+":
29         scoreMin = 9.0; scoreMax = 10.0
30 }
31
32 var results []CVEIDResult
33
34 err := db.Raw(`
35     SELECT DISTINCT c.id AS id
36     FROM CVEEntry c
37     JOIN Metric m ON c.id = m.cve_id
38     WHERE c.published BETWEEN ? AND ?
39     GROUP BY c.id
40     HAVING MAX(m.baseScore) >= ? AND MAX(m.baseScore) < ?
41 `, start, end, scoreMin, scoreMax).Scan(&results).Error
42
43 if err != nil {
44     c.JSON(500, gin.H{"error": err.Error()})
45     return
46 }
47
48 var cveIDs []string
49 for _, r := range results {
50     cveIDs = append(cveIDs, r.ID)
51 }
52
53 c.JSON(200, cveIDs)
54 }

```

9. 前端部分 (ECharts)

前端使用 `echarts` 库构建柱状图, 并通过 `fetch()` 并发请求三个后端接口获取数据:

```

1 代码块 document.addEventListener('DOMContentLoaded', async function () {
2      const scoreChart = echarts.init(document.getElementById('score-chart'));
3
4      const [baseRes, impactRes, exploitRes] = await Promise.all([
5          fetch('/api/base-score-distribution'),
6          fetch('/api/impact-score-distribution'),
7          fetch('/api/exploitability-score-distribution'),
8      ]);
9
10     const [baseData, impactData, exploitData] = await Promise.all([
11         baseRes.json(),
12         impactRes.json(),
13         exploitRes.json(),
14     ]);
15
16     // 构造 X 轴标签与 Y 轴数据
17     const xLabels = ['0-1', '1-2', '2-3', '3-4', '4-5', '5-6', '6-7', '7-8',
18         '8-9', '9+'];
19
20     const getCounts = (data) => xLabels.map(label => {
21         const item = data.find(d => d.score_range === label);
22         return item ? item.score_count : 0;
23     });
24
25     const baseCounts = getCounts(baseData);
26     const impactCounts = getCounts(impactData);
27     const exploitCounts = getCounts(exploitData);
28
29     // 初始化 ECharts 配置
30     scoreChart.setOption({
31         tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
32         legend: { data: ['Base Score', 'Impact Score', 'Exploitability Score'] },
33         xAxis: { type: 'category', data: xLabels, name: 'Score Range' },
34         yAxis: { type: 'value', name: 'Count' },
35         series: [
36             { name: 'Base Score', type: 'bar', data: baseCounts },
37             { name: 'Impact Score', type: 'bar', data: impactCounts },
38             { name: 'Exploitability Score', type: 'bar', data: exploitCounts }
39         ]
40     });

```

前端页面中只需引入 `echarts.js` 并放置一个 `<div id="score-chart">` 容器即可渲染完整图表。

三、优势与特点总结

特点	描述
实时数据驱动	所有图表数据来源于数据库实时聚合查询，确保统计结果准确无误
异步加载优化体验	使用 Promise.all 并发请求多个接口，减少页面等待时间
横向对比能力强	支持三种评分在同一图表中对比显示，便于分析风险来源
自动化扩展友好	可轻松添加新的统计维度（如按年份、厂商、产品等）
响应式设计兼容性强	图表自动适配屏幕大小，支持 PC、平板、手机等设备访问
低耦合高可维护	前后端分离架构，便于后续升级或更换可视化库（如 Chart.js）

表 2-10 echarts优势表

2.4.5 用户管理板块实现

一、用户登陆和注册

① 只允许使用邮箱登陆

代码块

```
1  <form id="login-form">
2    <div class="mb-2">
3      <input type="email" class="form-control" placeholder="you@company.com"
4        id="email" required autofocus />
5    </div>
6    <div class="mb-2">
7      <input type="password" class="form-control" placeholder="Password"
8        id="password" required />
9    </div>
10   <div class="form-text text-start mb-3">
11     <a href="#" class="text-decoration-none">Forgot your password?</a>
12   </div>
13   <button type="submit" class="btn btn-primary w-100">Log In</button>
14 </form>
```

代码块

```
1  try {
2    const response = await fetch("/api/auth/login", {
3      method: "POST",
```

```

4         headers: { "Content-Type": "application/json" },
5         body: JSON.stringify({ email, password }),
6         credentials: "include",
7     });
8
9     if (response.ok) {
10         const data = await response.json();
11         alert("Login successful!");
12         window.location.href = "/db_admin.html";
13     } else {
14         const error = await response.json();
15         alert("Login failed: " + error.error);
16     }
17 } catch (err) {
18     alert("Network error: " + err.message);
19 }
20

```

② 注册主函数

代码块

```

1 func (h *authHandler) Login(c *gin.Context) {
2     var in loginCreds
3     if err := c.ShouldBindJSON(&in); err != nil {
4         c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
5         return
6     }
7
8     var user User
9     if err := h.db.Where("email = ? OR username = ?", in.Email,
10 in.Email).First(&user).Error; err != nil {
11         c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid email or
12 password"})
13         return
14     }
15
16     if bcrypt.CompareHashAndPassword(user.PasswordHash, []byte(in.Password))
17 != nil {
18         c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid email or
19 password"})
20         return
21     }
22
23     token, err := h.issueToken(user.ID, user.Email, user.Username)
24     if err != nil {
25

```

```

21         c.JSON(http.StatusInternalServerError, gin.H{"error": "Unable to
generate token"})
22         return
23     }
24
25     setTokenCookie(c, token)
26     c.JSON(http.StatusOK, gin.H{"token": token})
27 }

```

③ 设置JWT和Cookie

代码块

```

1  // 生成 JWT token
2  func (h *authHandler) issueToken(uid uint, email, username string) (string,
error) {
3      claims := jwt.MapClaims{
4          "user_id": uid,
5          "email": email,
6          "username": username, // 需要传入
7          "exp": time.Now().Add(24 * time.Hour).Unix(),
8          "iat": time.Now().Unix(),
9      }
10     return jwt.NewWithClaims(jwt.SigningMethodHS256,
claims).SignedString(h.jwtSecret)
11 }
12
13 // 设置 Cookie
14 func setTokenCookie(c *gin.Context, token string) {
15     domain := ""
16     if strings.Contains(c.Request.Host, "tidb-webapp.local") {
17         domain = "tidb-webapp.local"
18     }
19     secure := domain != ""
20
21     c.SetCookie("token", token, 24*3600, "/", domain, secure, true)
22 }

```

④ 重复性检查

代码块

```

1  // 检查邮箱是否已注册
2  func (h *authHandler) emailExists(email string) bool {
3      var cnt int64
4      h.db.Model(&User{}).Where("email = ?", email).Count(&cnt)

```

```

5     return cnt > 0
6 }
7
8 func (h *authHandler) usernameExists(username string) bool {
9     var cnt int64
10    h.db.Model(&User{}).Where("username = ?", username).Count(&cnt)
11    return cnt > 0
12 }

```

二、登出

登出的解决方法是：设置cookie的时间为负数，即cookie就无效了

代码块

```

1 // GET /api/auth/logout
2 func (h *authHandler) Logout(c *gin.Context) {
3     domain := ""
4     if strings.Contains(c.Request.Host, "tidb-webapp.local") {
5         domain = "tidb-webapp.local"
6     }
7     secure := domain != ""
8     // 清除 token cookie
9     c.SetCookie("token", "", -1, "/", domain, secure, true)
10    c.JSON(http.StatusOK, gin.H{"message": "Successfully logged out"})
11 }

```

三、权限控制

具体演示

① 使用admin5:admin@admin.com:admin注册

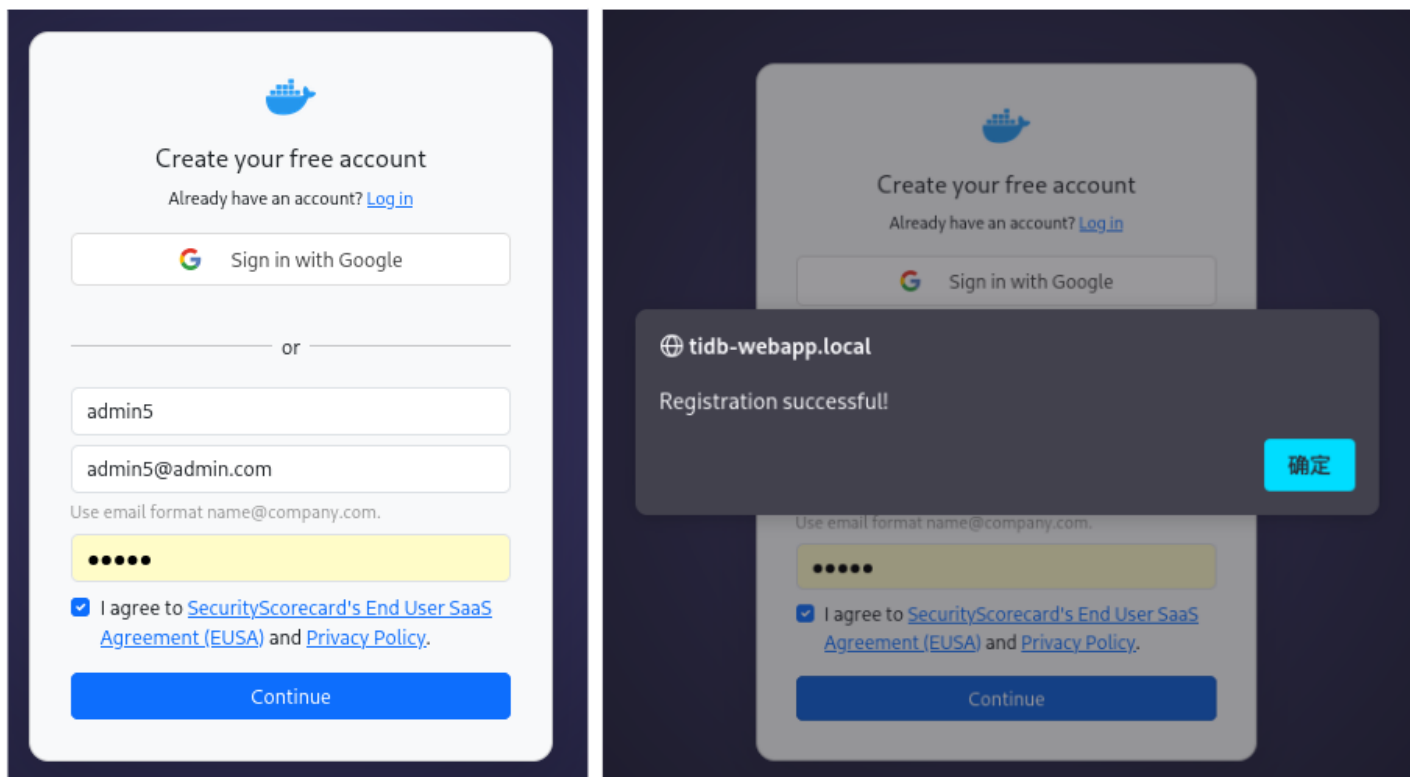


图 2-26 登陆展示图

② 这里只是注册，没有授予权限

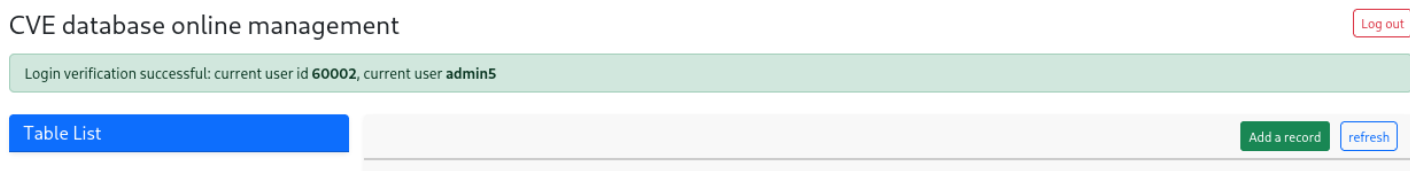


图 2-27 后台页面展示图

没有信息，需要管理员授予权限

30001	30001	Weakness	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30002	30001	vulnerabilityyearmonth	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30003	30001	vulnerability_type_stats	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30004	30001	vuln_types_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30005	30001	vuln_type_summary_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30006	30001	vuln_type_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30007	30001	vuln_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30008	30001	vuln_stats_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30009	30001	vuln_impact_types_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30010	30001	vuln_impact_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30011	30001	view_cve_vuln_type_summary	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30012	30001	view_cve_vuln_summary	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30013	30001	VendorComment	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30014	30001	VectorEmbedding	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30015	30001	users	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30016	30001	test	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30017	30001	Reference	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30018	30001	permissions	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30019	30001	ParsedVector	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30020	30001	p_results_display_of_cve	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30021	30001	Metric	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30022	30001	impact_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30023	30001	exploitability_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30024	30001	Description	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30025	30001	CVETag	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30026	30001	CVEEntry	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30027	30001	cve_severity_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30028	30001	cve_category_match	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30029	30001	cpe_entries	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30030	30001	Configuration	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30031	30001	category_lookup	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30032	30001	base_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09

64 rows in set (0.002 sec)

图 2-28 permissions数据表图

可以通过sql语句进行权限授予，也可以登陆admin@admin.com:admin

③ 使用sql语句授予

代码块

```

1  INSERT INTO permissions (
2      id,
3      user_id,
4      table_name,
5      can_select,
6      can_insert,
7      can_update,
8      can_delete,
9      created_at,
10     updated_at
11 ) VALUES (
12     NULL, 60002, 'users', 1, 1, 1, 0, NOW(), NOW()
13 ), (
14     NULL, 60002, 'CVEEntry', 1, 1, 0, 0, NOW(), NOW()
15 ), (
16     NULL, 60002, 'Description', 1, 0, 0, 0, NOW(), NOW()
17 );

```

CVE database online management

Log out

Login verification successful: current user id 60002, current user admin5

Table List

CVEEntry

Description

users

CVEEntry

Add a record

refresh

id	sourceidentifier	published	lastModified	vulnStatus	evaluatorComment	evaluatorImpact	evaluatorS
CVE-1999-0001	cve@mitre.org	1999-12-30T05:00:00-05:00	2024-11-20T23:27:31-05:00	Modified			
CVE-1999-0002	cve@mitre.org	1998-10-12T04:00:00-04:00	2024-11-20T23:27:34-05:00	Modified			
CVE-1999-0003	cve@mitre.org	1998-04-01T05:00:00-05:00	2024-11-20T23:27:35-05:00	Modified			
CVE-1999-0004	cve@mitre.org	1997-12-16T05:00:00-05:00	2024-11-20T23:27:35-05:00	Modified			
CVE-1999-0005	cve@mitre.org	1998-07-20T04:00:00-04:00	2024-11-20T23:27:35-05:00	Modified			
CVE-1999-0006	cve@mitre.org	1998-07-14T04:00:00-04:00	2024-11-20T23:27:35-05:00	Modified			

图 2-29 后台页面展示图

④ 后台管理系统进行插入

Add a record

user_id

60002

table_name

test

can_select

1

can_insert

1

can_update

1

can_delete

1

created_at

Cancel

Save

图 2-30 后端编辑页面图

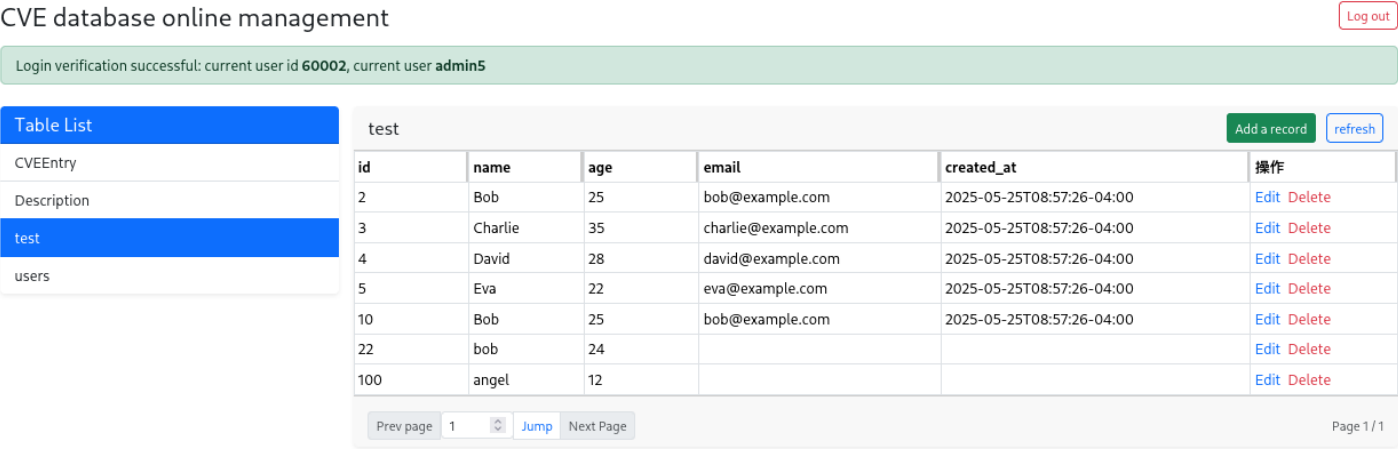


图 2-31 后端编辑页面验证图

2.4.6 后台管理板块

为了提升系统的可维护性与安全性，系统为管理员提供了专用的后台管理界面和接口，支持对数据库表的结构查看、数据维护、权限配置、用户管理等操作。该模块不仅便于开发团队进行数据调试，也适用于运维人员进行日常巡检与安全加固。

一、功能概述

本模块主要实现以下核心功能：

功能	描述
数据库表结构浏览	支持查看所有表名、字段名、类型、主键信息，便于理解数据模型
表数据增删改查	提供基本的数据编辑能力，如添加漏洞条目、修改 CVE 描述、删除无效数据等
用户管理	查看、新增、编辑、删除系统用户，设置角色与权限
权限分配	管理不同用户对各表的操作权限（SELECT / INSERT / UPDATE / DELETE）

表 2-11 后台管理页面功能表

二、技术实现说明

1. 合法性登陆检查

安全机制	实现方式
身份认证	基于 JWT 验证，仅管理员 Token 可访问
接口权限控制	使用中间件限制非管理员调用
输入验证	所有请求参数均做合法性校验，防止注入攻击

表 2-12 后台登陆验证检查表

A. 检查是否是已注册用户

代码块

```
1  const res = await fetch('/api/auth/protected', { credentials: 'include' });
```

代码块

```
1  // ProtectedHandler 只有通过 JWTMiddleware 验证后才能访问
2  func ProtectedHandler(c *gin.Context) {
3      // 从 context 里读取中间件事先放进去的字段
4      userIDVal, _ := c.Get("user_id")
5      emailVal, _ := c.Get("email")
6      usernameVal, _ := c.Get("username") // 新增
7
8      // 基于安全起见再做一次存在性校验
9      if userIDVal == nil || emailVal == nil {
10         c.JSON(401, gin.H{"error": "unauthorized"})
11         return
12     }
13
14     c.JSON(200, gin.H{
15         "user_id": userIDVal,
16         "email":    emailVal,
17         "username": usernameVal, // 可选返回 username
18         "status":  "token valid",
19     })
20 }
```

B. 检查登陆用户的权限

代码块

```
1  // 获取指定用户在某张表上的权限标志
2  func (a *App) getPerm(userID uint, table string) (permFlags, error) {
3      var p permFlags
4      err := a.DB.
5          Table("permissions").
6          Select("can_select, can_insert, can_update, can_delete").
7          Where("user_id = ? AND table_name = ?", userID, table).
8          Scan(&p).Error
9      return p, err
10 }
```

没有权限的无法通过下面的语句获取表的信息

代码块

```
1  const res = await fetch(`${api}/tables`, { credentials: 'include' });
```

代码块

```
1  func (a *App) ListTables(c *gin.Context) {
2      userIDAny, ok := c.Get("user_id")
3      if !ok {
4          c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "missing
user_id"})
5          return
6      }
7      userID := userIDAny.(uint)
8
9      rows, err := a.DB.Raw("SHOW TABLES").Rows()
10     if err != nil {
11         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
12         return
13     }
14     defer rows.Close()
15
16     var tables []string
17     for rows.Next() {
18         var t string
19         rows.Scan(&t)
20
21         // 查询权限，只保留有 SELECT 权限的
22         p, _ := a.getPerm(userID, t)
23         if p.allow("select") {
24             tables = append(tables, t)
25         }
26     }
27     c.JSON(http.StatusOK, tables)
28 }
```

C. 根据获取的权限展示后端返回的信息

CVE database online management

Log out

Login verification successful: current user id 60002, current user admin5

Table List

CVEEntry

Description

test

users

test

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete

Prev page 1 Jump Next Page

Add a recordrefresh

Page 1 / 1

图 2-33 后台页面展示图

2. 数据库元信息查询

A. 受保护的api接口

代码块

```
1 dbmsadmin := &dbadmin.App{DB: db}
2 protecteddbms := r.Group("/api/db")
3 protecteddbms.Use(handlers.JWTMiddleware())
4 {
5     protecteddbms.GET("/tables", dbmsadmin.ListTables)           // list all
6     protecteddbms.GET("/:table", dbmsadmin.ListRecords)         // list
7     protecteddbms.GET("/:table/:id", dbmsadmin.GetRecord)       // fetch
8     protecteddbms.POST("/:table", dbmsadmin.CreateRecord)       // insert
9     protecteddbms.PUT("/:table/:id", dbmsadmin.UpdateRecord)    // update
10    protecteddbms.DELETE("/:table/:id", dbmsadmin.DeleteRecord) // delete
11    protecteddbms.GET("/:table/count", dbmsadmin.CountRecords)
12 }
```

代码块

```
1 // -- 权限检查
2 uidAny, _ := c.Get("user_id")
3 if uidAny == nil {
4     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
5         "unauthorized"})
6     return
7 }
```

```

6  }
7  uid := uidAny.(uint)
8  p, _ := a.getPerm(uid, table)
9  if !p.allow("select") {
10     c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "no select
        permission"})
11     return
12 }

```

前面的权限验证成功后，就可以调用被保护对应的api接口，获取表名、表数据等

B. 获取数据

代码块

```

1  func (a *App) GetRecord(c *gin.Context) {
2     table := c.Param("table")
3
4     uidAny, _ := c.Get("user_id")
5     if uidAny == nil {
6         c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
7         return
8     }
9     uid := uidAny.(uint)
10    p, _ := a.getPerm(uid, table)
11    if !p.allow("select") {
12        c.JSON(http.StatusForbidden, gin.H{
13            "error": "permission denied",
14            "message": fmt.Sprintf("You do not have permission to perform the
                query operation, access to table %s is denied", table),
15            "redirect": "/permission-denied.html",
16        })
17        return
18    }
19
20    id, err := parseID(c)
21    if err != nil {
22        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
23        return
24    }
25
26    sqlStr := fmt.Sprintf("SELECT * FROM %s WHERE id = ? LIMIT 1",
        quoteIdent(table))
27    rows, err := a.DB.Raw(sqlStr, id).Rows()
28    if err != nil {
29        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
30        return

```

```

31     }
32     defer rows.Close()
33
34     data, err := rowsToMaps(rows)
35     if err != nil {
36         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
37         return
38     }
39     if len(data) == 0 {
40         c.JSON(http.StatusNotFound, gin.H{"error": "not found"})
41         return
42     }
43     c.JSON(http.StatusOK, data[0])
44 }

```

CVE database online management

[Log out](#)

Login verification successful: current user id 60002, current user admin5

Table List	test						Add a record	refresh
CVEEntry								
Description								
test								
users								

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete

Prev page 1 [Jump](#) Next Page

Page 1 / 1

图 2-34 后台页面展示图

C. 插入数据

代码块

```

1 // CreateRecord inserts a row – expects arbitrary JSON object.
2 func (a *App) CreateRecord(c *gin.Context) {
3     table := c.Param("table")
4
5     // -- 权限检查
6     uidAny, _ := c.Get("user_id")
7     if uidAny == nil {
8         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
9             "unauthorized"})
10        return
11    }
12    uid := uidAny.(uint)
13    p, _ := a.getPerm(uid, table)
14    if !p.allow("insert") {

```

```

14         c.JSON(http.StatusForbidden, gin.H{
15             "error": "permission denied",
16             "message": fmt.Sprintf("You do not have permission to perform the
insert operation, access to table %s is denied", table),
17             "redirect": "/permission-denied.html",
18         })
19         c.Abort()
20         return
21     }
22
23     var payload map[string]interface{}
24     if err := c.BindJSON(&payload); err != nil {
25         c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
26         return
27     }
28
29     result := a.DB.Table(table).Create(&payload)
30     if result.Error != nil {
31         c.JSON(http.StatusInternalServerError, gin.H{"error":
result.Error.Error()})
32         return
33     }
34     c.JSON(http.StatusCreated, gin.H{"affected": result.RowsAffected, "id":
payload["id"]})
35 }

```

在 test 表中插入 id = 101 name = Make age = 20

插入前：

test						Add a record	refresh
id	name	age	email	created_at	操作		
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete		
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete		
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete		
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete		
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete		
22	bob	24			Edit Delete		
100	angel	12			Edit Delete		

Prev page 1 [Jump](#) Next Page

Page 1 / 1

图 2-35 插入信息前展示图

```
MySQL [cve_db]> select * from test;
+-----+-----+-----+-----+-----+
| id | name | age | email | created_at |
+-----+-----+-----+-----+-----+
| 2 | Bob | 25 | bob@example.com | 2025-05-25 08:57:26 |
| 3 | Charlie | 35 | charlie@example.com | 2025-05-25 08:57:26 |
| 4 | David | 28 | david@example.com | 2025-05-25 08:57:26 |
| 5 | Eva | 22 | eva@example.com | 2025-05-25 08:57:26 |
| 10 | Bob | 25 | bob@example.com | 2025-05-25 08:57:26 |
| 22 | bob | 24 | NULL | NULL |
| 100 | angel | 12 | NULL | NULL |
+-----+-----+-----+-----+-----+
7 rows in set (0.002 sec)
```

图 2-36 插入前数据库表图

插入后：

test Add a record refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete
101	Make	20			Edit Delete

Prev page 1 Jump Next Page Page 1 / 1

图 2-37 插入信息后展示图

```
MySQL [cve_db]> select * from test;
+-----+-----+-----+-----+-----+
| id | name | age | email | created_at |
+-----+-----+-----+-----+-----+
| 2 | Bob | 25 | bob@example.com | 2025-05-25 08:57:26 |
| 3 | Charlie | 35 | charlie@example.com | 2025-05-25 08:57:26 |
| 4 | David | 28 | david@example.com | 2025-05-25 08:57:26 |
| 5 | Eva | 22 | eva@example.com | 2025-05-25 08:57:26 |
| 10 | Bob | 25 | bob@example.com | 2025-05-25 08:57:26 |
| 22 | bob | 24 | NULL | NULL |
| 100 | angel | 12 | NULL | NULL |
| 101 | Make | 20 | NULL | NULL |
+-----+-----+-----+-----+-----+
8 rows in set (0.002 sec)
```

图 2-38 插入后数据库表图

D. 更新数据

代码块

```
1 // UpdateRecord updates a row by `id` column.
2 func (a *App) UpdateRecord(c *gin.Context) {
```

```

3     table := c.Param("table")
4
5     // -- 权限检查
6     uidAny, _ := c.Get("user_id")
7     if uidAny == nil {
8         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"unauthorized"})
9         return
10    }
11    uid := uidAny.(uint)
12    p, _ := a.getPerm(uid, table)
13    if !p.allow("update") {
14        c.JSON(http.StatusForbidden, gin.H{
15            "error": "permission denied",
16            "message": fmt.Sprintf("You do not have permission to perform the
update operation, access to table %s is denied", table),
17            "redirect": "/permission-denied.html",
18        })
19        c.Abort()
20        return
21    }
22
23    id, err := parseID(c)
24    if err != nil {
25        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
26        return
27    }
28    var payload map[string]interface{}
29    if err := c.BindJSON(&payload); err != nil {
30        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
31        return
32    }
33
34    result := a.DB.Table(table).Where("id = ?", id).Updates(payload)
35    if result.Error != nil {
36        c.JSON(http.StatusInternalServerError, gin.H{"error":
result.Error.Error()})
37        return
38    }
39    c.JSON(http.StatusOK, gin.H{"affected": result.RowsAffected})
40 }

```

讲Make的年龄更新为25

更新前：

test

Add a record

refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete
101	Make	20			Edit Delete

Prev page

1

Jump

Next Page

Page 1 / 1

图 2-39 更新信息前展示图

更新后

test

Add a record

refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete
101	Make	20			Edit Delete

Prev page

1

Jump

Next Page

Page 1 / 1

图 2-40 更新信息后展示图

Add a record

id

101

name

Make

age

25

email

created_at

Cancel

Save

图 2-41 后端添加页面图

test

Add a record

refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete
101	Make	25			Edit Delete

Prev page

1

Jump

Next Page

Page 1 / 1

图 2-42 添加成功展示图

```
MySQL [cve_db]> select * from test;
```

id	name	age	email	created_at
2	Bob	25	bob@example.com	2025-05-25 08:57:26
3	Charlie	35	charlie@example.com	2025-05-25 08:57:26
4	David	28	david@example.com	2025-05-25 08:57:26
5	Eva	22	eva@example.com	2025-05-25 08:57:26
10	Bob	25	bob@example.com	2025-05-25 08:57:26
22	bob	24	NULL	NULL
100	angel	12	NULL	NULL
101	Make	25	NULL	NULL

8 rows in set (0.002 sec)

图 2-43 添加成功数据库表图

E. 删除数据

代码块

```
1 func (a *App) DeleteRecord(c *gin.Context) {
2     table := c.Param("table")
3
4     uidAny, _ := c.Get("user_id")
5     if uidAny == nil {
6         c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
7         return
8     }
9     uid := uidAny.(uint)
10    p, _ := a.getPerm(uid, table)
11    if !p.allow("delete") {
12        // 返回 JSON 告诉前端权限不足，前端负责跳转
13        c.JSON(http.StatusForbidden, gin.H{
14            "error": "permission denied",
15            "message": fmt.Sprintf("You do not have permission to perform the
delete operation, access to table %s is denied", table),
16            "redirect": "/permission-denied.html", // 前端根据这个字段跳转
17        })
18        c.Abort()
19        return
20    }
21
22    id, err := parseID(c)
23    if err != nil {
24        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
25        return
26    }
27
28    result := a.DB.Table(table).Where("id = ?", id).Delete(nil)
29    if result.Error != nil {
```

```
30         c.JSON(http.StatusInternalServerError, gin.H{"error":
result.Error.Error()})
31         return
32     }
33     c.JSON(http.StatusOK, gin.H{"affected": result.RowsAffected})
34 }
```

test

Add a recordrefresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete
101	Make	25			Edit Delete

Prev page1JumpNext Page

Page 1 / 1

图 2-44 后端删除展示图

test

Add a recordrefresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete

Prev page1JumpNext Page

Page 1 / 1

```
MySQL [cve_db]> select * from test;
+----+-----+-----+-----+-----+
| id | name  | age  | email                | created_at |
+----+-----+-----+-----+-----+
|  2 | Bob   |  25 | bob@example.com      | 2025-05-25 08:57:26 |
|  3 | Charlie | 35 | charlie@example.com  | 2025-05-25 08:57:26 |
|  4 | David |  28 | david@example.com    | 2025-05-25 08:57:26 |
|  5 | Eva   |  22 | eva@example.com      | 2025-05-25 08:57:26 |
| 10 | Bob   |  25 | bob@example.com      | 2025-05-25 08:57:26 |
| 22 | bob   |  24 | NULL                 | NULL       |
| 100 | angel |  12 | NULL                 | NULL       |
+----+-----+-----+-----+-----+
7 rows in set (0.002 sec)
```

图 2-45 删除后页面和数据库表展示图

F. 权限不足时的更新，插入，删除操作

```
MySQL [cve_db]> select * from permissions where user_id = 60002;
```

id	user_id	table_name	can_select	can_insert	can_update	can_delete	created_at	updated_at
60001	60002	users	1	1	1	0	2025-06-07 03:06:42	2025-06-07 03:06:42
60002	60002	CVEEntry	1	1	0	0	2025-06-07 03:06:42	2025-06-07 03:06:42
60003	60002	Description	1	0	0	0	2025-06-07 03:06:42	2025-06-07 03:06:42
60004	60002	test	1	1	1	1	NULL	NULL

4 rows in set (0.004 sec)

这里的Description只有查看权限

1) 插入测试操作

Add a record

cve_id

CVE-2025-0001

lang

en

value

This is a test.

Cancel

Save

tidb-webapp.local

You do not have permission to perform the insert operation, access to table Description is denied

确定

图 2-46 测试插入操作图

这里提示权限不足

Insufficient permissions (403 Forbidden)

You do not have permission to perform this operation, access is denied.

[Back to Home](#)

[GIN]	2025/06/07 - 04:06:59	403	2.705956ms	127.0.0.1	POST	"/api/db/Description"
[GIN]	2025/06/07 - 04:07:56	200	1.319291ms	127.0.0.1	GET	"/permission-denied.html"

图 2-47 权限不足及拒绝删除示意图

2) 测试更新操作

测试 user_id = 30001 的用户

```
MySQL [cve_db]> select * from permissions where user_id = 30001;
```

id	user_id	table_name	can_select	can_insert	can_update	can_delete	created_at	updated_at
30001	30001	Weakness	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30002	30001	vulnerabilityyearmonth	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30003	30001	vulnerability_type_stats	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30004	30001	vuln_types_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30005	30001	vuln_type_summary_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30006	30001	vuln_type_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30007	30001	vuln_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30008	30001	vuln_stats_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30009	30001	vuln_impact_types_per_year	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30010	30001	vuln_impact_stats_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30011	30001	view_cve_vuln_type_summary	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30012	30001	view_cve_vuln_summary_cache_data	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30013	30001	VendorComment	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30014	30001	VectorEmbedding	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30015	30001	users	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30016	30001	test	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30017	30001	Reference	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30018	30001	permissions	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30019	30001	ParsedVector	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30020	30001	p_results_display_of_cve_table_tasketch	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30021	30001	Metric	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30022	30001	impact_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30023	30001	exploitability_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30024	30001	Description	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30025	30001	CVETag	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30026	30001	CVEEntry	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30027	30001	cve_severity_view	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30028	30001	cve_category_match	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30029	30001	cpe_entries	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30030	30001	Configuration	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30031	30001	category_lookup	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09
30032	30001	base_score_distribution	1	1	0	0	2025-05-27 08:47:09	2025-05-27 08:47:09

32 rows in set (0.007 sec)

图 2-48 用户30001权限图

所有的更新和插入操作全都是0（false）

test

Add a record

refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete

Prev page

1

Jump

Next Page

Page 1 / 1

ger
1000

Edit Record

×

id

100

name

angel

age

20

email

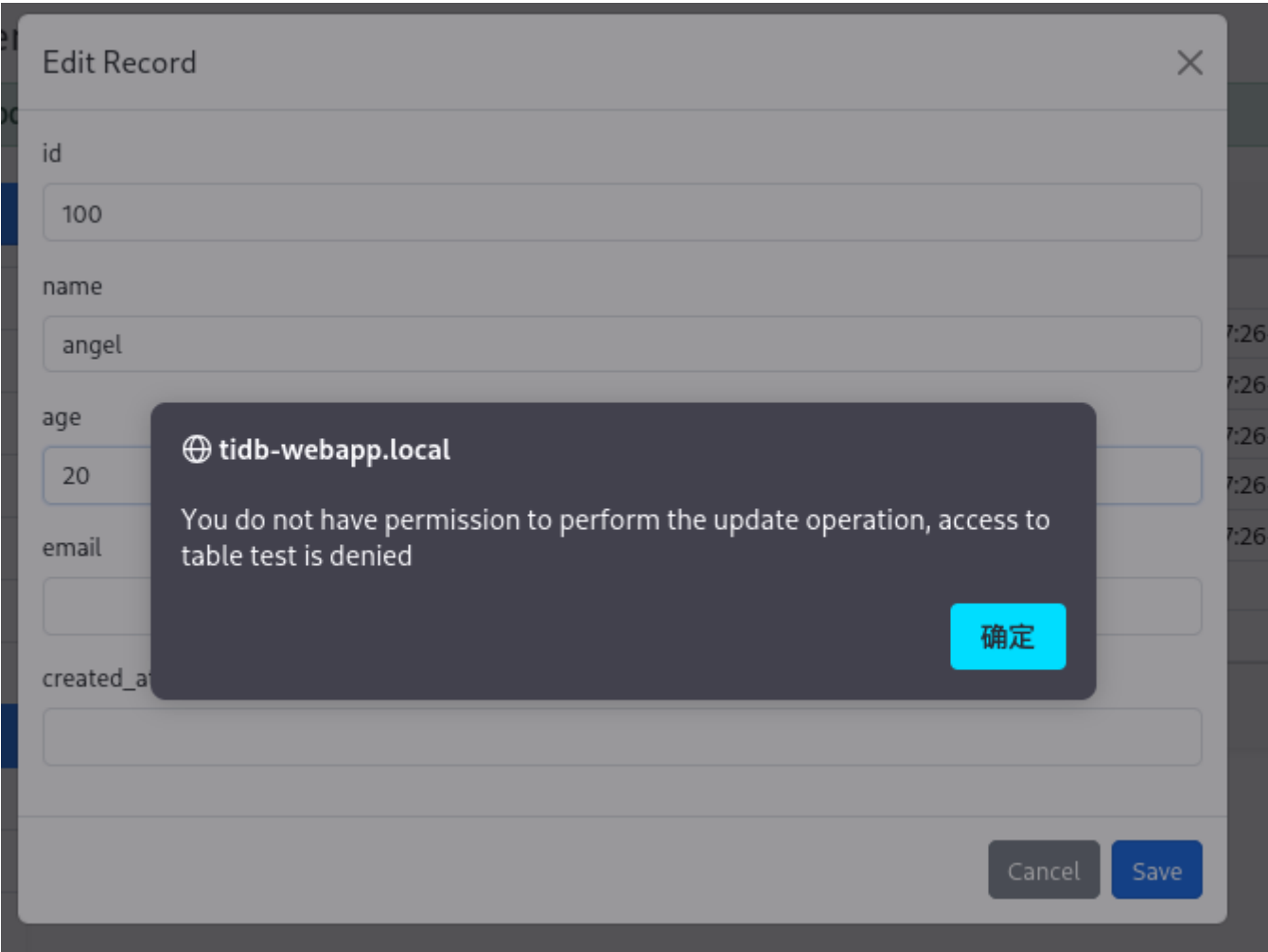
created_at

Cancel

Save

图 2-49 更新操作示意图

权限拒绝



3) 测试删除操作

test Add a record refresh

id	name	age	email	created_at	操作
2	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
3	Charlie	35	charlie@example.com	2025-05-25T08:57:26-04:00	Edit Delete
4	David	28	david@example.com	2025-05-25T08:57:26-04:00	Edit Delete
5	Eva	22	eva@example.com	2025-05-25T08:57:26-04:00	Edit Delete
10	Bob	25	bob@example.com	2025-05-25T08:57:26-04:00	Edit Delete
22	bob	24			Edit Delete
100	angel	12			Edit Delete

Prev page 1 Jump Next Page

Page 1 / 1

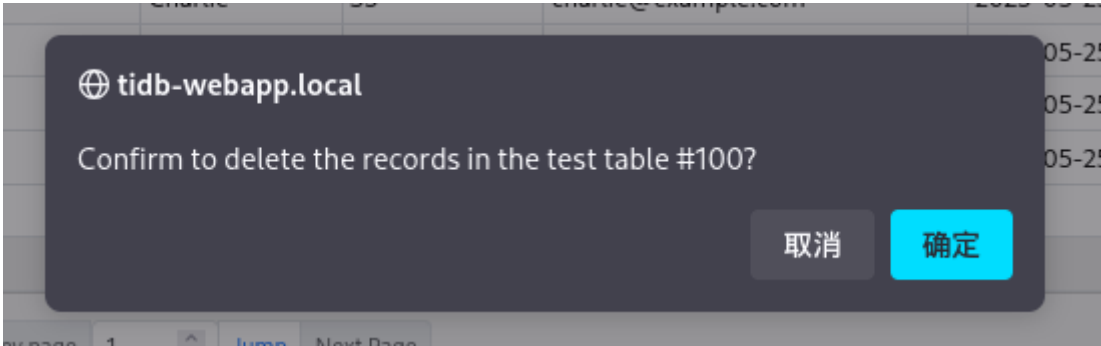


图 2-50 删除操作示意图

Insufficient permissions (403 Forbidden)

You do not have permission to perform this operation, access is denied.

[Back to Home](#)

2.5 安全机制设计

2.5.1 数据加密与传输安全（HTTPS、CORS策略）

为保障系统间数据通信的完整性和机密性，防止中间人攻击（MITM）和跨域请求伪造等安全风险，本系统在数据传输层采用了 HTTPS 加密协议，并通过严格的 CORS（Cross-Origin Resource Sharing，跨源资源共享）策略控制跨域访问行为。

一、HTTPS 加密通信

系统全面采用 HTTPS 协议保障对外通信的安全性，确保客户端与服务端之间的所有数据传输均经过加密处理。具体实现上，所有暴露给客户端的 API 接口均通过基于 TLS/SSL 的加密通道进行访问，有效防止数据在传输过程中被窃听、篡改或冒充。

在加密细节方面，系统采用了强加密套件（Cipher Suites），兼容现代浏览器和各类客户端的安全连接需求。同时，部署了由受信任证书颁发机构（CA）签发的 SSL 证书，确保通信双方的身份均经过严格验证，避免中间人攻击。为进一步提升性能和安全性，服务器支持 HTTP/2 协议，利用其多路复用和头部压缩等特性优化数据传输。

此外，系统禁用了所有不安全的旧版本协议，如 SSLv3、TLS 1.0 和 TLS 1.1，仅启用 TLS 1.2 及以上版本，以避免因协议漏洞带来的安全风险。通过这一系列 HTTPS 的部署措施，系统有效保障了用户隐私及业务数据的安全，满足了现代网络安全的基本要求。

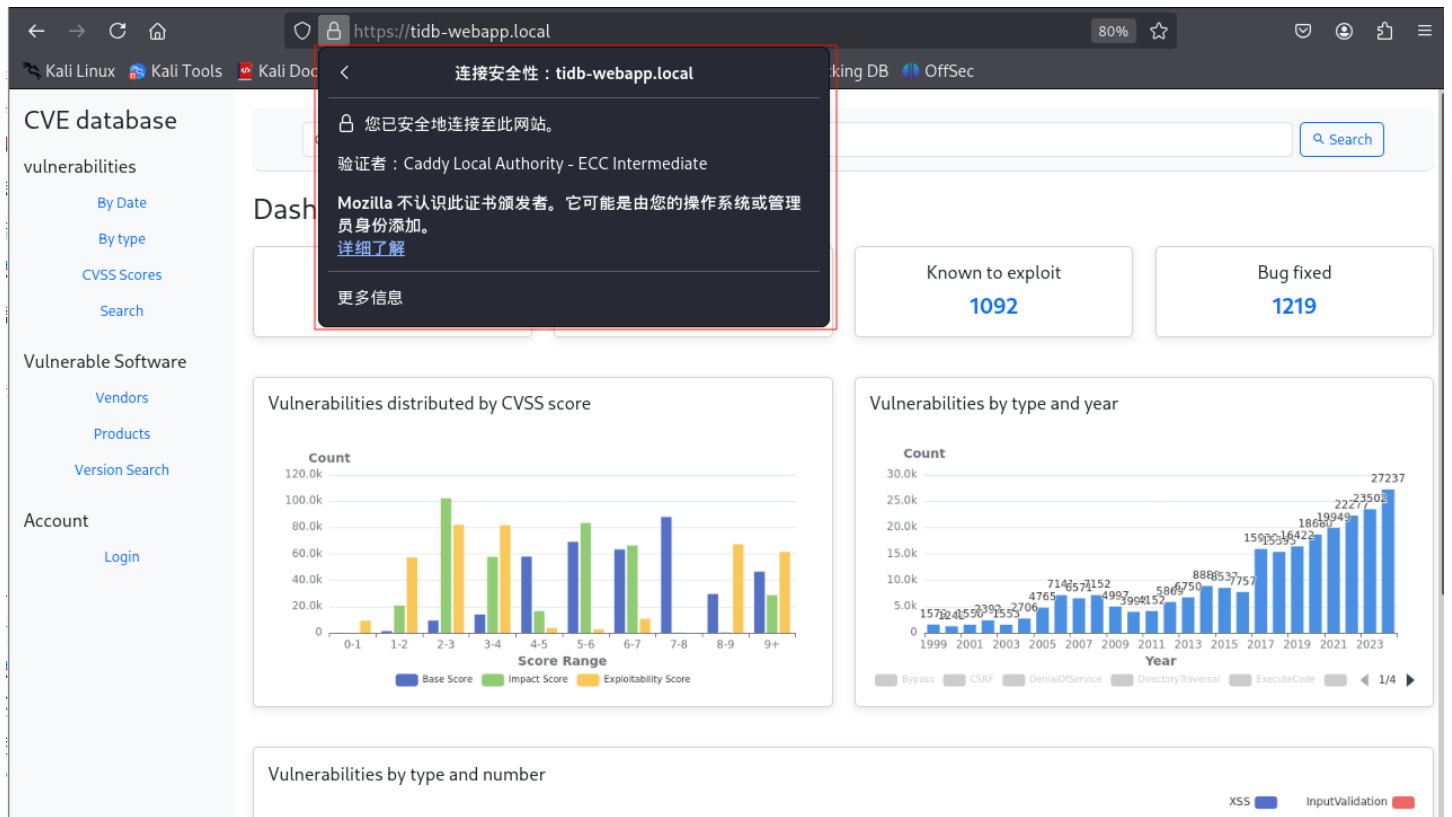


图 2-51 已启用https服务示意图

二、CORS 策略配置

为了防范跨域请求带来的安全风险，同时支持合法的前端应用访问后端资源，系统通过精细配置的 CORS（跨来源资源共享）策略对跨域请求来源进行严格限制。具体配置包括：

系统仅允许特定的可信域名（如 <https://tidb-webapp.local>）作为跨域请求的来源，通过设置 `Access-Control-Allow-Origin` 响应头来阻止来自未授权网站的访问，保障接口安全。对于需要携带身份认证信息（如 Cookie 或 Token）的请求，系统开启了 `Access-Control-Allow-Credentials`，确保认证凭据能安全传输，防止身份信息泄露。

此外，系统明确限定了允许的 HTTP 请求方法（如 GET、POST、PUT、DELETE、OPTIONS），避免非法方法调用带来的潜在威胁。对于请求头，系统通过 `Access-Control-Allow-Headers` 指定允许的字段（如 `Content-Type`、`Authorization`、`X-Requested-With`），防止恶意头部注入攻击。

对于复杂跨域请求，浏览器会自动发起 OPTIONS 预检请求，系统能正确响应这些预检请求，返回相应的 CORS 头部信息，确保合法请求能够通过浏览器的安全策略。通过上述配置，系统在保证安全性的前提下，实现了灵活且安全的跨域资源访问控制。

Caddyfile

```
1  tidb-webapp.local {
2    tls internal
3    encode zstd gzip
4    reverse_proxy 127.0.0.1:8080 {
5      header_up X-Forwarded-Proto {scheme}
6      header_up X-Read-IP {remote-host}
```

```

7      header_up X-Forwarded-For {forwarded_for}
8  }
9  header {
10     Access-Control-Allow-Origin https://tidb-webapp.local
11     Access-Control-Allow-Credentials true
12     Access-Control-Allow-Methods GET,POST,OPTIONS
13     Access-Control-Allow-Headers Content-Type,Authorization
14     Vary Origin
15  }
16  }

```

此外，在反向代理或网关层（如 Caddy）中统一配置 CORS 响应头，确保策略一致性与可维护性。

```

(base) [kali4redis@kali4redis ~/Desktop]
$ sudo systemctl status caddy
[sudo] password for kali4redis:
● caddy.service - Caddy
   Loaded: loaded (/usr/lib/systemd/system/caddy.service; disabled; preset: disabled)
   Active: active (running) since Sat 2025-06-07 09:09:07 EDT; 18min ago
 Invocation: b3dcbfd0cd064fa5a5aac2db5e9752d4
    Docs: https://caddyserver.com/docs/
   Main PID: 2215 (caddy)
      Tasks: 13 (limit: 14152)
  Memory: 60.9M (peak: 61.1M)
     CPU: 716ms
   CGroup: /system.slice/caddy.service
           └─2215 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile

Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"info","ts":1749301747.7977774,"logger":"tls.renew","msg":"releasing lock","identifier":"tidb-webapp.local"}
Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"info","ts":1749301747.7978003,"logger":"tls","msg":"reloading managed certificate","identifiers":["tidb-we"]
Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"info","ts":1749301747.79794,"logger":"tls.cache","msg":"replaced certificate in cache","subjects":["tidb-we"]
Jun 07 09:09:07 kali4redis sudo[2279]: pam_unix(sudo:auth): conversation failed
Jun 07 09:09:07 kali4redis sudo[2279]: pam_unix(sudo:auth): auth could not identify password for [caddy]
Jun 07 09:09:07 kali4redis sudo[2279]: caddy : user NOT in sudoers ; PWD=/ ; USER=root ; COMMAND=/usr/bin/tee /usr/local/share/ca-certificates/Caddy_Local
Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"error","ts":1749301747.800454,"logger":"pki.ca.local","msg":"failed to install root certificate","error":">
Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"info","ts":1749301747.801537,"msg":"autosaved config (load with --resume flag)","file":"/var/lib/caddy/.cop
Jun 07 09:09:07 kali4redis caddy[2215]: {"level":"info","ts":1749301747.8017638,"msg":"serving initial configuration"}
Jun 07 09:09:07 kali4redis systemd[1]: Started caddy.service - Caddy.
lines 1-22/22 (END)

```

图 2-52 Caddy 服务状态图

三、安全建议与最佳实践

为了提升系统的安全性，避免潜在风险，以下最佳实践被严格遵循：

首先，系统避免在 `Access-Control-Allow-Origin` 中使用通配符 `*`，尤其是在需要携带用户凭证（如 Cookie 或 Token）的场景下，防止任意域名访问敏感接口，确保跨域请求来源的严格限制。

其次，对于关键或敏感接口，除了基于 Token 的认证机制外，还结合 IP 白名单等多重安全措施，进一步强化访问控制，抵御非法请求和恶意攻击。

此外，系统定期更新和维护 SSL/TLS 证书，确保使用由权威证书颁发机构签发的证书，并及时检查服务器配置以防范已知安全漏洞（如 POODLE、BEAST、CRIME 等攻击），保障通信链路的安全性。

同时，启用 HSTS（HTTP Strict Transport Security）策略，强制浏览器始终使用 HTTPS 访问系统，防止协议降级攻击和中间人攻击，提高整体安全防护水平。

通过 HTTPS 与严格的 CORS 策略协同配合，系统构建了从传输加密到访问控制的多层次安全防护机制，显著增强了数据传输的安全性和系统抵御攻击的能力。

2.5.2 防御常见 Web 攻击（SQL 注入、XSS、CSRF）

一、SQL 注入攻击

1) 攻击原理概述

（1）概述

SQL 注入（SQL Injection）是一种常见的 Web 安全漏洞，攻击者通过在输入字段中注入恶意 SQL 代码，绕过应用程序的正常逻辑，直接操作数据库，从而实现非法查询、篡改数据、删除数据甚至获取数据库权限等行为。

例如：

代码块

```
1 ' OR '1'='1
```

或更复杂的联合查询语句，可能绕过登录验证、窃取敏感信息。

（2）SQL 注入攻击原理

当 Web 应用程序未对用户输入进行有效的过滤、转义或参数化处理时，攻击者可以通过构造恶意的 SQL 语句注入到数据库查询中，进而篡改或操控数据库操作。攻击手段多样，包括在表单输入框中提交带有 SQL 代码的特殊字符串，利用 URL 参数伪造请求，甚至通过 JSON 数据结构的字段注入恶意代码。此外，攻击者还可采用盲注（Blind SQL Injection）技术，逐步探测数据库结构和数据内容，从而制定更精准的攻击策略。

（3）SQL 注入危害

SQL 注入攻击会对系统安全造成极大威胁，主要表现为窃取敏感数据（如用户账号、密码和交易记录等），非法修改或删除关键数据，破坏数据库完整性。攻击者还可能绕过身份认证机制，实现未授权访问，甚至利用数据库特定功能执行系统命令，对服务器造成严重威胁。更严重时，SQL 注入还可能导致服务不可用（DoS），使系统瘫痪，影响正常业务运行。

2) 防御策略

为了防止 SQL 注入攻击，我们采用了以下多层次防护机制：

（1）使用 ORM 框架参数化查询

本项目使用 GORM（Go 的 ORM 框架），其核心优势之一就是自动将用户输入作为参数绑定到 SQL 查询中，而不是拼接字符串。这种方式能从根本上防止 SQL 注入。

示例：安全的插入操作

代码块

```
1 result := db.Table("users").Create(&User{Name: name, Email: email})
```

示例：安全的更新操作

代码块

```
1 db.Table("users").Where("id = ?", id).Updates(payload)
```

GORM 自动处理参数绑定，确保所有用户输入都被正确转义和隔离。

(2) 避免手动拼接 SQL 字符串

尽管部分接口存在动态 SQL 构造需求，但已采取以下措施降低风险：

代码块

```
1 sqlStr := fmt.Sprintf("SELECT * FROM %s ORDER BY %s ASC LIMIT ? OFFSET  
?", quoteIdent(table), quoteIdent(orderBy))
```

对于字段名和表名使用反引号包裹并转义：

代码块

```
1 func quoteIdent(name string) string {  
2     return "`" + strings.ReplaceAll(name, "`", "` `") + "`"  
3 }
```

后续重构为使用 GORM 的 `.Order()` 和 `.Select()` 方法替代手动拼接。

(3) 字段白名单控制

对排序字段进行优先级映射检查，确保只允许合法字段用于排序：

代码块

```
1 var priorityOrderMap = map[string][]string{  
2     "base_score_distribution": {"range_9_10", "range_8_9", "range_7_8"},  
3     // ...  
4 }
```

(4) 使用 Web 应用防火墙（WAF）

通过部署 WAF（如 Nginx 结合 ModSecurity、Cloudflare、AWS WAF 等）可以有效检测并拦截常见的 SQL 注入攻击特征码，为系统提供额外的安全防护层。WAF 能实时监控 HTTP 请求，过滤恶意流量，防止攻击者利用漏洞实施注入攻击，从而降低系统被攻破的风险。经过 WAF 的保护，系统中 SQL 注入风险已基本得到杜绝。同时，建议在应用层进一步减少直接使用原生 SQL，采用参数化查询和 ORM 框架，以提升整体安全性和代码的可维护性。

二、跨站脚本攻击（XSS）防御

1) 攻击原理概述

跨站脚本攻击（Cross-Site Scripting, XSS）指攻击者通过注入恶意脚本代码，使之在其他用户的浏览器中执行，达到窃取身份、劫持会话、修改页面等目的。XSS 分为以下几类：

- **反射型 XSS**：恶意脚本作为请求参数立即被响应返回。
- **存储型 XSS**：恶意脚本存储在数据库中，被持久化地注入到页面中。
- **DOM 型 XSS**：通过前端 JavaScript 操作引发脚本注入。

2) 防御措施

（1）前端防御（通过 Caddy 配置）：

在 Caddyfile 中配置了多个 HTTP 安全头来对抗 XSS 攻击：

Caddyfile

```
1 header {
2     X-Content-Type-Options "nosniff"
3     X-Frame-Options "DENY"
4     X-XSS-Protection "1; mode=block"
5     Content-Security-Policy: default-src 'self'; script-src 'self';
6 }
```

- `X-XSS-Protection`：启用浏览器内建的 XSS 过滤器。
- `Content-Security-Policy (CSP)`：严格限制加载的脚本来源，防止外部注入脚本。
- `X-Content-Type-Options`：防止浏览器 MIME 类型猜测。
- `X-Frame-Options`：防止被嵌套到 iframe 中，防范点击劫持。

（2）后端防御：

输出内容转义：所有向前端返回的 HTML 内容必须使用 `html/template` 自动转义，避免动态注入脚本。例如：

代码块

```
1 tmpl := template.Must(template.New("page").ParseFiles("template.html"))
2 tmpl.Execute(w, data) // 默认会自动 HTML 转义
```

严格的 Content-Type 设置：

确保 API 仅返回 `application/json` 或 `text/html`，避免脚本注入被解释为 HTML。

代码块

```
1 c.Header("Content-Type", "application/json")
```

不信任用户输入：

注册、登录接口中已通过 `binding:"required"` 等验证规则防止无效或注入型输入，例如：

代码块

```
1 type creds struct {
2     Email    string `json:"email" binding:"required,email"`
3     Username string `json:"username" binding:"required,min=3,max=32"`
4     Password string `json:"password" binding:"required,min=5,max=72"`
5 }
```

三、跨站请求伪造（CSRF）防御

1) 攻击原理概述

跨站请求伪造（Cross-Site Request Forgery, CSRF）攻击是指攻击者诱导已登录用户在未经授权的情况下向目标网站发送请求，利用其登录状态完成非法操作。

例如，用户登录后未登出，在其他恶意网站点击某链接会触发对目标网站的状态更改请求（如 POST `/api/auth/logout`）。

2) 防御策略

(1) 采用 JWT + Cookie 签名机制

你的系统登录后会返回 JWT 并通过 `Set-Cookie` 设置：

代码块

```
1 c.SetCookie("token", token, 24*3600, "/", domain, secure, true)
```

JWT 签名本身能防止伪造凭证。

(2) 开启 `SameSite=Lax` 限制跨站携带 Cookie

Caddyfile 中设置了 Cookie 属性：

代码块

```
1 Set-Cookie "SameSite=Lax"
```

这意味着：浏览器在从其他站点跳转或嵌套请求时**不会**自动携带 Cookie，有效防止了 GET/POST 类 CSRF 请求。

第三章 系统部署与测试

3.1 系统环境

3.1.1 操作系统版本

代码块

```
1 cat /etc/os-release
```

代码块

```
1 └─(kali㉿kali)-[/proc/sys]
2 └─$ cat /etc/os-release
3 PRETTY_NAME="Kali GNU/Linux Rolling"
4 NAME="Kali GNU/Linux"
5 VERSION_ID="2025.2"
6 VERSION="2025.2"
7 VERSION_CODENAME=kali-rolling
8 ID=kali
9 ID_LIKE=debian
10 HOME_URL="https://www.kali.org/"
11 SUPPORT_URL="https://forums.kali.org/"
12 BUG_REPORT_URL="https://bugs.kali.org/"
13 ANSI_COLOR="1;31"
```

3.1.2 内核版本

代码块

```
1 uname -a
```

代码块

```
1 └─(kali㉿kali)-[/proc/sys]
2 └─$ uname -a
3 Linux kali 6.12.25-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.25-1kali1 (2025-04-30) x86_64 GNU/Linux
```

3.1.3 硬件版本

代码块

```
1 sudo lshw
```


代码块

```
1  └─(kali㉿kali)-[~/Desktop]
2  └─$ sudo lshw
3  kali
4      description: Computer
5      product: VMware Virtual Platform
6      vendor: VMware, Inc.
7      version: None
8      serial: VMware-56 4d 7e 72 d0 99 94 00-cd c7 c8 ac b7 a8 c8 da
9      width: 64 bits
10     capabilities: smbios-2.7 dmi-2.7 smp vsyscall32
11     configuration: administrator_password=enabled boot=normal
frontpanel_password=unknown keyboard_password=unknown power-
on_password=disabled uuid=727e4d56-99d0-0094-cdc7-c8acb7a8c8da
```

3.1.4 CPU信息

代码块

```
1  lscpu
```

代码块

```
1  └─(kali㉿kali)-[~/Desktop]
2  └─$ lscpu
3  Architecture:                x86_64
4      CPU op-mode(s):          32-bit, 64-bit
5      Address sizes:            45 bits physical, 48 bits virtual
6      Byte Order:               Little Endian
7  CPU(s):                      8
8      On-line CPU(s) list:      0-7
9  Vendor ID:                   GenuineIntel
10     Model name:                13th Gen Intel(R) Core(TM) i7-13700H
11     CPU family:                6
12     Model:                    186
13     Thread(s) per core:       1
14     Core(s) per socket:       2
15     Socket(s):                4
16     Stepping:                 2
17     BogoMIPS:                 5836.79
18     Flags:                    fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp
```

```

19             lm constant_tsc arch_perfmon rep_good nopl xtopology
tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq ssse3 fma cx16 pc
20             id sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx
f16c rdrand hypervisor lahf_lm abm 3dnowprefetch pti ssbd ibrs ibpb stibp fsgsb
21             ase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid
rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni
ara
22             t umip gfni vaes vpclmulqdq rdpid movdiri movdir64b
fstrm md_clear serialize flush_l1d arch_capabilities
23 Virtualization features:
24   Hypervisor vendor:      VMware
25   Virtualization type:    full
26 Caches (sum of all):
27   L1d:                    384 KiB (8 instances)
28   L1i:                    256 KiB (8 instances)
29   L2:                     10 MiB (8 instances)
30   L3:                     96 MiB (4 instances)
31 NUMA:
32   NUMA node(s):          1
33   NUMA node0 CPU(s):     0-7
34 Vulnerabilities:
35   Gather data sampling:   Not affected
36   Itlb multihit:         Not affected
37   L1tf:                  Mitigation; PTE Inversion
38   Mds:                   Mitigation; Clear CPU buffers; SMT Host state unknown
39   Meltdown:              Mitigation; PTI
40   Mmio stale data:        Unknown: No mitigations
41   Reg file data sampling: Vulnerable: No microcode
42   Retbleed:              Mitigation; IBRS
43   Spec rstack overflow:   Not affected
44   Spec store bypass:      Mitigation; Speculative Store Bypass disabled via
prctl
45   Spectre v1:            Mitigation; usercopy/swapgs barriers and __user
pointer sanitization
46   Spectre v2:            Mitigation; IBRS; IBPB conditional; STIBP disabled;
RSB filling; PBRSE-eIBRS Not affected; BHI SW loop, KVM SW loop
47   Srbds:                 Not affected
48   Tsx async abort:       Not affected
49

```

3.1.5 网络信息

代码块

```
1 ip a
```

代码块

```
1  └─(kali㉿kali)-[~/Desktop]
2  └─$ ip a
3  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
    default qlen 1000
4      link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5      inet 127.0.0.1/8 scope host lo
6          valid_lft forever preferred_lft forever
7      inet6 ::1/128 scope host noprefixroute
8          valid_lft forever preferred_lft forever
9  2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    group default qlen 1000
10     link/ether 00:0c:29:a8:c8:da brd ff:ff:ff:ff:ff:ff
11     inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic
        noprefixroute eth0
12         valid_lft 1402sec preferred_lft 1402sec
13     inet6 fe80::20c:29ff:fea8:c8da/64 scope link noprefixroute
14         valid_lft forever preferred_lft forever
15  3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    group default qlen 1000
16     link/ether 00:0c:29:a8:c8:e4 brd ff:ff:ff:ff:ff:ff
17     inet 10.10.10.12/24 brd 10.10.10.255 scope global eth1
18         valid_lft forever preferred_lft forever
19     inet6 fe80::20c:29ff:fea8:c8e4/64 scope link proto kernel_ll
20         valid_lft forever preferred_lft forever
```

3.2 TiDB集群部署

3.2.1 下载并安装TiUP

代码块

```
1  curl --proto '=https' --tlsv1.2 -sSf https://tiup-
    mirrors.pingcap.com/install.sh | sh
```

安装完成后会提示如下信息：

```
(kali㉿kali)-[~/Desktop]
$ curl --proto '=https' --tlsv1.2 -sSf https://tiup-mirrors.pingcap.com/install.sh | sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Done    0     0    24.1M           0 --:--:-- --:--:-- --:--:-- 24.1M
Successfully set mirror to https://tiup-mirrors.pingcap.com
Detected shell: zsh
Shell profile: /home/kali/.zshrc
/home/kali/.zshrc has been modified to add tiup to PATH
open a new terminal or source /home/kali/.zshrc to use it
Installed path: /home/kali/.tiup/bin/tiup

=====
Have a try:    tiup playground
=====
```

请注意上述输出中的 Shell profile 文件路径，下一步中需要使用该路径。

3.2.2 声明全局环境变量

代码块

```
1 source ${your_shell_profile}
```

示例：

代码块

```
1 source /home/kali/.zshrc
```

```
(kali㉿kali)-[~/Desktop]
$ source /home/kali/.zshrc

(kali㉿kali)-[~/Desktop]
$
```

3.2.3 在当前session执行以下命令启动集群

代码块

```
1 tiup playground --tag ${tag_name}
```

示例：

代码块

```
1 tiup playground --tag tidb_test
```

下载和启动成功，命令输出中将显示集群的端点列表：

```
(kali@kali)-[~/Desktop]
$ tiup playground --tag tidb_test
Checking updates for component playground... Timedout (after 2s)
The component `playground` version is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/playground-v1.16.2-linux-amd64.tar.gz 8.20 MiB / 8.20 MiB 100.00% 204.70 MiB/s

Note: Version constraint is resolved to v8.5.2. If you'd like to use other versions:

Use exact version:    tiup playground v7.1.0
Use version range:    tiup playground ^5
Use nightly:          tiup playground nightly

The component `pd` version v8.5.2 is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/pd-v8.5.2-linux-amd64.tar.gz 54.32 MiB / 54.32 MiB 100.00% 15.70 MiB/s
Start pd instance: v8.5.2
The component `tikv` version v8.5.2 is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/tikv-v8.5.2-linux-amd64.tar.gz 363.63 MiB / 363.63 MiB 100.00% 11.00 MiB/s
Start tikv instance: v8.5.2
The component `tidb` version v8.5.2 is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/tidb-v8.5.2-linux-amd64.tar.gz 91.70 MiB / 91.70 MiB 100.00% 9.78 MiB/s
Start tidb instance: v8.5.2
Waiting for tidb instances ready
- TiDB: 127.0.0.1:4000 ... Done
The component `prometheus` version v8.5.2 is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/prometheus-v8.5.2-linux-amd64.tar.gz 126.65 MiB / 126.65 MiB 100.00% 11.75 MiB/s
download https://tiup-mirrors.pingcap.com/grafana-v8.5.2-linux-amd64.tar.gz 50.21 MiB / 50.21 MiB 100.00% 12.24 MiB/s
The component `tiflash` version v8.5.2 is not installed; downloading from repository.
download https://tiup-mirrors.pingcap.com/tiflash-v8.5.2-linux-amd64.tar.gz 510.19 MiB / 510.19 MiB 100.00% 10.88 MiB/s
Start tiflash instance: v8.5.2
Waiting for tiflash instances ready
- TiFlash: 127.0.0.1:3930 ... Done

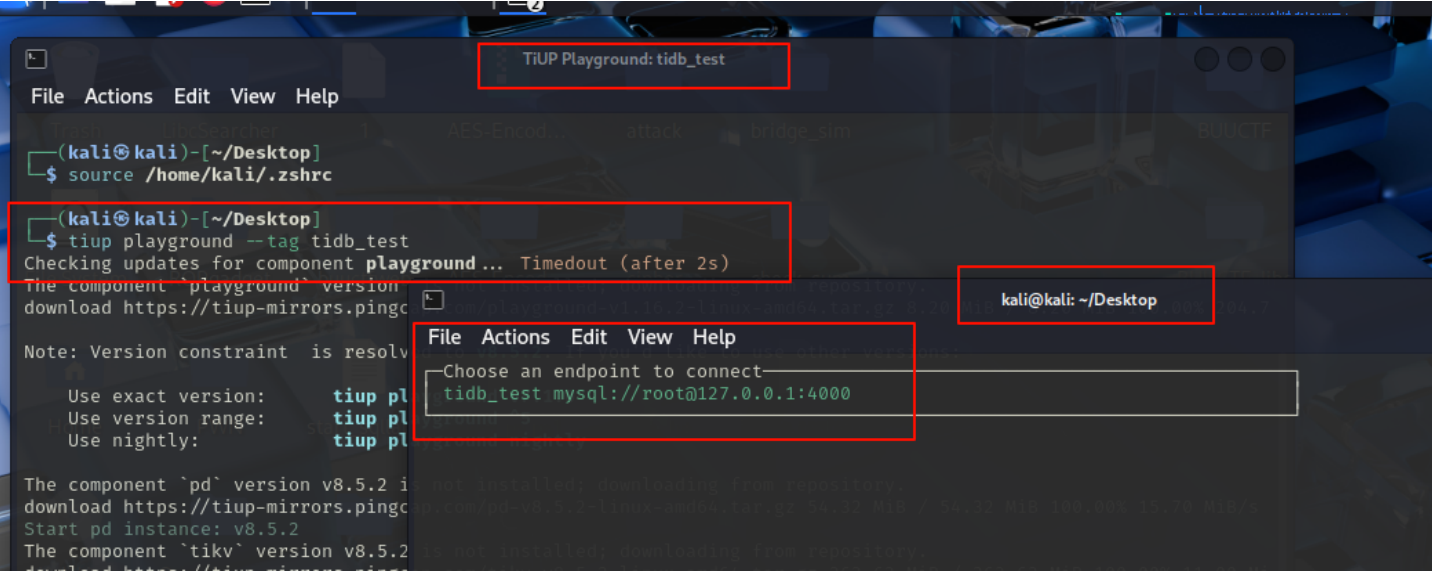
🎉 TiDB Playground Cluster is started, enjoy!

Connect TiDB:    mysql --comments --host 127.0.0.1 --port 4000 -u root
TiDB Dashboard: http://127.0.0.1:2379/dashboard
Grafana:         http://127.0.0.1:3000
```

3.2.4 另起一个终端，输入下面的命令，启动TiDB的客户端

代码块

```
1 tiup client
```



如果达到上面的展示，那就意味着TiDB已经部署成功了

3.3 Go环境部署

3.3.1 更新软件包

代码块

```
1 sudo apt update
```

3.3.2 安装go

代码块

```
1 sudo apt install golang-go
```

3.3.3 验证安装

代码块

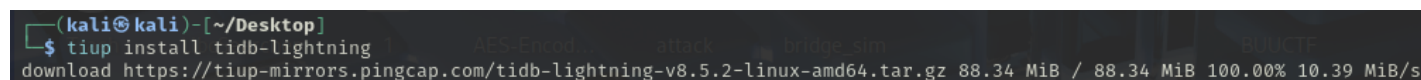
```
1 go version
```

3.3.4 数据库数据导入

1) 下载tidb的官方导入软件

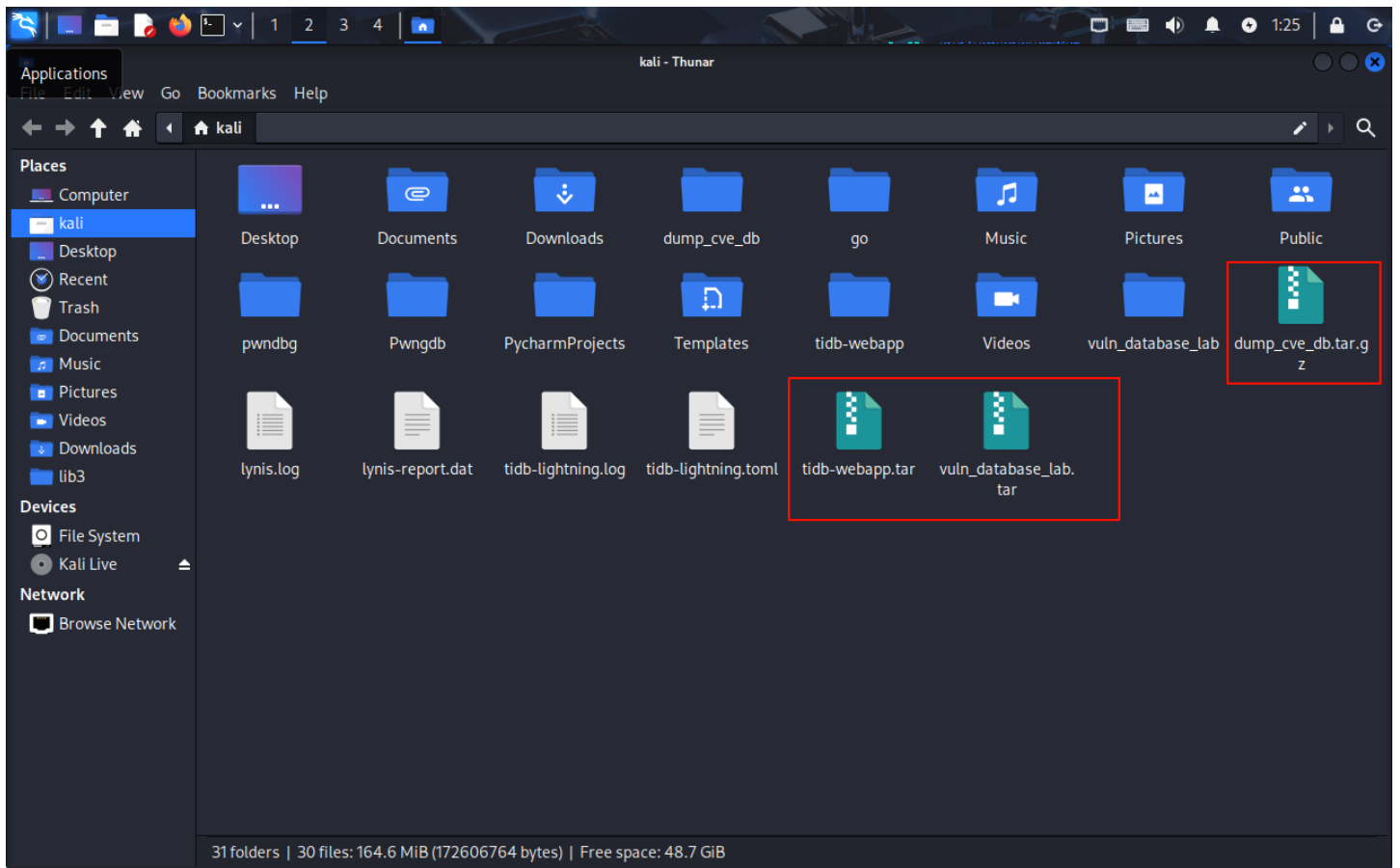
代码块

```
1 tiup install tidb-lightning
```



```
(kali㉿kali)-[~/Desktop]
$ tiup install tidb-lightning
download https://tiup-mirrors.pingcap.com/tidb-lightning-v8.5.2-linux-amd64.tar.gz 88.34 MiB / 88.34 MiB 100.00% 10.39 MiB/s
```

2) 将 `tidb-webapp.tar`、`vulndatabase_lab.tar`、`dump_cve_db.tar.gz` 放在同一个文件夹下



我这里是放在 `/home/kali` 也就是我的家目录下

3) 在 `/home/kali` 配置 `tidb-lightning.toml`

代码块

```
1  sudo vim tidb-lightning.toml
```

```
[lightning]
level = "info"
file = "tidb-lightning.log"

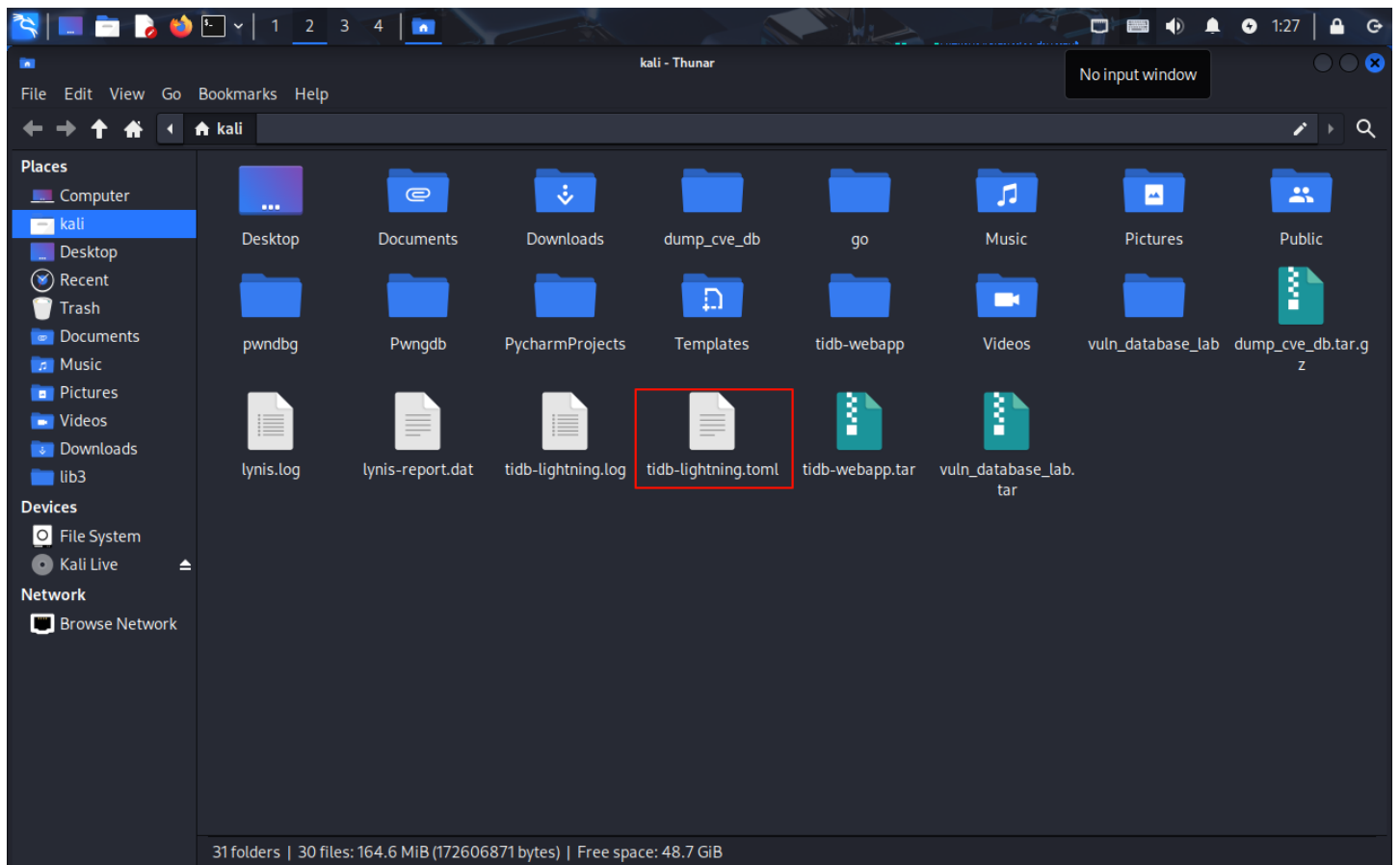
[tidb]
host = "127.0.0.1"
port = 4000
user = "root"
password = ""
status-port = 10080
pd-addr = "127.0.0.1:2379"
```

将下面的配置文件写进 `tidb-lightning.toml`

代码块

```
1  [lightning]
2  level = "info"
3  file = "tidb-lightning.log"
4
```

```
5 [tidb]
6 host = "127.0.0.1"
7 port = 4000
8 user = "root"
9 password = ""
10 status-port = 10080
11 pd-addr = "127.0.0.1:2379"
```



4) 解压三个压缩包

代码块

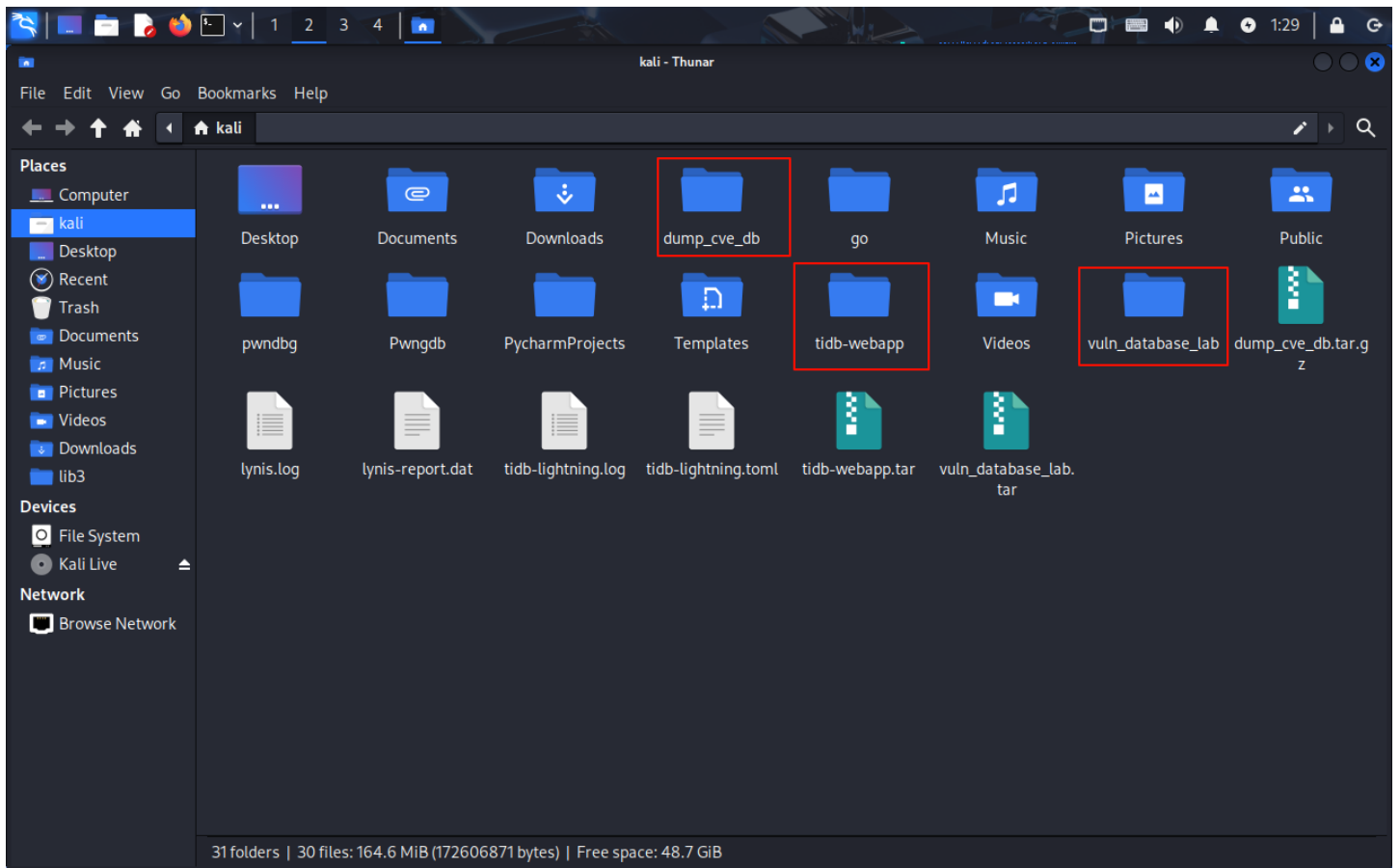
```
1 tar -xf tidb-webapp.tar
```

代码块

```
1 tar -xf vuln_database_lab.tar
```

代码块

```
1 tar -zxvf dump_cve_db.tar.gz
```

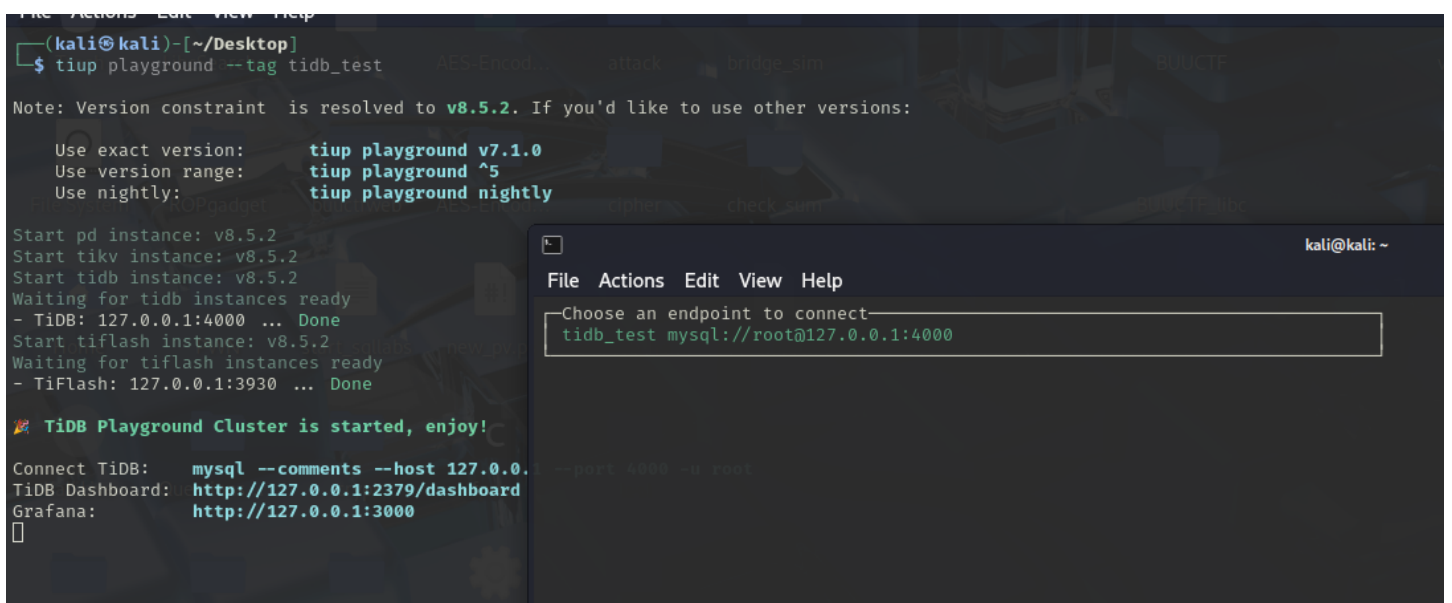
5) 先启动tidb的服务器和客户端

代码块

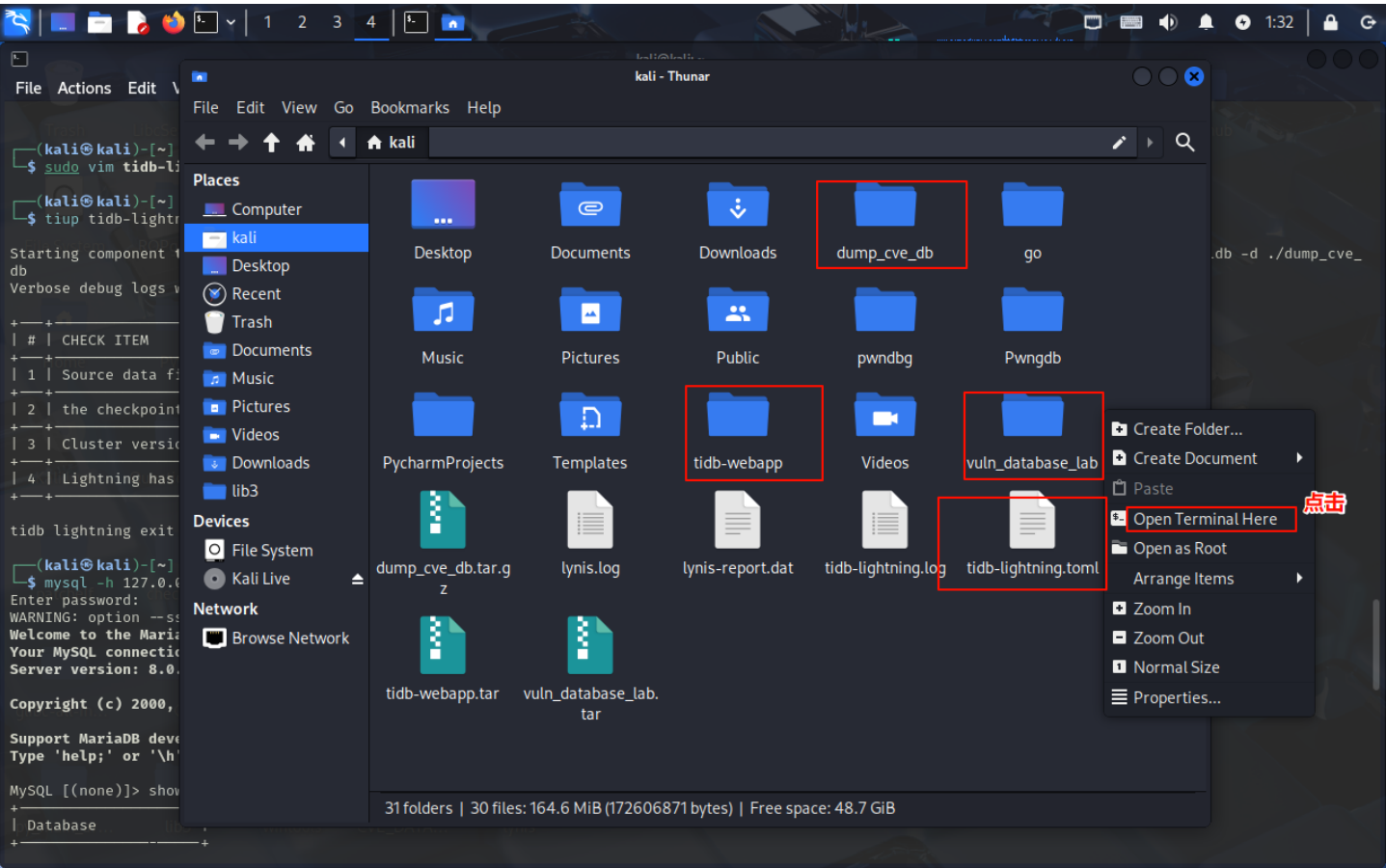
```
1 tiup playground --tag tidb_test
```

代码块

```
1 tiup client
```



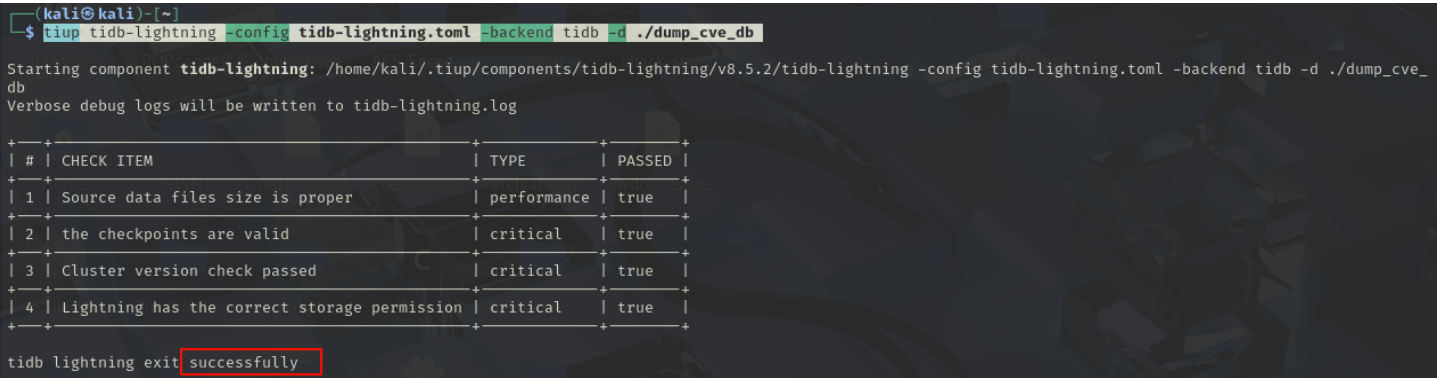
6) 开始迁移



注意，这里一定要 `tidb-webapp`、`dump_cve_db`、`vuln_database_lab`、`tidb-lightning.toml` 在同一个目录下，然后在这个终端执行：

代码块

```
1 tiup tidb-lightning -config tidb-lightning.toml -backend tidb -d ./dump_cve_db
```



执行成功后是这样子的

7) 检验

登陆数据库（必须要在tidb服务端和客户端启动时才可以）：

代码块

```
1 mysql -h 127.0.0.1 -P 4000 -u root -p
```

要输出密码，回车即可，安装我上面的步骤，是没有密码的

代码块

```
1 show schemas;
```

```
(kali㉿kali)-[~]
$ mysql -h 127.0.0.1 -P 4000 -u root -p
Enter password:
WARNING: option --ssl-verify-server-cert is disabled, because of an insecure passwordless login.
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 3940548978
Server version: 8.0.11-TiDB-v8.5.2 TiDB Server (Apache License 2.0) Community Edition, MySQL 8.0 compatible

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> show schemas;
+-----+
| Database |
+-----+
| INFORMATION_SCHEMA |
| METRICS_SCHEMA |
| PERFORMANCE_SCHEMA |
| cve_db |
| lightning_task_info |
| mysql |
| sys |
| test |
+-----+
8 rows in set (0.008 sec)
```

代码块

```
1 use cve_db
2 show tables;
```

```

MySQL [(none)]> use cve_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [cve_db]> show tables;
+-----+
| Tables_in_cve_db |
+-----+
| CVEEntry          |
| CVETag            |
| Configuration     |
| Description        |
| Metric            |
| ParsedVector      |
| Reference          |
| VectorEmbedding   |
| VendorComment     |
| Weakness          |
| category_lookup   |
| cpe_entries       |
| cve_category_match |
| permissions       |
| test              |
| user_audit_log    |
| users             |
| vuln_impact_types_per_year |
| vuln_stats_per_year |
| vuln_types_per_year |
| vulnerability_type_stats |
+-----+
21 rows in set (0.000 sec)

```

代码块

```
1 select * from CVEEntry limit 10;
```

```

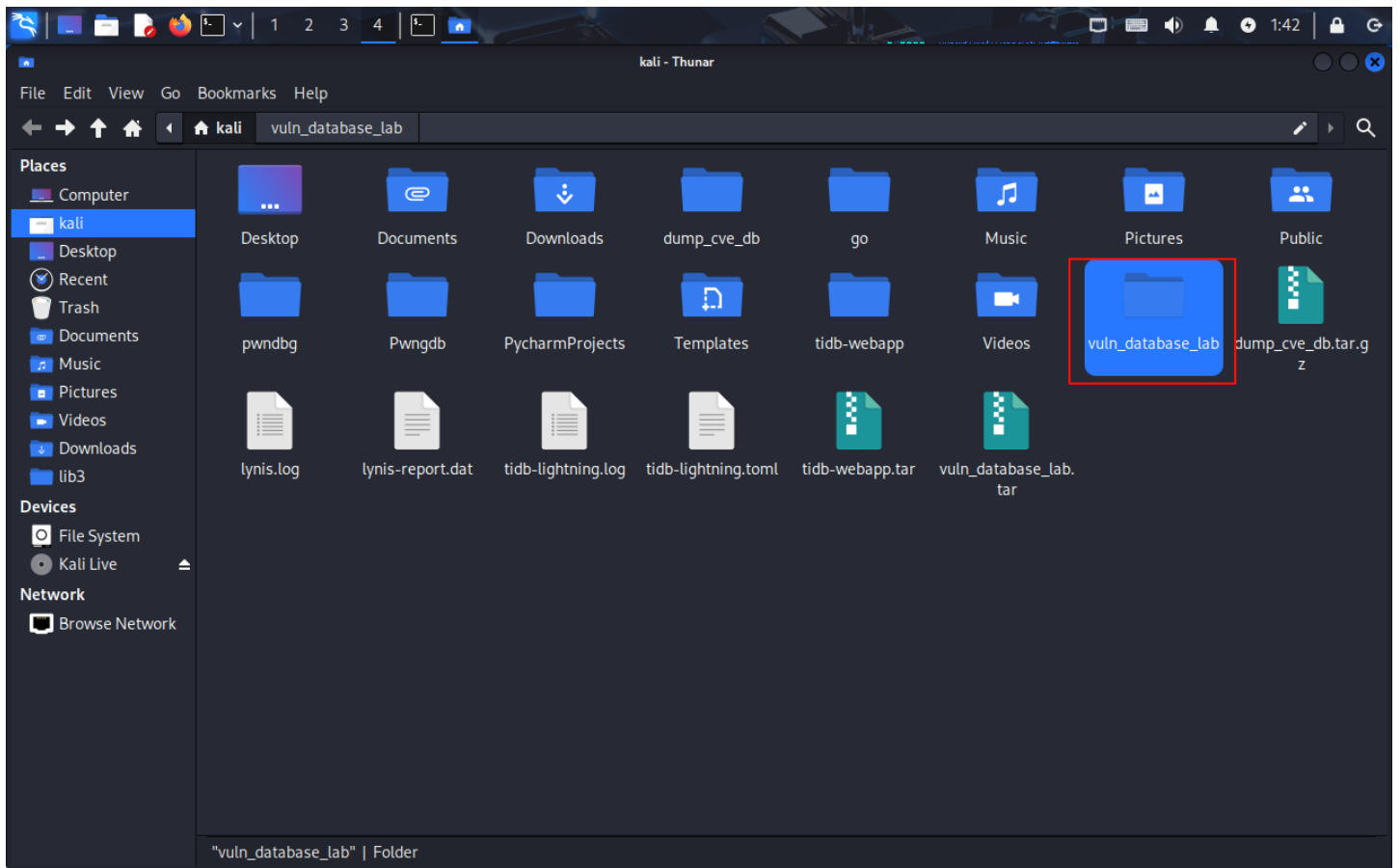
MySQL [cve_db]> select * from CVEEntry limit 10;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id      | sourceIdentifier | published      | lastModified      | vulnStatus | evaluatorComment | evaluatorImpact | evaluatorSolution |
| cisaActionDue | cisaExploitAdd | cisaRequiredAction | cisaVulnerabilityName |             |                  |                 |                   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| CVE-1999-0001 | cve@mitre.org   | 1999-12-30 05:00:00.000 | 2024-11-20 23:27:31.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0002 | cve@mitre.org   | 1998-10-12 04:00:00.000 | 2024-11-20 23:27:34.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0003 | cve@mitre.org   | 1998-04-01 05:00:00.000 | 2024-11-20 23:27:35.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0004 | cve@mitre.org   | 1997-12-16 05:00:00.000 | 2024-11-20 23:27:35.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0005 | cve@mitre.org   | 1998-07-20 04:00:00.000 | 2024-11-20 23:27:35.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0006 | cve@mitre.org   | 1998-07-14 04:00:00.000 | 2024-11-20 23:27:35.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0007 | cve@mitre.org   | 1998-06-26 04:00:00.000 | 2024-11-20 23:27:36.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0008 | cve@mitre.org   | 1998-06-08 04:00:00.000 | 2024-11-20 23:27:36.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0009 | cve@mitre.org   | 1998-04-08 04:00:00.000 | 2024-11-20 23:27:36.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
| CVE-1999-0010 | cve@mitre.org   | 1998-04-08 04:00:00.000 | 2024-11-20 23:27:36.000 | Modified   | NULL             | NULL           | NULL              |
| NULL        | NULL            | NULL              | NULL              |            | NULL             | NULL           | NULL              |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
10 rows in set (0.015 sec)

```

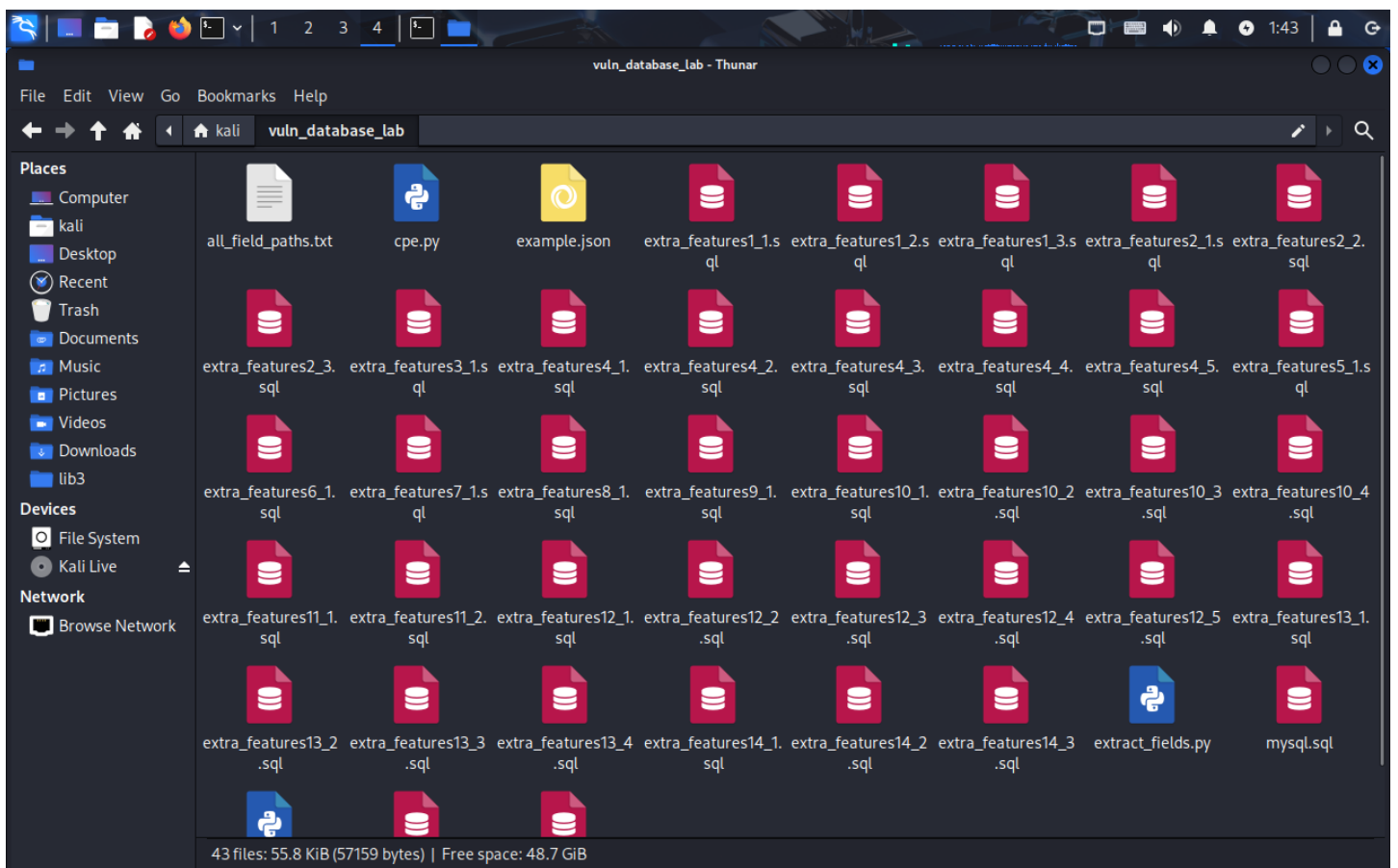
可以看到这个就算是导入成功了

8) 导入视图

回到这个页面



进入vuln_database_lab



可以看到这里很多sql脚本

Open Terminal Here (在这里打开终端)，然后输入：

代码块 `pwd`

```
(kali㉿kali)-[~/vuln_database_lab]
$ pwd
/home/kali/vuln_database_lab
```

要记住这个路径

登陆tidb数据库，执行（但是要记住这里的路径换成前面 `pwd` 出来的路径）

```
MySQL [cve_db]> source /home/kali/vuln_database_lab/extra_features1_3.sql
Query OK, 0 rows affected (0.061 sec)
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features1_3.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features2_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features2_2.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features2_3.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features3_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features4_3.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features4_4.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features4_5.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features5_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features6_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features7_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features9_1.sql
```

代码块

```
1 source /home/kali/vuln_database_lab/extra_features12_4.sql
```

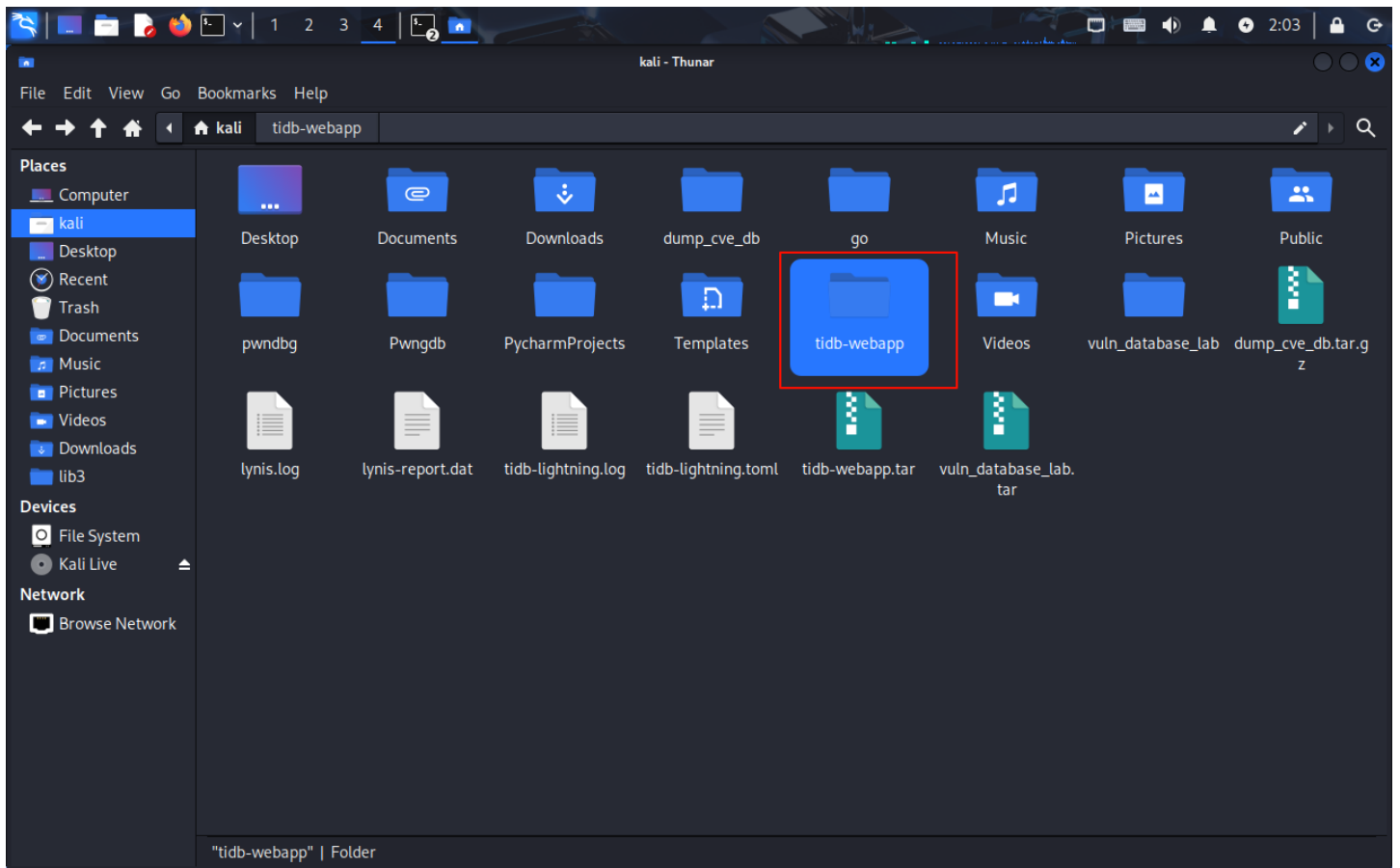
代码块

```
1 source /home/kali/vuln_database_lab/extra_features12_5.sql
```

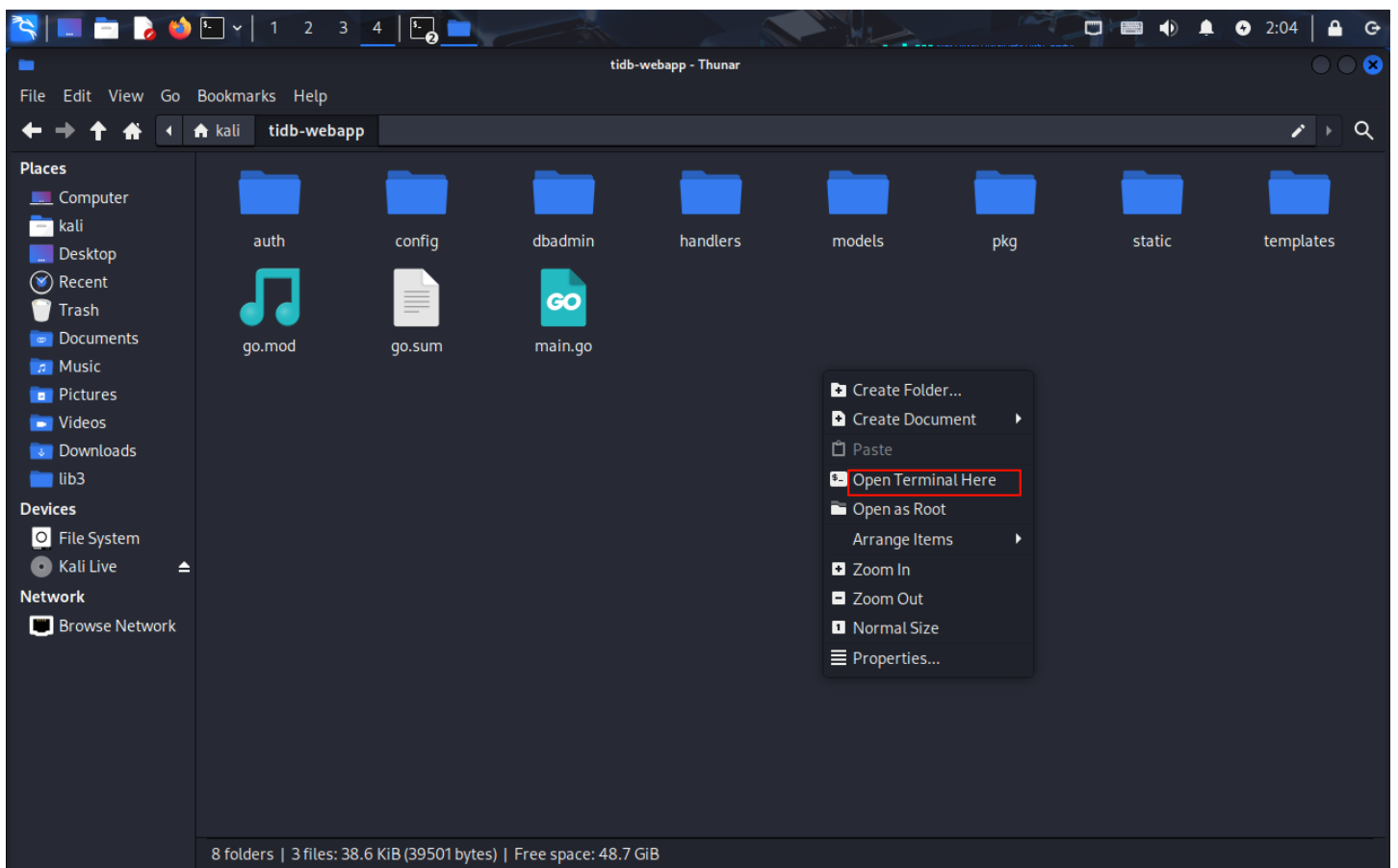
到这里你就基本配置好了，下一步可以启动程序了

9) 启动go程序

回到原来解压出来三个文件夹的路径，进入tidb-webapp



Open Terminal Here



输入：


```
1 go run .
```

```
kali@kali: ~/tidb-webapp
File Actions Edit View Help
(kali@kali)~/tidb-webapp
$ go run .
go: downloading gorm.io/gorm v1.26.1
go: downloading gorm.io/driver/mysql v1.5.7
go: downloading github.com/gin-contrib/sse v1.1.0
go: downloading golang.org/x/net v0.39.0
go: downloading github.com/go-playground/validator/v10 v10.26.0
go: downloading github.com/pelletier/go-toml/v2 v2.2.4
go: downloading google.golang.org/protobuf v1.36.6
go: downloading github.com/go-sql-driver/mysql v1.9.2
go: downloading github.com/jinzhu/norm v1.1.5
go: downloading github.com/jinzhu/inflection v1.0.0
go: downloading github.com/gabriel-vasile/mimetype v1.4.9
go: downloading filippo.io/edwards25519 v1.1.0

2025/06/08 02:04:56 /home/kali/tidb-webapp/models/cve.go:43 Error 1170 (42000): BLOB/TEXT column 'cve_id' used in key specification without a key length
[5.630ms] [rows:0] CREATE TABLE `descriptions` (`cve_id` longtext,`lang` longtext,`value` longtext,CONSTRAINT `fk_CVEEntry_descriptions` FOREIGN KEY (`cve_id`) REFERENCES `CVEEntry`(`id`) ON DELETE CASCADE)
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:   export GIN_MODE=release
- using code:  gin.SetMode(gin.ReleaseMode)

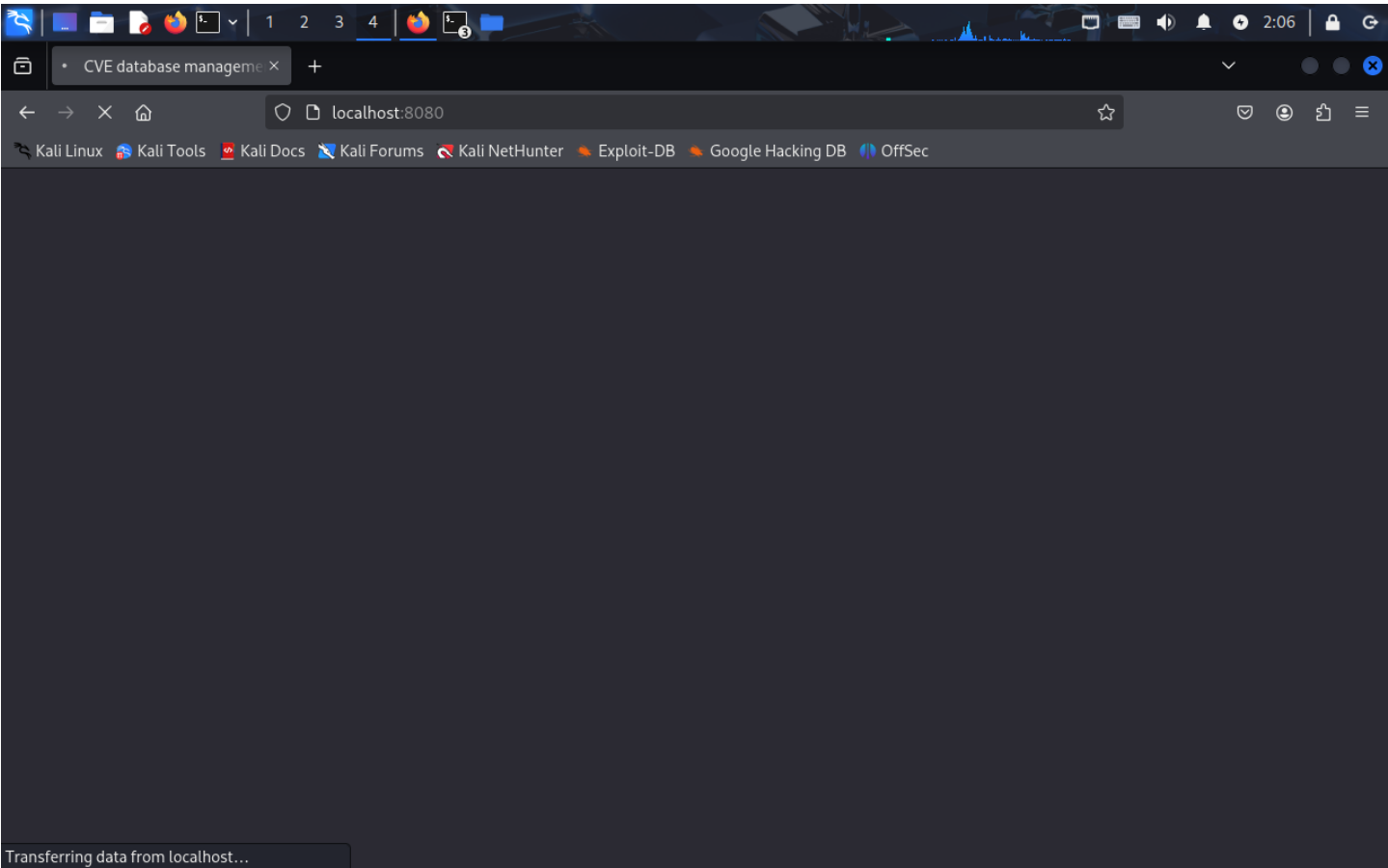
[GIN-debug] POST    /api/auth/login    → tidb-webapp/auth.(*authHandler).Login-fm (3 handlers)
[GIN-debug] POST    /api/auth/register → tidb-webapp/auth.(*authHandler).Register-fm (3 handlers)
[GIN-debug] Loaded HTML Templates (32):
- check.html
- cve_high_risk_details.html
- db_admin.html
- verify.html
- version.html
- vulnerabilities_details.html
- about.html
- cvebyyearmonth_detail.html
- product_vulnerabilities_details.html
- search.html
- vulnerabilities_type_total_details.html
- register.html
- vulnerabilities_type_details.html
- cvss_score.html
- permission-denied.html
```

```
kali@kali: ~/tidb-webapp
File Actions Edit View Help

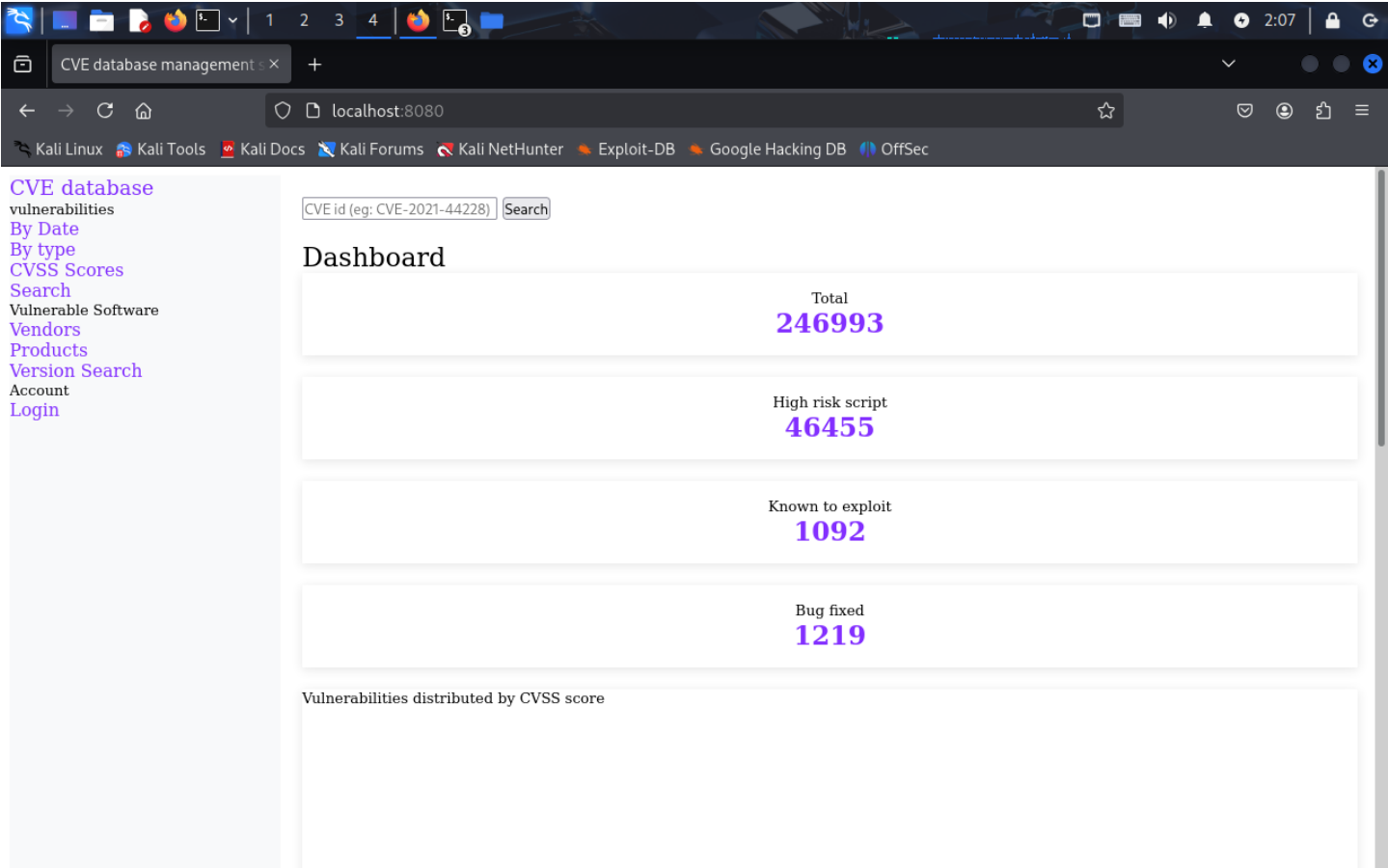
[GIN-debug] GET    /login.html        → main.registerRoutes.func24 (3 handlers)
[GIN-debug] GET    /register.html      → main.registerRoutes.func25 (3 handlers)
[GIN-debug] GET    /db_admin.html     → main.registerRoutes.func26 (3 handlers)
[GIN-debug] GET    /about.html        → main.registerRoutes.func27 (3 handlers)
[GIN-debug] GET    /api/vendors       → main.registerRoutes.GetVendors.func34 (3 handlers)
[GIN-debug] GET    /api/vendors/suggest → main.registerRoutes.VendorSuggest.func35 (3 handlers)
[GIN-debug] GET    /api/vendors/products/:vendor → main.registerRoutes.GetVendorProducts.func36 (3 handlers)
[GIN-debug] GET    /api/vendors/vulnerabilities/:vendor → main.registerRoutes.GetVendorVulnerabilities.func37 (3 handlers)
[GIN-debug] GET    /api/products/cveid → main.registerRoutes.GetProductVulnerabilities.func38 (3 handlers)
[GIN-debug] GET    /api/products      → main.registerRoutes.GetProducts.func39 (3 handlers)
[GIN-debug] GET    /api/products/suggest → main.registerRoutes.ProductSuggest.func40 (3 handlers)
[GIN-debug] GET    /search_version_cve → main.registerRoutes.SearchVersionCVEHandler.func41 (3 handlers)
[GIN-debug] GET    /advanced-search   → main.registerRoutes.NewAdvancedSearchHandlerRaw.func42 (3 handlers)
[GIN-debug] GET    /vulnerabilities_details → main.registerRoutes.GetCVEIDListHandler.func43 (3 handlers)
[GIN-debug] GET    /vulnerabilities_total_details → main.registerRoutes.GetCVEIDsByTypeHandler.func44 (3 handlers)
[GIN-debug] GET    /vulnerabilities_type_details → main.registerRoutes.GetCVEIDListtypeHandler.func45 (3 handlers)
[GIN-debug] GET    /vulnerabilities_type_total_details → main.registerRoutes.GetCVEIDsByTypeHandler.func46 (3 handlers)
[GIN-debug] GET    /api/cveids        → main.registerRoutes.GetCVEList.func47 (3 handlers)
[GIN-debug] GET    /api/high_basescore_cves → main.registerRoutes.NewHighBaseScoreHandler.func48 (3 handlers)
[GIN-debug] GET    /api/cisa_due_cves → main.registerRoutes.NewCISAActionDueHandler.func49 (3 handlers)
[GIN-debug] GET    /api/bug_cves      → main.registerRoutes.NewBUGHandler.func50 (3 handlers)
[GIN-debug] GET    /api/stats         → main.getStats (3 handlers)
[GIN-debug] GET    /api/type-year-stats → main.getTypeYearStats (3 handlers)
[GIN-debug] GET    /api/base-score-distribution → main.getBaseScoreDistribution (3 handlers)
[GIN-debug] GET    /api/exploitability-score-distribution → main.getexploitabilityScoreDistribution (3 handlers)
[GIN-debug] GET    /api/impact-score-distribution → main.getImpactScoreDistribution (3 handlers)
[GIN-debug] GET    /api/vulnerabilities-by-type → main.getvulnerabilitiesbytype (3 handlers)
[GIN-debug] GET    /api/cve           → main.getCveByYearMonth (3 handlers)
[GIN-debug] GET    /api/cvss_distribution → main.GetCVSSDistribution (3 handlers)
[GIN-debug] GET    /api/cvss_cves     → main.getCVESBySeverityRange (3 handlers)
[GIN-debug] GET    /api/auth/protected → tidb-webapp/handlers.ProtectedHandler (4 handlers)
[GIN-debug] POST   /api/auth/logout   → tidb-webapp/handlers.LogoutHandler (4 handlers)
[GIN-debug] GET    /api/db/tables     → tidb-webapp/dbadmin.(*App).ListTables-fm (4 handlers)
[GIN-debug] GET    /api/db/:table     → tidb-webapp/dbadmin.(*App).ListRecords-fm (4 handlers)
[GIN-debug] GET    /api/db/:table/:id → tidb-webapp/dbadmin.(*App).GetRecord-fm (4 handlers)
[GIN-debug] POST   /api/db/:table     → tidb-webapp/dbadmin.(*App).CreateRecord-fm (4 handlers)
[GIN-debug] PUT    /api/db/:table/:id → tidb-webapp/dbadmin.(*App).UpdateRecord-fm (4 handlers)
[GIN-debug] DELETE /api/db/:table/:id → tidb-webapp/dbadmin.(*App).DeleteRecord-fm (4 handlers)
[GIN-debug] GET    /api/db/:table/count → tidb-webapp/dbadmin.(*App).CountRecords-fm (4 handlers)
[GIN-debug] [WARNING] You trusted all proxies, this is NOT safe. We recommend you to set a value.
Please check https://pkg.go.dev/github.com/gin-gonic/gin#readme-don-t-trust-all-proxies for details.
[GIN-debug] Listening and serving HTTP on :8080
```

可以看到这里就表示可以用了

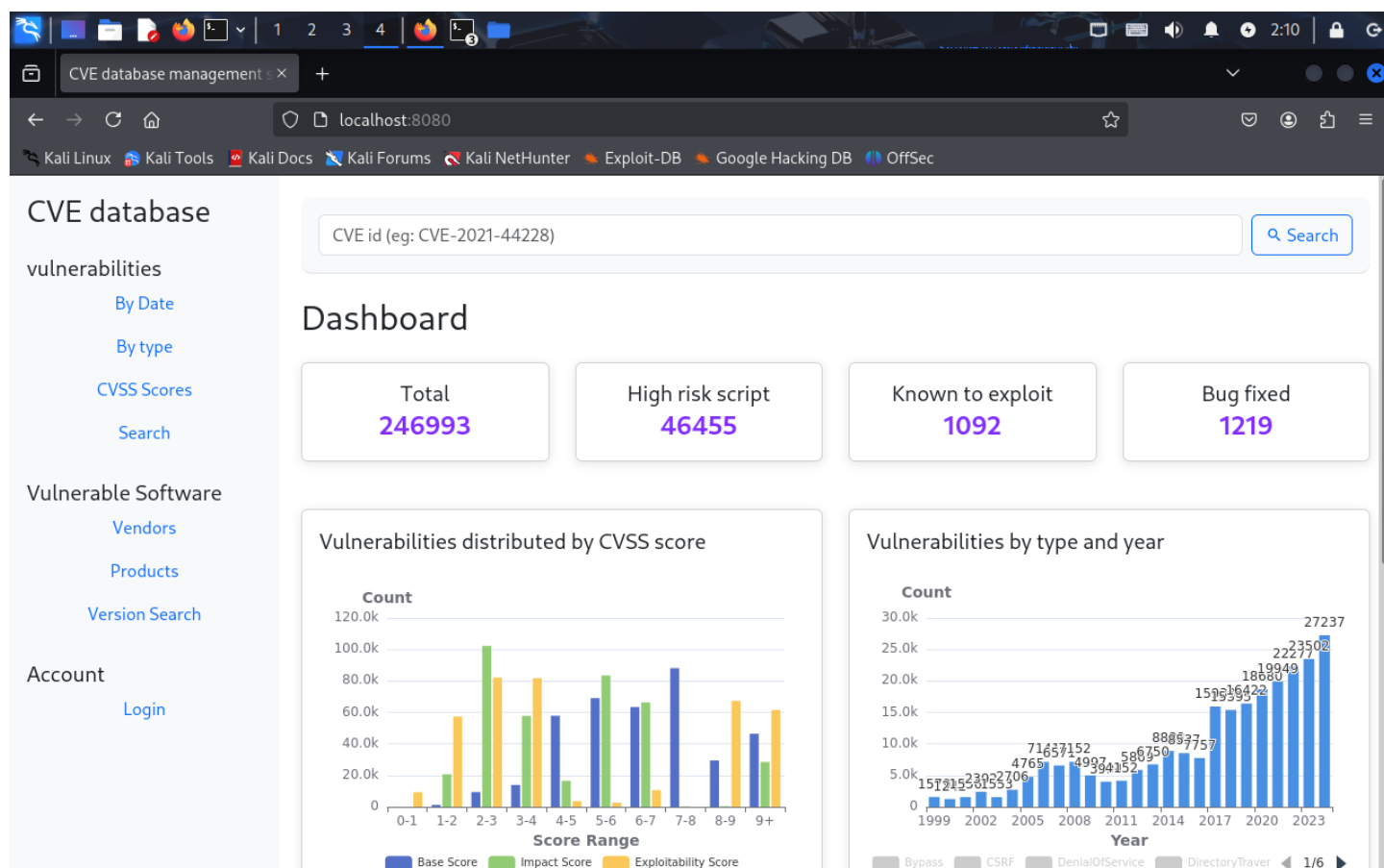
打开浏览器，输入：<http://localhost:8080>



稍等一会，要先加载一些数据



这里没关系，只是CSS样式还没有加载出来，多刷新几次就行了



到这里就已经部署完成了，接下来就是Caddy反向代理（可以使用https）。

3.4 Caddy代理部署

3.4.1 官方路径安装Caddy:<https://caddyserver.com/docs/install#debian-ubuntu-raspbian>

代码块

```
1 sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
```

代码块

```
1 curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
```

代码块

```
1 curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/sources.list.d/caddy-stable.list
```

```
(kali㉿kali)-[~]
$ curl -sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
File '/usr/share/keyrings/caddy-stable-archive-keyring.gpg' exists. Overwrite? (y/N) y
这里是我已经安装过了，才有这个
(kali㉿kali)-[~]
$ curl -sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/sources.list.d/caddy-stable.list
# Source: Caddy
# Site: https://github.com/caddyserver/caddy
# Repository: Caddy / stable
# Description: Fast, multi-platform web server with automatic HTTPS

deb [signed-by=/usr/share/keyrings/caddy-stable-archive-keyring.gpg] https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version main
deb-src [signed-by=/usr/share/keyrings/caddy-stable-archive-keyring.gpg] https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version main
```

代码块

```
1 sudo apt update
```

```
(kali㉿kali)-[~]
$ sudo apt update
Hit:1 https://mirrors.aliyun.com/kali kali-rolling InRelease
Get:2 https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version InRelease [8266 B]
Get:3 https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version/main amd64 Packages [4071 B]
Fetched 12.3 kB in 1s (9660 B/s)
267 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

代码块

```
1 sudo apt install caddy
```

```
(kali㉿kali)-[~]
$ sudo apt install caddy
Installing:
caddy

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 267
Download size: 15.0 MB
Space needed: 42.0 MB / 53.5 GB available

Get:1 https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version/main amd64 caddy amd64 2.10.0 [15.0 MB]
Fetched 15.0 MB in 4s (3533 kB/s)
Selecting previously unselected package caddy.
(Reading database ... 557813 files and directories currently installed.)
Preparing to unpack .../caddy_2.10.0_amd64.deb ...
Unpacking caddy (2.10.0) ...
Setting up caddy (2.10.0) ...
caddy.service is a disabled or a static unit, not starting it.
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.2.3) ...
```

3.4.2 配置文件

代码块

```
1 sudo vim /etc/caddy/Caddyfile
```



```
File Actions Edit View Help
#
# To use your own domain name (with automatic HTTPS), first make
# sure your domain's A/AAAA DNS records are properly pointed to
# this machine's public IP, then replace ":80" below with your
# domain name.
#
#::80 {
#   # Set this path to your site's directory.
#   #root * /usr/share/caddy
#
#   # Enable the static file server.
#   #file_server
#
#   # Another common task is to set up a reverse proxy:
#   # reverse_proxy localhost:8080
#
#   # Or serve a PHP site through php-fpm:
#   # php_fastcgi localhost:9000
#}
#
# Refer to the Caddy docs for more information:
# https://caddyserver.com/docs/caddyfile
tidb-webapp.local
  tls internal
  encode zstd gzip
  reverse_proxy 127.0.0.1:8080 {
    header_up X-Forwarded-Proto {scheme}
    header_up X-Real-IP {remote-host}
    header_up X-Forwarded-For {forwarded_for}
  }
  header {
    Access-Control-Allow-Origin https://tidb-webapp.local
    Access-Control-Allow-Credentials true
    Access-Control-Allow-Methods GET,POST,OPTIONS
    Access-Control-Allow-Headers Content-Type,Authorization
    Vary Origin
  }
}
```

是这样的，退出vim的编辑模式（多按几次 `esc` ），然后输入 `:wq`

3.4.3 检验Caddyfile文件的合法性

代码块

```
1 sudo caddy validate --config /etc/caddy/Caddyfile
```

```
(kali@kali)-[~]
$ sudo caddy validate --config /etc/caddy/Caddyfile
2025/06/08 06:57:14.173 INFO using config from file {"file": "/etc/caddy/Caddyfile"}
2025/06/08 06:57:14.173 WARN caddyfile Unnecessary header_up X-Forwarded-Proto: the reverse proxy's default behavior is to pass headers to the upst
ream
2025/06/08 06:57:14.174 INFO adapted config to JSON {"adapter": "caddyfile"}
2025/06/08 06:57:14.174 WARN Caddyfile input is not formatted; run 'caddy fmt --overwrite' to fix inconsistencies {"adapter": "caddyfile", "file": "/e
tc/caddy/Caddyfile", "line": 12}
2025/06/08 06:57:14.175 INFO tls.cache.maintenance started background certificate maintenance {"cache": "0xc000524200"}
2025/06/08 06:57:14.183 INFO http.auto_https server is listening only on the HTTPS port but has no TLS connection policies; adding one to enable TLS {"se
rver_name": "srv0", "https_port": 443}
2025/06/08 06:57:14.183 INFO http.auto_https enabling automatic HTTP→HTTPS redirects {"server_name": "srv0"}
2025/06/08 06:57:14.184 INFO tls.cache.maintenance stopped background certificate maintenance {"cache": "0xc000524200"}
Valid configuration
```

输出: `Valid configuration` 即表示合法

3.4.4 配置本地 `/etc/hosts` 文件

代码块

```
1 sudo vim /etc/hosts
```

在最前面添加: `127.0.0.1 tidb-webapp.local`

代码块 `echo "127.0.0.1 tidb-webapp.local" | sudo tee -a /etc/hosts`

```
127.0.0.1 tidb-webapp.local
127.0.0.1 localhost
127.0.1.1 kali.kali kali

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
127.0.0.1 test.local
```

测试 `/etc/hosts` 文件是否配置成功

代码块

```
1 ping tidb-webapp.local
```

```
(kali㉿kali)-[~]
$ ping tidb-webapp.local
PING tidb-webapp.local (127.0.0.1) 56(84) bytes of data.
64 bytes from tidb-webapp.local (127.0.0.1): icmp_seq=1 ttl=64 time=1.25 ms
64 bytes from tidb-webapp.local (127.0.0.1): icmp_seq=2 ttl=64 time=0.064 ms
64 bytes from tidb-webapp.local (127.0.0.1): icmp_seq=3 ttl=64 time=0.022 ms
64 bytes from tidb-webapp.local (127.0.0.1): icmp_seq=4 ttl=64 time=0.022 ms
64 bytes from tidb-webapp.local (127.0.0.1): icmp_seq=5 ttl=64 time=0.017 ms
^C
— tidb-webapp.local ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4062ms
rtt min/avg/max/mdev = 0.017/0.275/1.250/0.487 ms
```

3.5.5 配置 `/var/lib/caddy`

有些系统不会自己创建caddy目录，即可以执行下面这个：

代码块

```
1 sudo mkdir -p /var/lib/caddy
2 sudo chown -R caddy:caddy /var/lib/caddy
3 sudo chown -R caddy:caddy /etc/caddy
```

启动caddy服务：

代码块

```
1 sudo systemctl start caddy
```

查看caddy服务的启动状态：

代码块

```
1 sudo systemctl status caddy
```

重新启动caddy:

代码块

```
1 sudo systemctl restart caddy
```

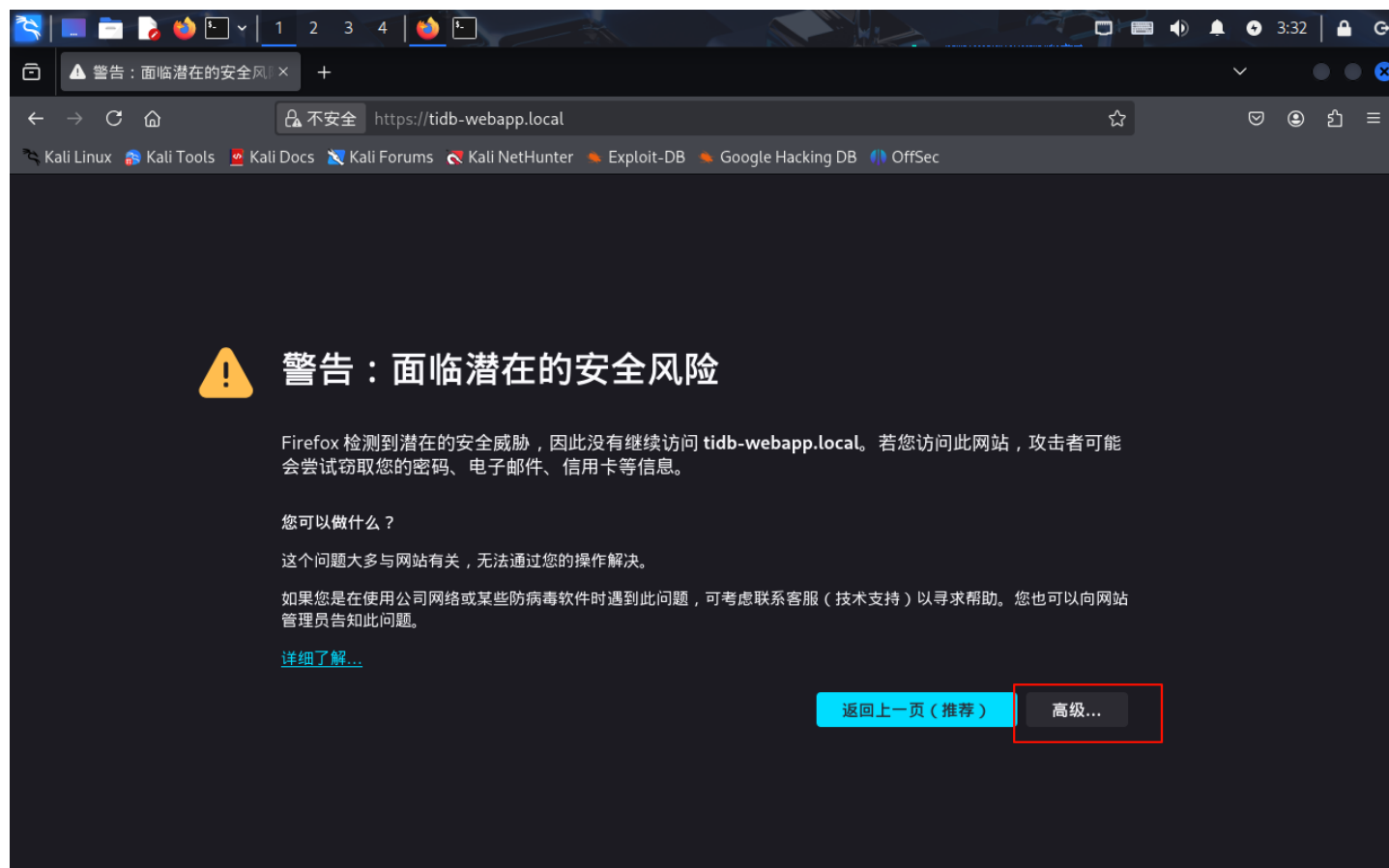
```
(kali@kali)-[~]
$ sudo systemctl start caddy
(kali@kali)-[~]
$ sudo systemctl status caddy
● caddy.service - Caddy
   Loaded: loaded (/usr/lib/systemd/system/caddy.service; disabled; preset: disabled)
   Active: active (running) since Sun 2025-06-08 03:22:40 EDT; 4min 49s ago
 Invocation: 362f37f825e64b949b5f35a29dece688
    Docs: https://caddyserver.com/docs/
   Main PID: 194885 (caddy)
      Tasks: 13 (limit: 14152)
     Memory: 16.8M (peak: 21.5M)
        CPU: 116ms
    CGroup: /system.slice/caddy.service
            └─194885 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile

Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1054487,"logger":"http.log","msg":"server running","name":"remaining_auto_https_redirec
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1054523,"logger":"http","msg":"enabling automatic TLS certificate management","domains">
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.105626,"msg":"autosaved config (load with --resume flag)","file":"/var/lib/caddy/.confi
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.105689,"msg":"serving initial configuration"}
Jun 08 03:22:40 kali systemd[1]: Started caddy.service - Caddy.
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1070445,"logger":"tls.obtain","msg":"acquiring lock","identifier":"tidb-webapp.local"}
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1075866,"logger":"tls.obtain","msg":"lock acquired","identifier":"tidb-webapp.local"}
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1076338,"logger":"tls.obtain","msg":"obtaining certificate","identifier":"tidb-webapp.l
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.110266,"logger":"tls.obtain","msg":"certificate obtained successfully","identifier":"ti
Jun 08 03:22:40 kali caddy[194885]: {"level":"info","ts":1749367360.1103537,"logger":"tls.obtain","msg":"releasing lock","identifier":"tidb-webapp.local"}
```

3.5.6 使用浏览器访问:

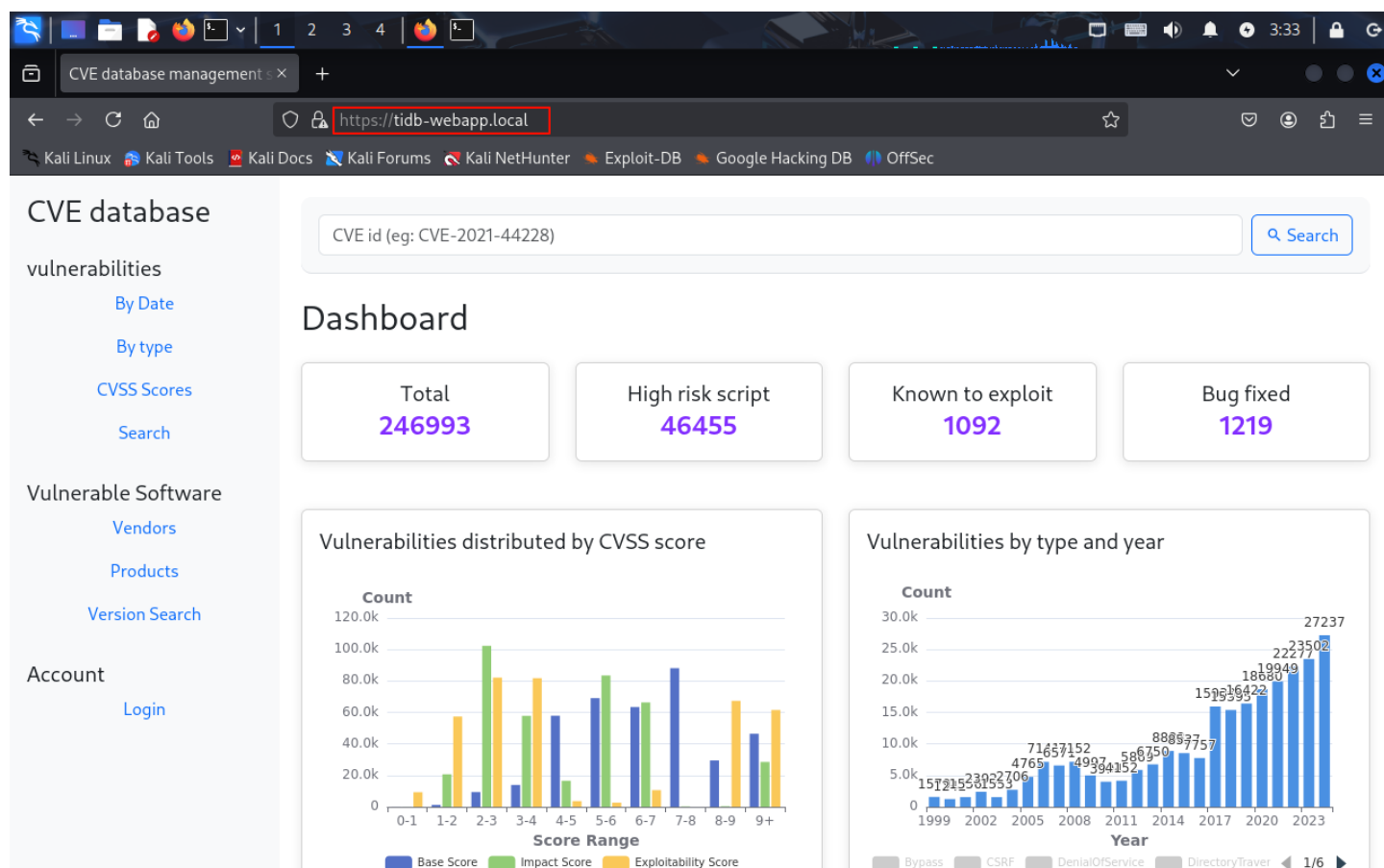
代码块

```
1 https://tidb-webapp.local
```

点击右下角的高级，继续访问即可





可以看到这里已经是可以用https访问了

3.5 CA证书部署

3.5.1 触发证书生成

代码块

```
1 curl -k https://tidb-webapp.local
```

3.5.2 提取证书

对于以 systemd 方式运行的 `caddy.service`，它运行在 `caddy` 用户下，所以根证书应该在：

`/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt`

代码块

```
1 sudo cat /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
```

```
(kali㉿kali)-[~]
$ sudo cat /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
-----BEGIN CERTIFICATE-----
MIIBpDCCAUqgAwIBAgIRANa7UHZB59B84KR7jGrJb2gwCgYIKoZIzj0EAwIwMDEu
MCwGA1UEAxMlQ2FkZkZkTG9jYWwgQXV0aG9vaXR5IC0gMjAyNSBFQ0MgUm9vdAe
Fw0yNTA2MDgwNzIyNDBaFw0yMjAyNSBFQ0MgUm9vdAeBAQMDAxLjAsBgNVBAMTJUNhZGR5
IExvY2FsIEF1dGhvcml0eSAwDgYDVRQTAQH/BAgwBgEB/wIBATAdBgNVHQ4EFgQU1oZ8I30uJz68hXalQobp3RbF
IkIwCgYIKoZIzj0EAwIDSAAwRQIgUJYFzFXKEzluLP8v+MdHvQdtifBVeEhj4Y/T
prrSJ0cCIQDTbQA9ECNnUoWzm11COz+C9zLKcqjOfSai7A+MiHqYBw=
-----END CERTIFICATE-----
```

看一下指纹：

代码块

```
1 sudo openssl x509 -noout -fingerprint -sha256 -in
/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
```

```
(kali㉿kali)-[~]
$ sudo openssl x509 -noout -fingerprint -sha256 -in /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
[sudo] password for kali:
sha256 Fingerprint=:66:4C:1F:DD:F8:11:40:08:D5
```

提取证书：

代码块

```
1 sudo cp /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
/home/kali/caddy_tidb_webapp_local.crt
```

```
(kali㉿kali)-[~]
$ sudo cp /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt /home/kali/caddy_tidb_webapp_local.crt
```

检查证书的权限：

代码块

```
1 ls -liah caddy_tidb_webapp_local.crt
```

```
(kali㉿kali)-[~]
$ ls -liah caddy_tidb_webapp_local.crt
393155 -rw----- 1 root root 631 Jun 8 04:13 caddy_tidb_webapp_local.crt
```

看到这里我们是没有读权限的

授予读权限：

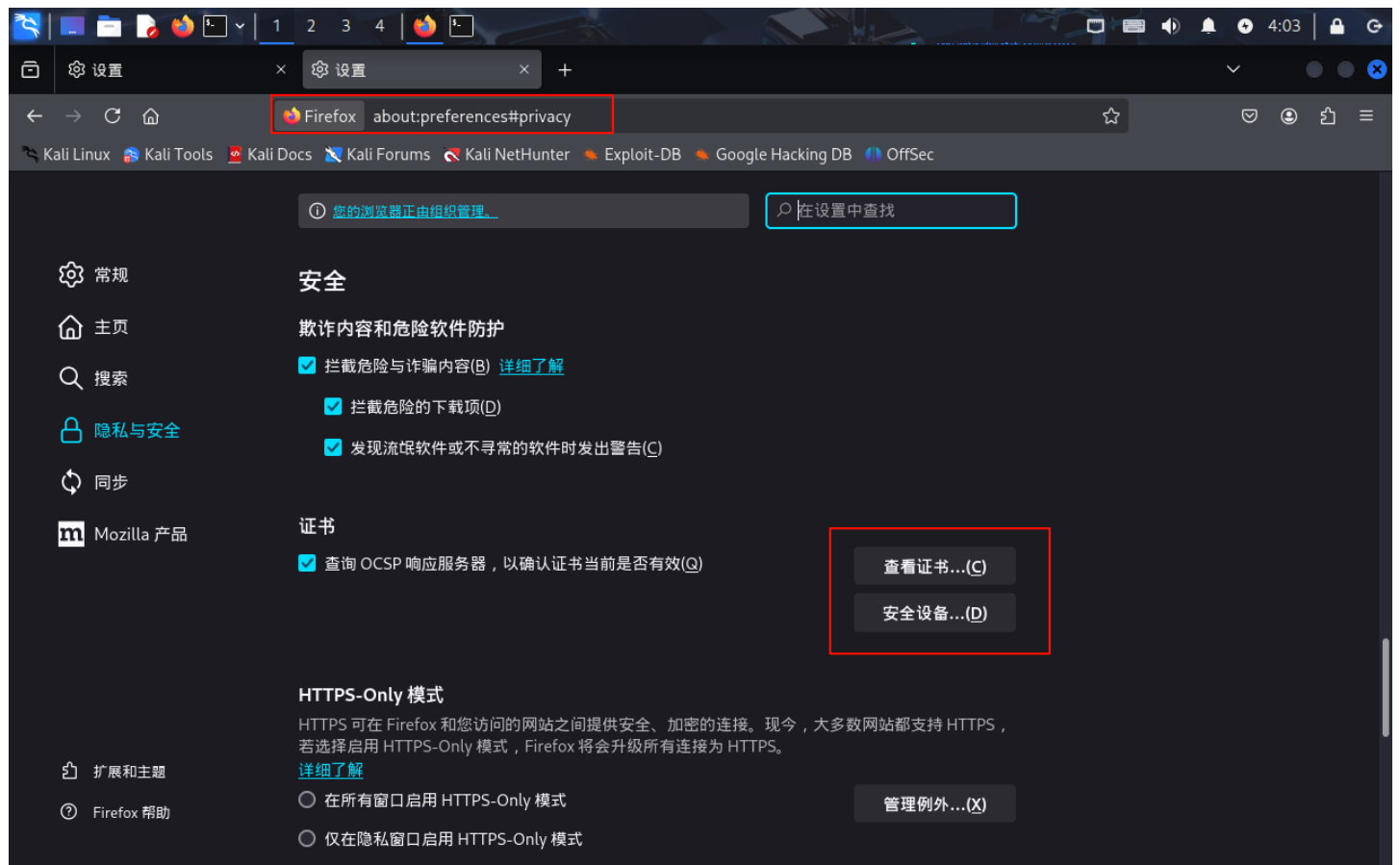
代码块

```
1 sudo chmod 644 caddy_tidb_webapp_local.crt
```

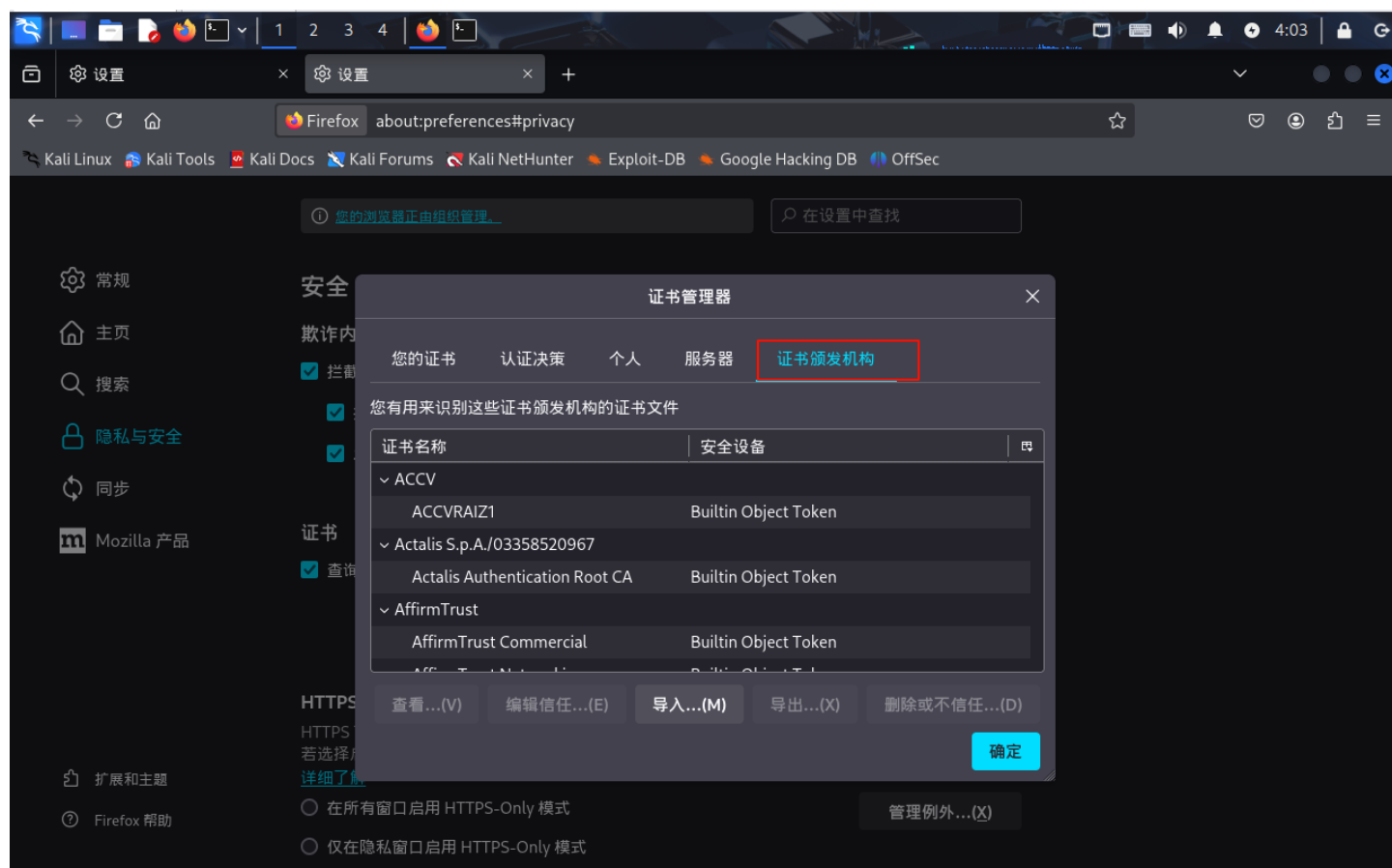
```
(kali@kali)-[~]
$ sudo chmod 644 caddy_tidb_webapp_local.crt
[sudo] password for kali:
(kali@kali)-[~]
$ ls -liah caddy_tidb_webapp_local.crt
393155 -rw-r--r-- 1 root root 631 Jun  8 04:13 caddy_tidb_webapp_local.crt
```

3.5.3 将证书录入浏览器

打开浏览器，并输入：`about:preferences#privacy`



下滑，找到查看证书这个位置，点击查看证书

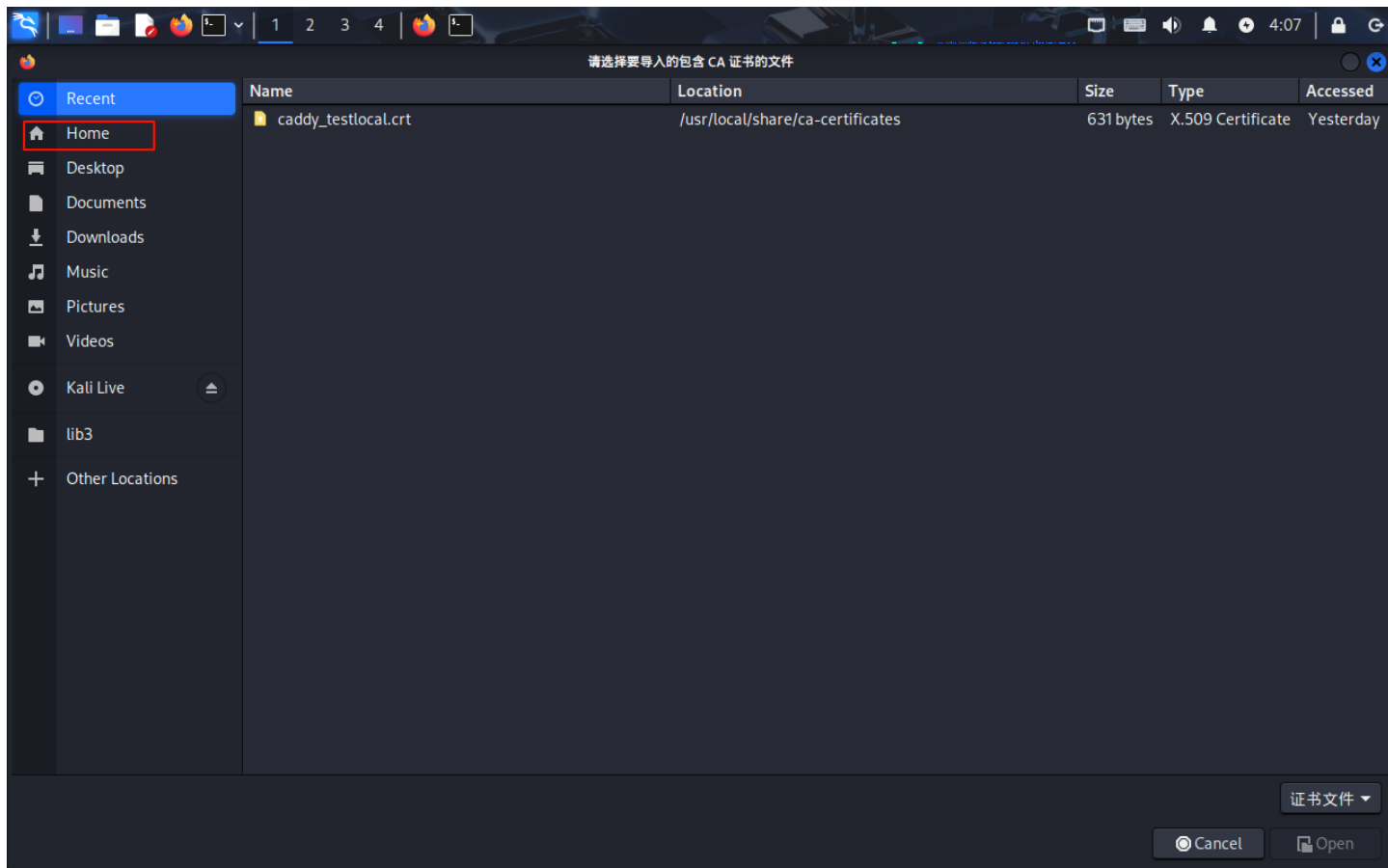


点击导入，我点击后是这样子的，应该是我前面配置过的原因，但是要找到我们前面复制到的那个文件：

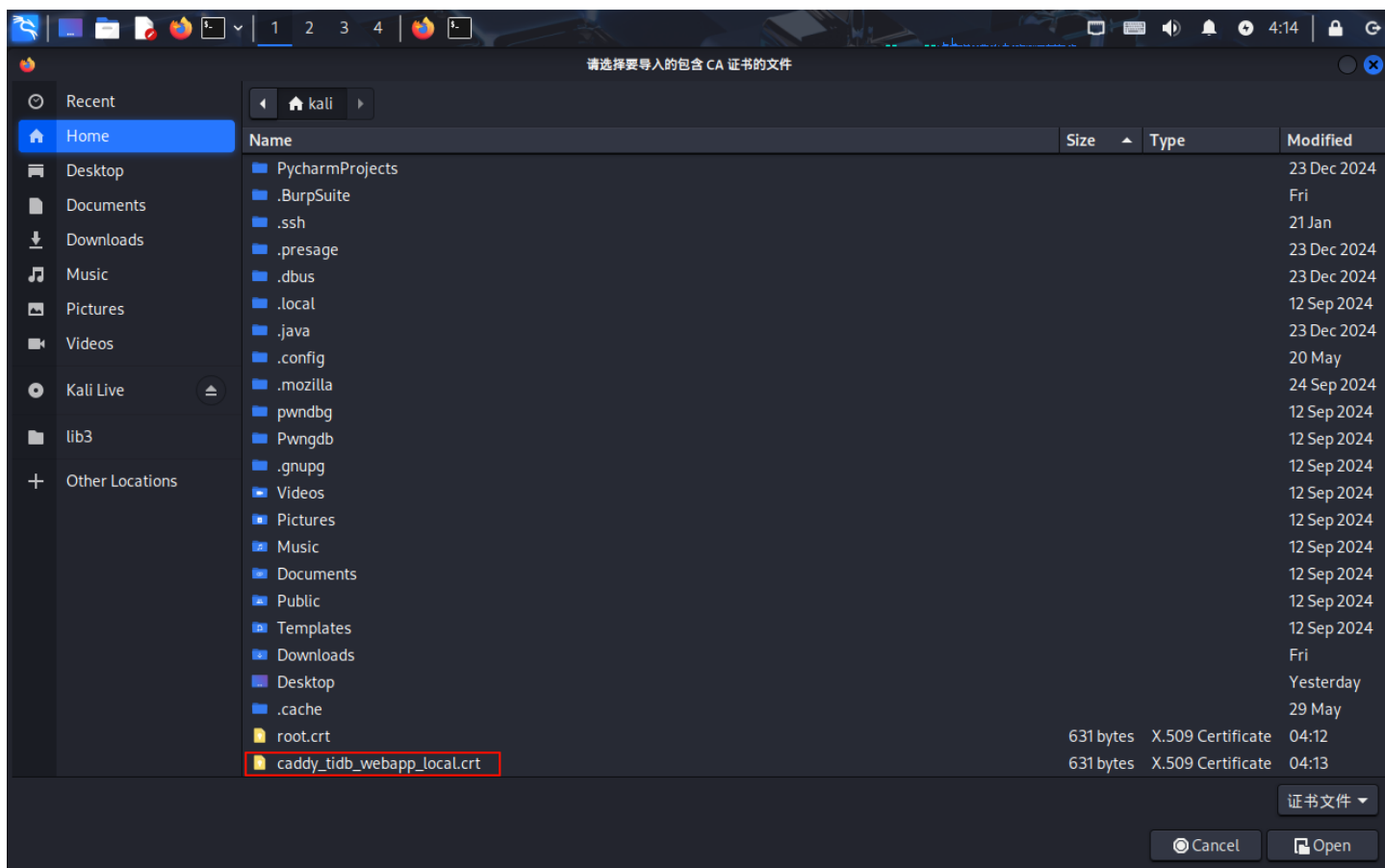
代码块

```
1 sudo cp /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt  
/home/kali/caddy_tidb_webapp_local.crt
```

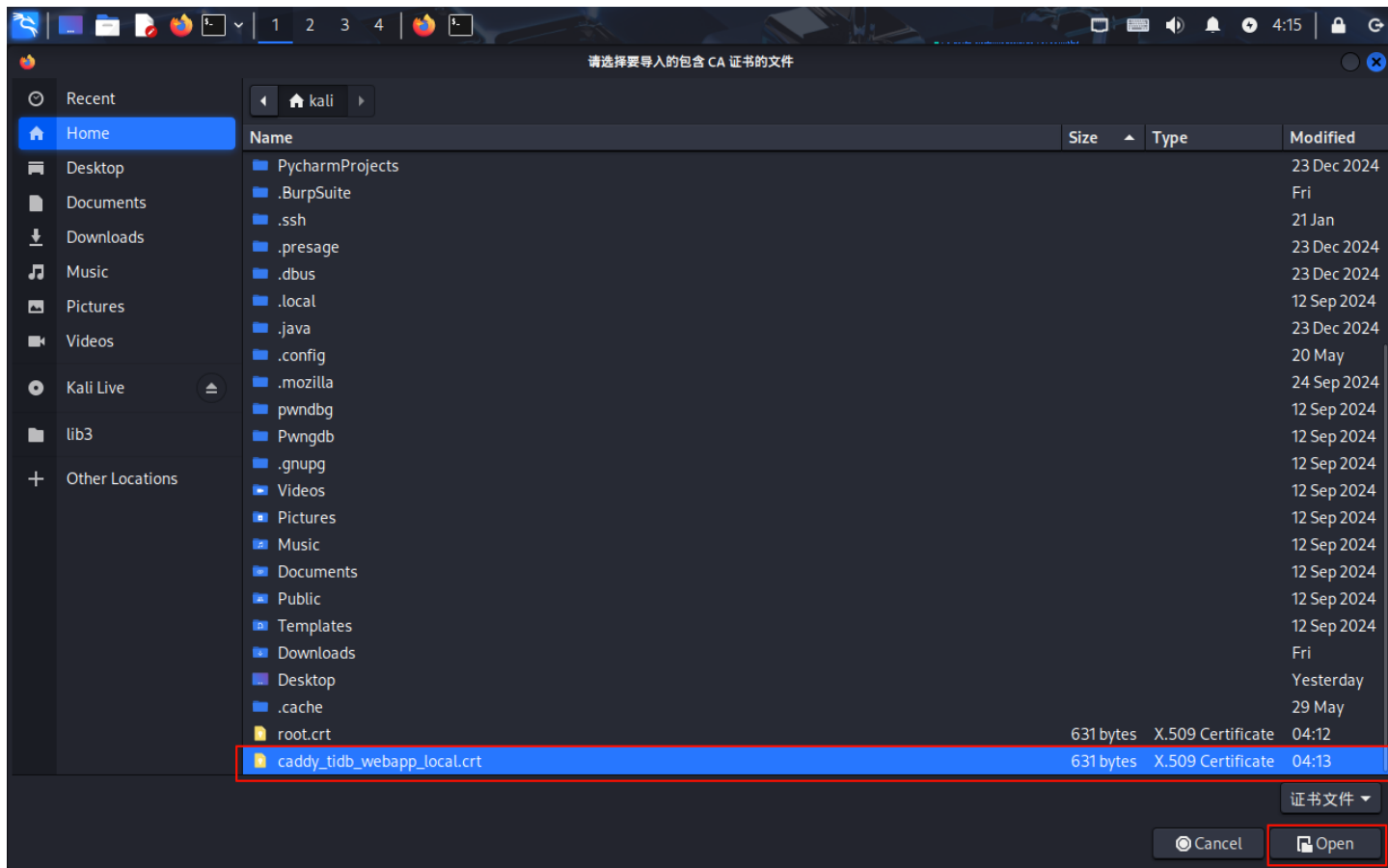
找到 `/home/kali/caddy_tidb_webapp_local.crt`



点击home，往下翻，可以看到caddy_tidb_webapp_local.crt



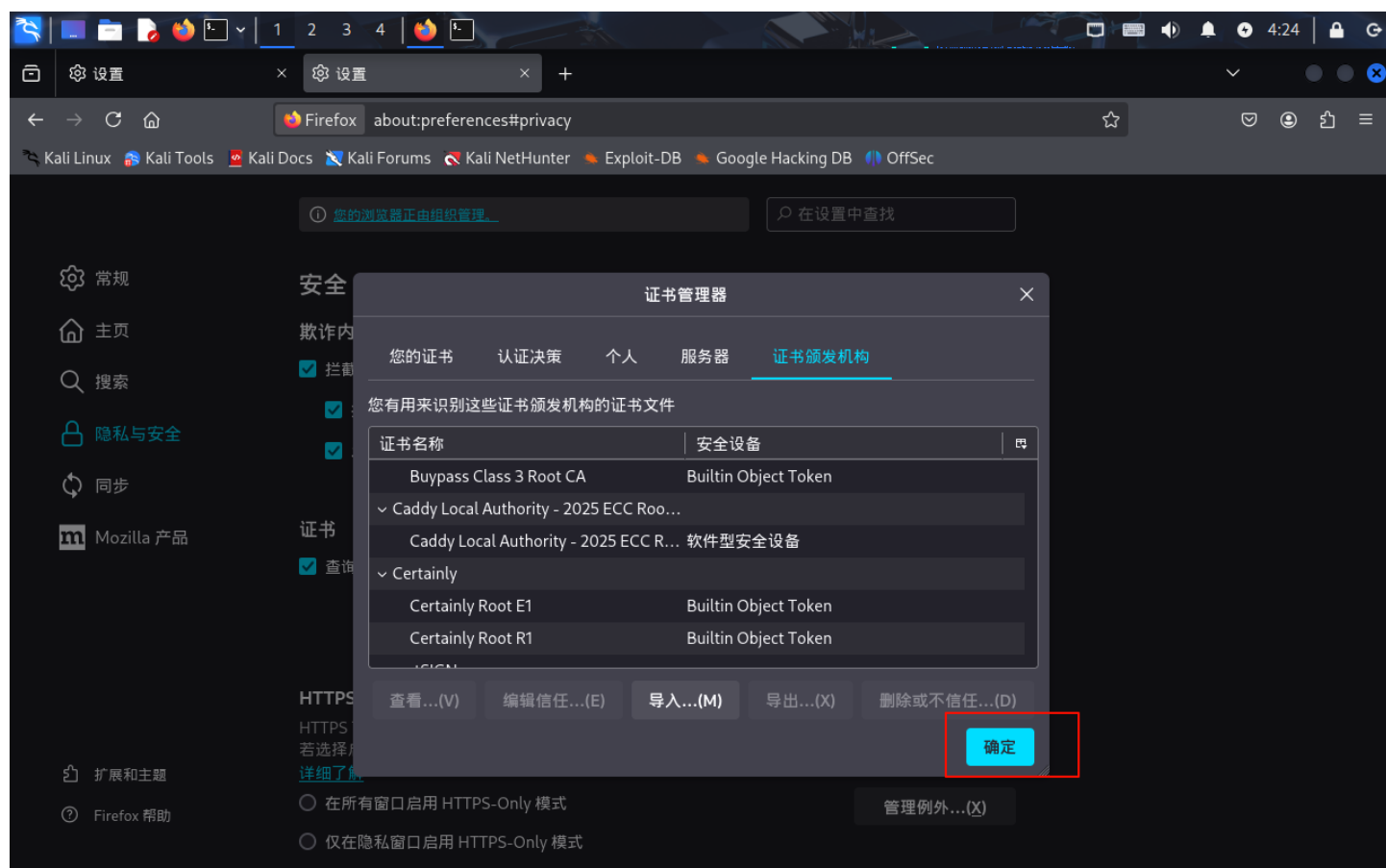
选择这个文件打开即可



点击红框框起来的部分：



再点击确定即可



重新访问，如果有必要的话，可以先清理一下历史记录：

代码块

```
1 https://tidb-webapp.local
```

使用https的话，可能有些css加载慢点，还需要多次刷新

3.5.4 错误排查

如果上面进行完之后，证书显示还是无效的话，那可以检查一下已经导入浏览器的证书的签名和前面用openssl查出来的前面一不一样，如果不一样，那就是配置有问题，需要把已经配置的caddy证书删掉

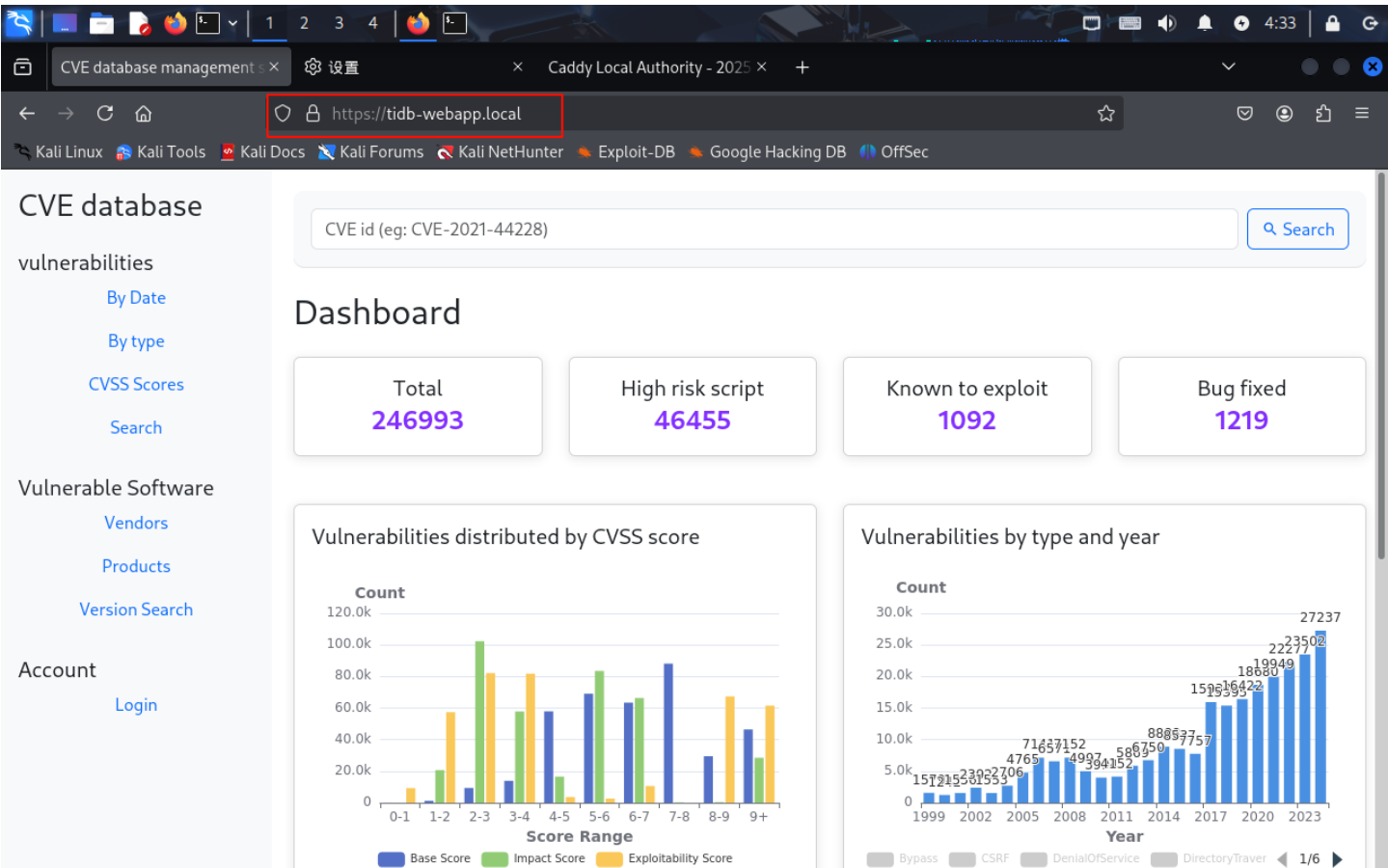


双击蓝色的地方


```
(kali㉿kali)-[~]
$ sudo openssl x509 -noout -fingerprint -sha256 -in /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt

[sudo] password for kali:
sha256 Fingerprint: 66:4C:1F:DD:F8:11:40:08:D5
```

加载出来了，也没有报证书相关的错，到这一步就是完全结束了



3.6 测试用例与结果

3.6.1 单元测试（Unit Test）

- **测试目的：**验证各功能模块（如数据库操作、逻辑处理等）是否工作正常。
- **测试工具：**Go 标准库 `testing`
- **测试准备：**需要把 `/tidb-webapp/auth/handlers.go`、`/tidb-webapp/dbadmin/db_admin_api.go` 中标注的地方取消掉注释，并把前面同样字段的进行注释

```
// 工厂：确保表存在（用纯 SQL），返回处理器
func New(db *gorm.DB, jwtSecret []byte) *authHandler {
    const ddl = `
CREATE TABLE IF NOT EXISTS users (
    id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    email         VARCHAR(255) NOT NULL UNIQUE,
    username      VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARBINARY(60) NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`

    db.Exec(ddl)

    //////////////////////////////////////////test////////////////////////////////////////
    //////////////////////////////////////////test////////////////////////////////////////
    //////////////////////////////////////////test////////////////////////////////////////
    // 这里下面的一段是测试字段，专为测试设计的，在测试的时候需要取消注释，在取消掉注释的同时，需要把上面同函数名的注释掉
    // const ddl = `
    // CREATE TABLE IF NOT EXISTS users (
    //     id            INTEGER PRIMARY KEY AUTOINCREMENT,
    //     email         TEXT NOT NULL UNIQUE,
    //     username      TEXT NOT NULL UNIQUE,
    //     password_hash BLOB NOT NULL,
    //     created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    //     updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    // )`
    // db.Exec(ddl)

    //////////////////////////////////////////test////////////////////////////////////////
    //////////////////////////////////////////test////////////////////////////////////////
    //////////////////////////////////////////test////////////////////////////////////////

    return &authHandler{db: db, jwtSecret: jwtSecret}
}
```

```

//////////////////////////////////test//////////////////////////////////
//////////////////////////////////test//////////////////////////////////
//////////////////////////////////test//////////////////////////////////
// 这里下面的一段是测试字段，专为测试设计的，在测试的时候需要取消注释，在取消掉注释的同时，需要把上面同函数名的注释掉
// func (a *App) ListTables(c *gin.Context) {
//     userID, ok := a.getUserID(c)
//     if !ok {
//         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "missing user_id"})
//         return
//     }
//     var tables []string
//     // SQLite 查询表名改成这个：
//     err := a.DB.Raw("SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%']").Scan(&tables).Error
//     if err != nil {
//         c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
//         return
//     }
//     var allowedTables []string
//     for _, t := range tables {
//         p, _ := a.getPerm(userID, t)
//         if p.allow("select") {
//             allowedTables = append(allowedTables, t)
//         }
//     }
//     c.JSON(http.StatusOK, allowedTables)
// }

// func (a *App) GetRecord(c *gin.Context) {
//     table := c.Param("table")
//     userID, ok := a.getUserID(c)
//     if !ok {
//         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
//         return
//     }
//     p, _ := a.getPerm(userID, table)
//     if !p.allow("select") {
//         c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "permission denied"})
//         return
//     }
//     id, err := parseID(c)
//     if err != nil {
//         c.AbortWithStatusJSON(http.StatusBadRequest, gin.H{"error": err.Error()})
//         return
//     }
//     var result map[string]interface{}
//     // 用 Raw + Scan 替代 Table().Where().First()
//     err = a.DB.Raw("SELECT * FROM "+table+" WHERE id = ?", id).Scan(&result).Error
//     if err != nil {
//         if err == gorm.ErrRecordNotFound {
//             c.AbortWithStatusJSON(http.StatusNotFound, gin.H{"error": "not found"})
//         } else {
//             c.AbortWithStatusJSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
//         }
//         return
//     }
//     c.JSON(http.StatusOK, result)
// }
//////////////////////////////////test//////////////////////////////////
//////////////////////////////////test//////////////////////////////////
//////////////////////////////////test//////////////////////////////////

```

- 执行命令：

代码块

```
1 go test ./... -v
```

- 测试结果截图：

```

(kali@kali)~/tidb-webapp
$ go test ./... -v
?      tidb-webapp      [no test files]
=== RUN   TestRegister
2025/06/08 08:47:35 [Auth] User test@example.com registered successfully
--- PASS: TestRegister (0.05s)
=== RUN   TestLogin
2025/06/08 08:47:35 [Auth] User login@example.com registered successfully
2025/06/08 08:47:35 [Auth] User login@example.com logged in successfully
--- PASS: TestLogin (0.10s)
PASS
ok      tidb-webapp/auth      (cached)
=== RUN   TestGetSecret_FromEnv
--- PASS: TestGetSecret_FromEnv (0.00s)
=== RUN   TestGetSecret_Default
--- PASS: TestGetSecret_Default (0.00s)
PASS
ok      tidb-webapp/config    (cached)
=== RUN   TestListTables
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:   export GIN_MODE=release
- using code:  gin.SetMode(gin.ReleaseMode)

[GIN-debug] GET    /tables              → tidb-webapp/dbadmin.TestListTables.func1 (3 handlers)
[GIN] 2025/06/08 - 08:47:35 | 200 |      256.61µs |      | GET    "/tables"
--- PASS: TestListTables (0.00s)
=== RUN   TestCreateRecord
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:   export GIN_MODE=release
- using code:  gin.SetMode(gin.ReleaseMode)

[GIN-debug] POST   /api/:table          → tidb-webapp/dbadmin.TestCreateRecord.func1 (3 handlers)
[GIN] 2025/06/08 - 08:47:35 | 201 |      112.4µs | 192.0.2.1 | POST   "/api/test_table"
--- PASS: TestCreateRecord (0.00s)
=== RUN   TestGetRecord
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:   export GIN_MODE=release
- using code:  gin.SetMode(gin.ReleaseMode)

[GIN-debug] GET    /api/:table/:id      → tidb-webapp/dbadmin.TestGetRecord.func1 (3 handlers)
[GIN] 2025/06/08 - 08:47:35 | 200 |      102.92µs |      | GET    "/api/test_table/1"
--- PASS: TestGetRecord (0.00s)
PASS
ok      tidb-webapp/dbadmin   (cached)
=== RUN   TestProtectedHandler
=== RUN   TestProtectedHandler/valid_user_info
=== RUN   TestProtectedHandler/missing_user_id
=== RUN   TestProtectedHandler/missing_email
--- PASS: TestProtectedHandler (0.00s)
--- PASS: TestProtectedHandler/valid_user_info (0.00s)
--- PASS: TestProtectedHandler/missing_user_id (0.00s)
--- PASS: TestProtectedHandler/missing_email (0.00s)
=== RUN   TestLogoutHandler
=== RUN   TestLogoutHandler/no_domain
=== RUN   TestLogoutHandler/with_tidb-webapp.local_domain
--- PASS: TestLogoutHandler (0.00s)
--- PASS: TestLogoutHandler/no_domain (0.00s)
--- PASS: TestLogoutHandler/with_tidb-webapp.local_domain (0.00s)
PASS
ok      tidb-webapp/handlers   (cached)
=== RUN   TestCVEEntryCRUD
2025/06/08 08:49:27 /home/kali/tidb-webapp/models/cve_test.go:89 record not found
[0.020ms] [rows:0] SELECT * FROM "CVEEntry" WHERE id = "CVE-1999-0001" ORDER BY "CVEEntry"."id" LIMIT 1
--- PASS: TestCVEEntryCRUD (0.00s)
PASS
ok      tidb-webapp/models     0.007s

```

全部通过测试，这里只测试了一部分

3.6.2 接口测试（API Test）

- 测试目的：验证 RESTful 接口是否能正确返回预期数据。
- 测试工具：curl
- 测试接口示例：

	图表名称	图表/可视化数据实现的功能	api接口	调用的函数	
1	Total	获取 CVE 总数	/api/stats	getStats	获
2	High risk script	获取 CVE 中严重 (CVSS 分数 >= 9) 数量	/api/stats	getStats	获取 (CVS
3	Known to exploit	获取有 CISA 行动到期的 CVE 数量	/api/stats	getStats	获取有自
4	Bug fixed	获取有 CISA 行动到期的 CVE 数量	/api/stats	getStats	获取有自
5	Vulnerabilities distributed by CVSS score	Base Score 分布	/api/base-score-distribution	getBaseScoreDistribution	衡量性, 反
6	Vulnerabilities distributed by CVSS score	Impact Score 分布	/api/impact-score-distribution	getImpactScoreDistribution	反映漏洞完整
7	Vulnerabilities distributed by CVSS score	Exploitability Score 分布	/api/exploitability-score-distribution	getExploitabilityScoreDistribution	衡量漏洞该漏
8	Vulnerabilities by type and year	查看漏洞类型与年份之间关系	/api/type-year-stats	getTypeYearStats	展现漏洞
9	Vulnerabilities by type and number	查看不同漏洞之间的比例	/api/vulnerabilites-by-type	getVulnerabilitiesByType	体现漏洞
10	Vulnerabilities By Types/Categories (1999-2024)	查看年份和漏洞类别的数量关系	/vulnerabilities-type	vulnerabilityTypeHandler(db)	揭示年份之
11	Vulnerabilities by Impact Types (1999-2024)	查看年份和漏洞影响类型的数量关系	/vulnerabilities	vulnerabilitiesHandler(db)	揭示年份类型
12	CVSS Score Range - Bar Chart	查看特定时间段 CVSS 的分布情况	/api/cvss_cves?range=\${range}&start=\${start}&end=\${end}	getCVEsBySeverityRange	查看特自
13	CVSS Distribution Bubble Chart	查看特定时间段 CVSS 的分布情况	/api/cvss_cves?range=\${range}&start=\${start}&end=\${end}	getCVEsBySeverityRange	查看特自

• 示例请求 (curl) :

代码块

```
1 curl -X GET "http://localhost:8080/api/stats"
2 curl -X GET "http://localhost:8080/api/impact-score-distribution"
```



```

3 curl -X GET "http://localhost:8080/vulnerabilities"
4 curl -X GET "http://localhost:8080/vulnerabilities-type"
5 curl -X GET "http://localhost:8080/api/cvss_distribution?start=2023-01-01&end=2023-12-31"

```

接口响应截图：

```

(kali@kali)-[~/Desktop]
$ curl -X GET "http://localhost:8080/api/stats"
{"total":246993,"critical":46455,"exploitable":1092,"patched":1219}

```

```

(kali@kali)-[~/Desktop]
$ curl -X GET "http://localhost:8080/api/impact-score-distribution"
[{"score_range": "0-1", "score_count": 36}, {"score_range": "3-4", "score_count": 57845}, {"score_range": "2-3", "score_count": 102296}, {"score_range": "4-5", "score_count": 16573}, {"score_range": "1-2", "score_count": 20705}, {"score_range": "6-7", "score_count": 66331}, {"score_range": "7-8", "score_count": 229}, {"score_range": "5-6", "score_count": 83450}, {"score_range": "9+", "score_count": 28522}, {"score_range": "8-9", "score_count": 367}]

```

```

(kali@kali)-[~/Desktop]
$ curl -X GET "http://localhost:8080/vulnerabilities"
{"data": {"1999": {"Bypass": 39, "CodeExecution": 0, "DenialOfService": 298, "InformationLeak": 0, "PrivilegeEscalation": 179, "2000": {"Bypass": 53, "CodeExecution": 0, "DenialOfService": 322, "InformationLeak": 0, "PrivilegeEscalation": 79, "2001": {"Bypass": 79, "CodeExecution": 0, "DenialOfService": 359, "InformationLeak": 2, "PrivilegeEscalation": 51, "2002": {"Bypass": 132, "CodeExecution": 0, "DenialOfService": 548, "InformationLeak": 6, "PrivilegeEscalation": 49, "2003": {"Bypass": 72, "CodeExecution": 1, "DenialOfService": 386, "InformationLeak": 4, "PrivilegeEscalation": 40, "2004": {"Bypass": 139, "CodeExecution": 1, "DenialOfService": 634, "InformationLeak": 7, "PrivilegeEscalation": 9, "2005": {"Bypass": 253, "CodeExecution": 5, "DenialOfService": 761, "InformationLeak": 7, "PrivilegeEscalation": 8, "2006": {"Bypass": 260, "CodeExecution": 5, "DenialOfService": 969, "InformationLeak": 8, "PrivilegeEscalation": 15, "2007": {"Bypass": 229, "CodeExecution": 9, "DenialOfService": 1103, "InformationLeak": 10, "PrivilegeEscalation": 16, "2008": {"Bypass": 393, "CodeExecution": 10, "DenialOfService": 988, "InformationLeak": 10, "PrivilegeEscalation": 14, "2009": {"Bypass": 245, "CodeExecution": 12, "DenialOfService": 988, "InformationLeak": 17, "PrivilegeEscalation": 4, "2010": {"Bypass": 191, "CodeExecution": 10, "DenialOfService": 835, "InformationLeak": 30, "PrivilegeEscalation": 4, "2011": {"Bypass": 192, "CodeExecution": 32, "DenialOfService": 1107, "InformationLeak": 30, "PrivilegeEscalation": 9, "2012": {"Bypass": 315, "CodeExecution": 38, "DenialOfService": 1330, "InformationLeak": 21, "PrivilegeEscalation": 11, "2013": {"Bypass": 433, "CodeExecution": 30, "DenialOfService": 1525, "InformationLeak": 35, "PrivilegeEscalation": 11, "2014": {"Bypass": 478, "CodeExecution": 41, "DenialOfService": 1615, "InformationLeak": 20, "PrivilegeEscalation": 17, "2015": {"Bypass": 628, "CodeExecution": 59, "DenialOfService": 2017, "InformationLeak": 46, "PrivilegeEscalation": 45, "2016": {"Bypass": 366, "CodeExecution": 271, "DenialOfService": 1606, "InformationLeak": 133, "PrivilegeEscalation": 66, "2017": {"Bypass": 466, "CodeExecution": 1026, "DenialOfService": 2854, "InformationLeak": 565, "PrivilegeEscalation": 160, "2018": {"Bypass": 496, "CodeExecution": 1044, "DenialOfService": 1905, "InformationLeak": 550, "PrivilegeEscalation": 217, "2019": {"Bypass": 584, "CodeExecution": 1547, "DenialOfService": 1685, "InformationLeak": 874, "PrivilegeEscalation": 223, "2020": {"Bypass": 690, "CodeExecution": 1477, "DenialOfService": 1879, "InformationLeak": 784, "PrivilegeEscalation": 301, "2021": {"Bypass": 694, "CodeExecution": 1676, "DenialOfService": 2451, "InformationLeak": 793, "PrivilegeEscalation": 284, "2022": {"Bypass": 772, "CodeExecution": 1743, "DenialOfService": 2107, "InformationLeak": 717, "PrivilegeEscalation": 289, "2023": {"Bypass": 918, "CodeExecution": 2329, "DenialOfService": 2054, "InformationLeak": 892, "PrivilegeEscalation": 361, "2024": {"Bypass": 943, "CodeExecution": 2027, "DenialOfService": 2270, "InformationLeak": 589, "PrivilegeEscalation": 369, "totals": {"Bypass": 10060, "CodeExecution": 13393, "DenialOfService": 34596, "InformationLeak": 6150, "PrivilegeEscalation": 2831, "years": ["1999", "2000", "2001", "2002", "2003", "2004", "2005", "2006", "2007", "2008", "2009", "2010", "2011", "2012", "2013", "2014", "2015", "2016", "2017", "2018", "2019", "2020", "2021", "2022", "2023", "2024"]}}

```

```

(kali@kali)-[~/Desktop]
$ curl -X GET "http://localhost:8080/vulnerabilities-type"
{"data": {"1999": {"CSRF": 0, "DirectoryTraversal": 10, "FileInclusion": 0, "InputValidation": 0, "MemoryCorruption": 1, "OpenRedirect": 0, "Overflow": 284, "SQLInjection": 0, "SSRF": 0, "XSS": 2, "XXE": 0}, "2000": {"CSRF": 0, "DirectoryTraversal": 26, "FileInclusion": 0, "InputValidation": 0, "MemoryCorruption": 0, "OpenRedirect": 0, "Overflow": 225, "SQLInjection": 2, "SSRF": 0, "XSS": 5, "XXE": 0}, "2001": {"CSRF": 0, "DirectoryTraversal": 123, "FileInclusion": 2, "InputValidation": 0, "MemoryCorruption": 0, "OpenRedirect": 0, "Overflow": 234, "SQLInjection": 7, "SSRF": 0, "XSS": 31, "XXE": 0}, "2002": {"CSRF": 2, "DirectoryTraversal": 115, "FileInclusion": 14, "InputValidation": 2, "MemoryCorruption": 2, "OpenRedirect": 0, "Overflow": 402, "SQLInjection": 46, "SSRF": 0, "XSS": 216, "XXE": 1}, "2003": {"CSRF": 0, "DirectoryTraversal": 55, "FileInclusion": 17, "InputValidation": 0, "MemoryCorruption": 7, "OpenRedirect": 0, "Overflow": 290, "SQLInjection": 46, "SSRF": 0, "XSS": 128, "XXE": 0}, "2004": {"CSRF": 5, "DirectoryTraversal": 112, "FileInclusion": 38, "InputValidation": 4, "MemoryCorruption": 25, "OpenRedirect": 0, "Overflow": 395, "SQLInjection": 149, "SSRF": 0, "XSS": 302, "XXE": 0}, "2005": {"CSRF": 9, "DirectoryTraversal": 199, "FileInclusion": 94, "InputValidation": 2, "MemoryCorruption": 29, "OpenRedirect": 1, "Overflow": 448, "SQLInjection": 601, "SSRF": 0, "XSS": 764, "XXE": 5}, "2006": {"CSRF": 18, "DirectoryTraversal": 334, "FileInclusion": 902, "InputValidation": 3, "MemoryCorruption": 95, "OpenRedirect": 1, "Overflow": 582, "SQLInjection": 998, "SSRF": 0, "XSS": 1323, "XXE": 0}, "2007": {"CSRF": 68, "DirectoryTraversal": 338, "FileInclusion": 637, "InputValidation": 9, "MemoryCorruption": 127, "OpenRedirect": 2, "Overflow": 735, "SQLInjection": 689, "SSRF": 1, "XSS": 848, "XXE": 0}, "2008": {"CSRF": 114, "DirectoryTraversal": 459, "FileInclusion": 200, "InputValidation": 7, "MemoryCorruption": 141, "OpenRedirect": 21, "Overflow": 554, "SQLInjection": 1490, "SSRF": 0, "XSS": 998, "XXE": 1}, "2009": {"CSRF": 86, "DirectoryTraversal": 250, "FileInclusion": 98, "InputValidation": 5, "MemoryCorruption": 194, "OpenRedirect": 7, "Overflow": 551, "SQLInjection": 679, "SSRF": 0, "XSS": 768, "XXE": 2}, "2010": {"CSRF": 60, "DirectoryTraversal": 237, "FileInclusion": 69, "InputValidation": 4, "MemoryCorruption": 328, "OpenRedirect": 11, "Overflow": 295, "SQLInjection": 523, "SSRF": 0, "XSS": 495, "XXE": 2}, "2011": {"CSRF": 69, "DirectoryTraversal": 101, "FileInclusion": 5, "InputValidation": 3, "MemoryCorruption": 480, "OpenRedirect": 14, "Overflow": 339, "SQLInjection": 156, "SSRF": 0, "XSS": 475, "XXE": 7}, "2012": {"CSRF": 152, "DirectoryTraversal": 122, "FileInclusion": 12, "InputValidation": 1, "MemoryCorruption": 523, "OpenRedirect": 28, "Overflow": 406, "SQLInjection": 237, "SSRF": 0, "XSS": 868, "XXE": 18}, "2013": {"CSRF": 175, "DirectoryTraversal": 137, "FileInclusion": 5, "InputValidation": 2, "MemoryCorruption": 595, "OpenRedirect": 32, "Overflow": 402, "SQLInjection": 202, "SSRF": 5, "XSS": 831, "XXE": 39}, "2014": {"CSRF": 183, "DirectoryTraversal": 212, "FileInclusion": 7, "InputValidation": 9, "MemoryCorruption": 703, "OpenRedirect": 47, "Overflow": 386, "SQLInjection": 324, "SSRF": 10, "XSS": 1103, "XXE": 80}, "2015": {"CSRF": 226, "DirectoryTraversal": 187, "FileInclusion": 7, "InputValidation": 17, "MemoryCorruption": 1164, "OpenRedirect": 61, "Overflow": 451, "SQLInjection": 316, "SSRF": 6, "XSS": 1023, "XXE": 58}, "2016": {"CSRF": 135, "DirectoryTraversal": 102, "FileInclusion": 8, "InputValidation": 25, "MemoryCorruption": 872, "OpenRedirect": 29, "Overflow": 439, "SQLInjection": 109, "SSRF": 15, "XSS": 578, "XXE": 53}, "2017": {"CSRF": 249, "DirectoryTraversal": 320, "FileInclusion": 13, "InputValidation": 122, "MemoryCorruption": 1265, "OpenRedirect": 52, "Overflow": 1034, "SQLInjection": 459, "SSRF": 51, "XSS": 1385, "XXE": 77}, "2018": {"CSRF": 377, "DirectoryTraversal": 325, "FileInclusion": 21, "InputValidation": 167, "MemoryCorruption": 1187, "OpenRedirect": 39, "Overflow": 1351, "SQLInjection": 415, "SSRF": 81, "XSS": 1760, "XXE": 137}, "2019": {"CSRF": 376, "DirectoryTraversal": 301, "FileInclusion": 20, "InputValidation": 219, "MemoryCorruption": 1443, "OpenRedirect": 58, "Overflow": 882, "SQLInjection": 387, "SSRF": 76, "XSS": 1617, "XXE": 88}, "2020": {"CSRF": 396, "DirectoryTraversal": 331, "FileInclusion": 27, "InputValidation": 305, "MemoryCorruption": 1414, "OpenRedirect": 55, "Overflow": 980, "SQLInjection": 535, "SSRF": 125, "XSS": 2034, "XXE": 103}, "2021": {"CSRF": 380, "DirectoryTraversal": 380, "FileInclusion": 34, "InputValidation": 382, "MemoryCorruption": 1734, "OpenRedirect": 58, "Overflow": 1224, "SQLInjection": 575, "SSRF": 129, "XSS": 2246, "XXE": 99}, "2022": {"CSRF": 597, "DirectoryTraversal": 471, "FileInclusion": 44, "InputValidation": 302, "MemoryCorruption": 1828, "OpenRedirect": 73, "Overflow": 1591, "SQLInjection": 1433, "SSRF": 163, "XSS": 2819, "XXE": 88}, "2023": {"CSRF": 948, "DirectoryTraversal": 468, "FileInclusion": 55, "InputValidation": 362, "MemoryCorruption": 1966, "OpenRedirect": 73, "Overflow": 1503, "SQLInjection": 1390, "SSRF": 140, "XSS": 3607, "XXE": 95}, "2024": {"CSRF": 883, "DirectoryTraversal": 577, "FileInclusion": 124, "InputValidation": 295, "MemoryCorruption": 1687, "OpenRedirect": 77, "Overflow": 1392, "SQLInjection": 2113, "SSRF": 247, "XSS": 5526, "XXE": 63}, "totals": {"CSRF": 5508, "DirectoryTraversal": 6292, "FileInclusion": 2453, "InputValidation": 2243, "MemoryCorruption": 17810, "OpenRedirect": 739, "Overflow": 17375, "SQLInjection": 13881, "SSRF": 1049, "XSS": 31752, "XXE": 1016}, "years": ["1999", "2000", "2001", "2002", "2003", "2004", "2005", "2006", "2007", "2008", "2009", "2010", "2011", "2012", "2013", "2014", "2015", "2016", "2017", "2018", "2019", "2020", "2021", "2022", "2023", "2024"]}}

```

```

(kali@kali)-[~/Desktop]
$ curl -X GET "http://localhost:8080/api/cvss_distribution?start=2023-01-01&end=2023-12-31"
[{"range": "", "count": 0, "avg_score": 0}, {"range": "", "count": 1, "avg_score": 0}, {"range": "", "count": 48, "avg_score": 0}, {"range": "", "count": 206, "avg_score": 0}, {"range": "", "count": 1483, "avg_score": 0}, {"range": "", "count": 3772, "avg_score": 0}, {"range": "", "count": 6056, "avg_score": 0}, {"range": "", "count": 3344, "avg_score": 0}, {"range": "", "count": 4088, "avg_score": 0}]

```

3.6.3 压力测试 (Performance/Load Test)

测试工具：wrk

- **测试命令：**

代码块

```
1 wrk -t8 -c100 -d30s http://localhost:8080/api/stats
```

8 个线程：模拟并发工作线程数；一般等于 CPU 核数比较合理。

100 个连接：并发连接数中等，不算太高，能基本反映吞吐能力。

- **结果示例：**

代码块

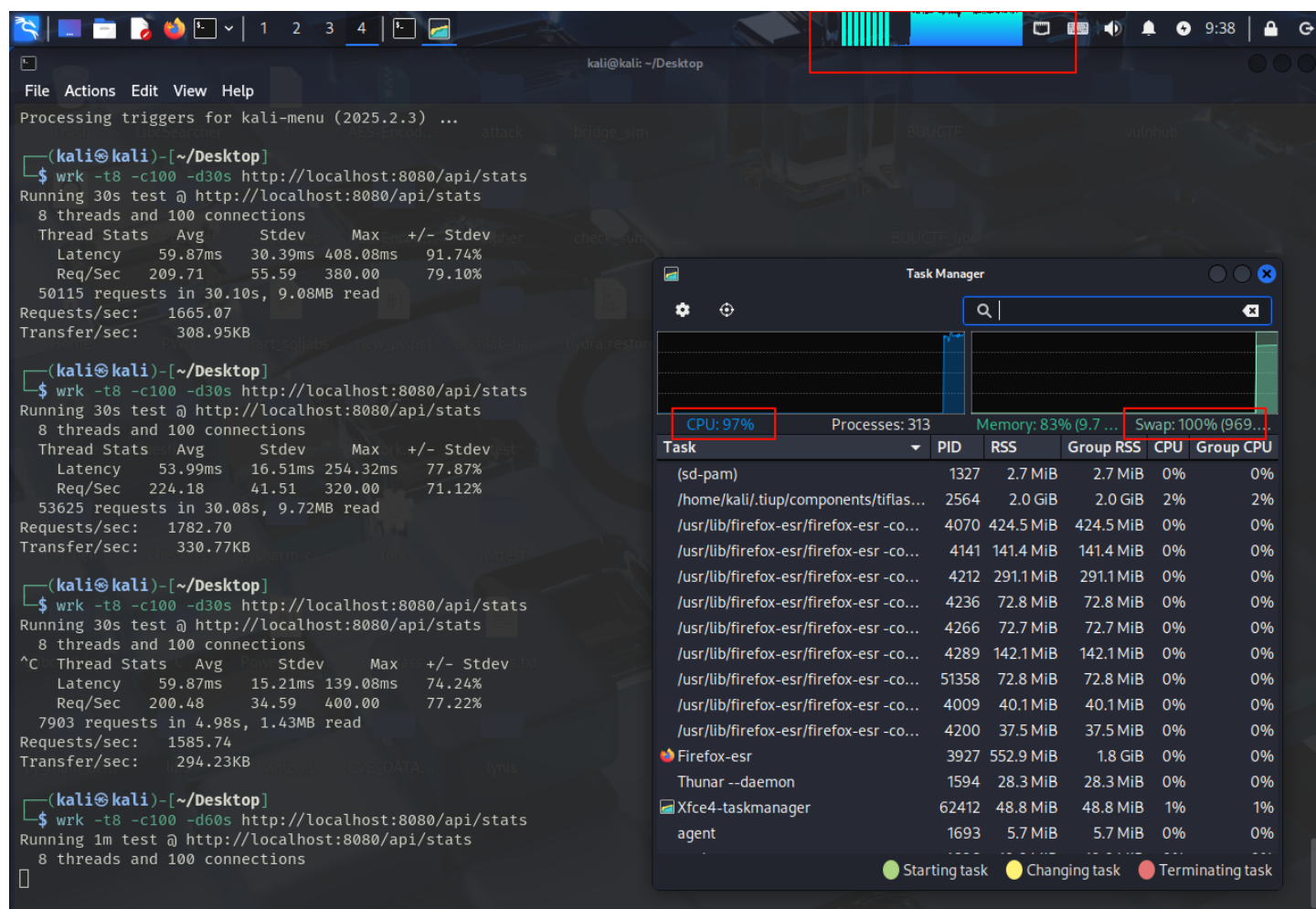
```
1 Running 30s test @ http://localhost:8080/api/stats
2 8 threads and 100 connections
3 Thread Stats Avg Stdev Max +/- Stdev
4 Latency 59.87ms 30.39ms 408.08ms 91.74%
5 Req/Sec 209.71 55.59 380.00 79.10%
6 50115 requests in 30.10s, 9.08MB read
7 Requests/sec: 1665.07
8 Transfer/sec: 308.95KB
```

总请求数：50,115 次请求在 30 秒内完成，说明系统承载能力不错。

每秒请求数 (RPS)：1665 次请求/秒，说明后端性能已经达到了中等偏上的水平。

传输速率：约 309KB/s，这取决于 `/api/stats` 的响应内容大小，说明单个响应体大约 6.4KB 左右。

- **截图展示：**



```
(kali@kali)-[~/Desktop]
$ wrk -t8 -c100 -d30s http://localhost:8080/api/stats
Running 30s test @ http://localhost:8080/api/stats
8 threads and 100 connections
Thread Stats      Avg      Stdev     Max    +/-  Stdev
Latency           59.87ms   30.39ms  408.08ms  91.74%
Req/Sec           209.71    55.59   380.00    79.10%
50115 requests in 30.10s, 9.08MB read
Requests/sec:     1665.07
Transfer/sec:      308.95KB
```

看到这里的性能还是比较有限的

- 延迟分析:

平均延迟: 约 60ms, 表现不错, 适合用户访问的 API 接口。

最大延迟: 408ms, 略高, 但在压力测试下可接受。

标准差大, 说明波动较大: 这可能表明部分请求出现了偶发慢响应 (如首次加载、GC 抖动、连接等待等)。

3.7 TiDB Dashboard

这里的数据记录的是上面测试出来的数据

3.7.1 数据库时间

1) Database Time by SQL Types

图表显示横轴为时间（09:38:00 至 09:42:00），纵轴为数据库时间（秒），不同颜色代表不同类型的 SQL 操作，包括其他类型（other）、通用（general）、使用数据库（Use）、显示信息（Show）、设置参数（Set）、查询数据（Select）、描述表结构（DescTable）及总数据库时间（database time）。观察结果显示，09:40:00 左右 Select 类型的 SQL 占用了约 50 秒的数据库时间，表明该时段查询操作频繁且复杂，而其他类型的 SQL 占用时间较少，说明它们并非主要性能瓶颈。总数据库时间峰值与 Select 查询高负载同步。针对这一情况，可能的原因包括索引缺失、查询逻辑复杂或数据量过大，建议优化查询语句，避免全表扫描和低效 JOIN，并通过添加合适索引来提升查询效率。

2) Database Time by SQL Phase

图表显示，横轴为时间（09:38:00 到 09:42:00），纵轴为数据库时间（秒），不同颜色代表 SQL 执行的不同阶段，包括获取令牌（get token）、执行（execute）、编译（compile）、解析（parse）以及总数据库时间。观察发现，09:40:00 左右执行阶段（execute）耗时最多，约 50 秒，表明 SQL 实际执行时间较长；解析和编译阶段耗时较低，说明这两个阶段性能表现正常。总数据库时间与执行阶段耗时高度相关。建议重点检查查询的执行计划，确保索引和分区策略最优，同时对复杂聚合或排序操作进行优化，必要时增加硬件资源。

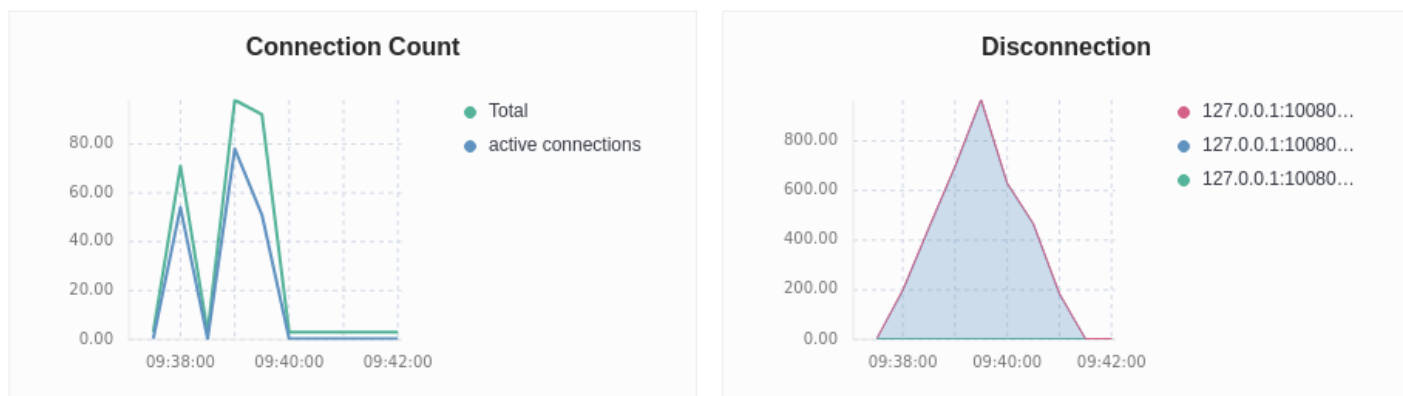
3) SQL Execute Time Overview

图表中，横轴时间范围为 09:38:00 至 09:42:00，纵轴为 SQL 执行时间（秒），颜色区分多种执行操作，包括等待全局事务 ID 的时间（tso_wait）、扫描锁（ScanLock）、扫描数据（Scan）、预写入（Prewrite）、悲观锁（PessimisticLock）、多版本并发控制获取（MvccGetByKey）、获取数据（Get）、COP 处理时间（Cop）和提交事务时间（Commit）。在 09:40:00 附近，tso_wait 和 Scan 的时间较长，分别约 40 秒和 30 秒，预写入和提交事务时间也较高，说明事务处理存在延迟，而其他操作耗时较少。针对这些情况，建议检查 PD（Placement Driver）性能以缓解 tso_wait 时间过长，优化查询以减少扫描数据量，并关注事务冲突及网络延迟以缩短 Prewrite 和 Commit 时间。

综合分析：

分析表明，09:40:00 左右数据库出现明显性能瓶颈，Select 查询耗时较高是主要原因。执行阶段的长时间运行反映出数据扫描和计算开销大，tso_wait 和 Scan 时间的增加则显示 PD 压力和数据扫描效率存在问题。针对这些瓶颈，应从查询优化、索引策略、PD 性能以及事务隔离和网络环境等多方面入手提升数据库性能。

3.7.2 应用连接



1) Connection Count

图表显示，横轴为时间（09:38:00 到 09:42:00），纵轴表示连接数（单位为个），绿色代表总连接数（Total），蓝色代表活跃连接数（active connections）。观察到在 09:38:00 左右，总连接数和活跃连接数迅速上升，分别达到约 80 和 60。随后在 09:40:00 附近，连接数急剧下降，最终接近 0。总连接数和活跃连接数变化趋势一致，表明大多数连接均处于活跃状态。

连接数的剧烈波动可能反映了应用负载的显著变化，或者存在批量创建和关闭连接的操作。如果连接数频繁激增，可能导致数据库资源竞争加剧，从而影响整体性能。建议检查应用的连接管理策略，确保连接池配置合理，避免频繁且不必要的连接创建与销毁，提高系统稳定性。

2) Disconnection

图表中，横轴为时间（09:38:00 至 09:42:00），纵轴为断开连接数量（单位为个），不同颜色表示不同 IP 地址。观察发现，在 09:40:00 附近，断开连接数量达到峰值，约 800 个，且与连接数下降趋势同步。来自 127.0.0.1:10080 等本地 IP 的连接占断开连接的大部分，说明这些连接主要源自本地服务或测试环境。

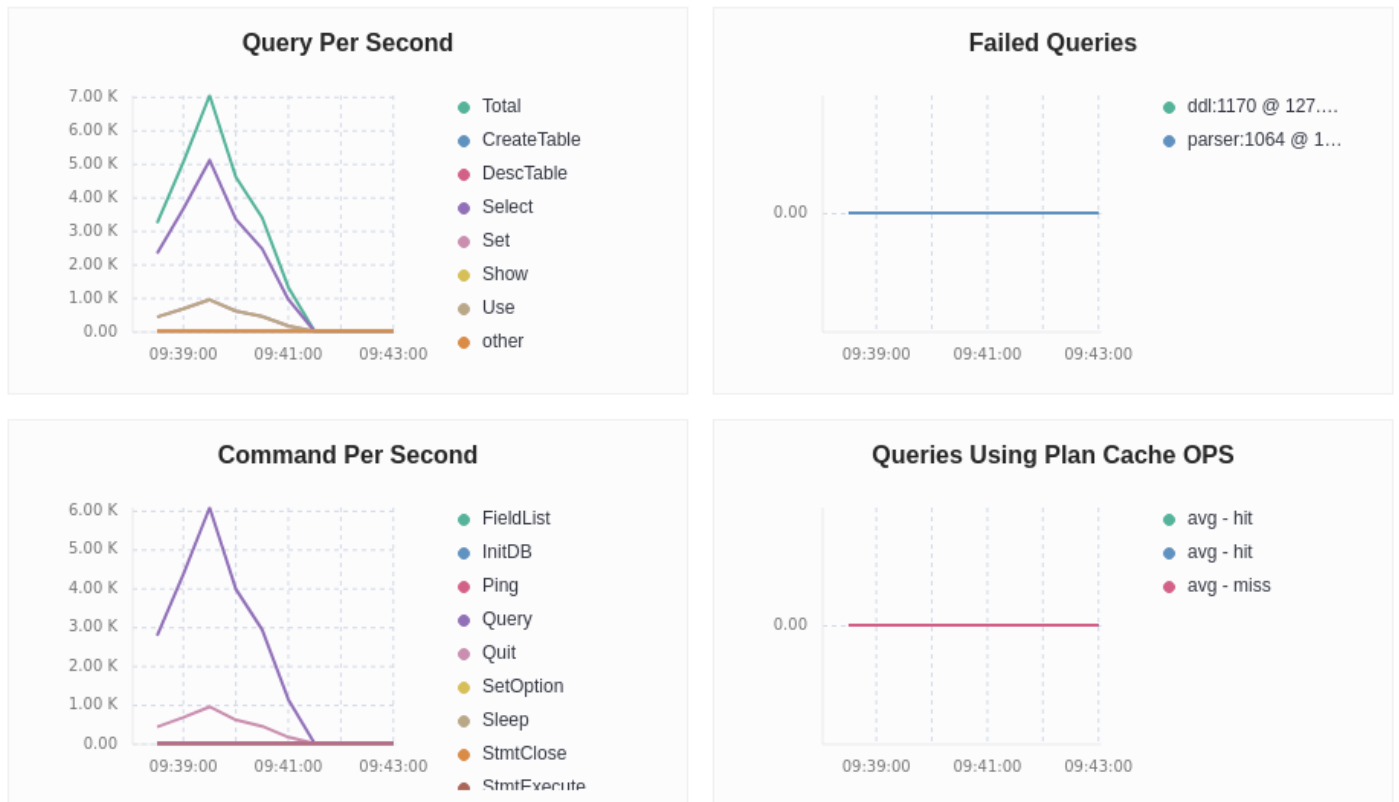
大量断开连接可能表示应用在短时间内释放了大量连接，原因可能包括任务完成、异常终止或连接超时。频繁断开连接会增加连接重建开销，影响系统稳定性。建议检查应用的连接生命周期管理，确保连接创建和释放过程平稳，避免集中式断开带来的负面影响。

综合分析：

分析表明，09:40:00 左右数据库出现了显著的连接数波动和断开连接高峰。这种现象可能与应用连接管理策略不合理或负载快速变化有关，需针对连接池配置和应用逻辑进行优化，以提升系统的稳定性和性能表现。

3.7.3 SQL数量

SQL 数量



1) Query Per Second

图表显示，横轴为时间（09:39:00 到 09:43:00），纵轴表示每秒查询数（单位为个，K 表示千），不同颜色代表不同的 SQL 类型。观察到在 09:40:00 左右，总查询数达到峰值约 7,000 QPS，其中 Select 查询占绝大部分，峰值接近 5,000 QPS，表明该时间段查询操作非常频繁。其他类型查询如 CreateTable、DescTable 数量较少，说明这些操作不是主要负载来源。总查询数在 09:41:00 后迅速下降，最终接近 0。高 QPS 导致数据库负载较高，尤其是 Select 查询占比较大，建议检查查询逻辑，确保索引有效并优化查询，同时通过分页查询或减少不必要的查询来降低整体 QPS。

2) Failed Queries

图表显示，在时间段内失败查询数量始终为 0，表明没有明显的查询失败情况。这是一个积极信号，说明查询执行过程中没有遇到显著错误或异常。如果未来出现失败查询，则需结合具体失败原因（如 DDL 或解析器错误）进行针对性排查。

3) Command Per Second

图表显示，横轴时间同上，纵轴为每秒命令数（单位为个，K 表示千），不同颜色代表不同命令类型。观察发现 09:40:00 左右 Query 命令频率达到峰值约 6,000 CPS，其他命令如 Ping、Quit 频率较低，表明查询命令是主要负载来源。高频 Query 命令可能导致资源竞争和性能瓶颈，建议检查查询复杂度和执行效率，确保查询快速完成。

4) Queries Using Plan Cache OPS

图表显示，时间段内使用计划缓存的操作数始终为 0，表明计划缓存未被充分利用。计划缓存未使用可能是查询多样性较高导致缓存命中率低。建议检查查询是否可以标准化，例如使用参数化查询或

减少动态 SQL，从而提高计划缓存利用率，提升执行效率。

综合分析：

分析显示，09:40:00 左右数据库负载显著增加，表现为总查询数和 Select 查询数激增，而计划缓存未被有效利用导致查询效率不高。虽然无失败查询是积极信号，但高负载可能引发性能瓶颈，建议针对查询优化和缓存利用进行改进。

3.7.4 延迟及拆解

延迟及拆解



1) Query Duration

在 09:41:00 左右，系统整体查询持续时间达到峰值，平均持续时间约为 40 毫秒，其中 Select 查询占据主要比重，是导致整体延迟升高的关键因素。其他类型的查询如 DescTable、Set、Use 等平均耗时相对较低，说明这些类型的操作并非性能瓶颈。这种 Select 查询延迟可能由大量数据扫描、缺乏有效索引或查询逻辑复杂引起，建议通过执行计划分析（如 EXPLAIN）进一步优化查询结构，提升响应效率。

2) Average Idle Connection Duration

在同一时段（约 09:41:00），非事务连接的空闲持续时间达到峰值，约为 4 毫秒，而事务内连接的空闲时间则几乎为零。这表明事务执行相对紧凑，但非事务连接在释放和复用上存在一定延迟，可能与连接池管理策略或客户端连接使用方式有关。虽然该持续时间不算严重，但仍建议检查应用层连接池配置，确保连接能在空闲后及时释放，避免资源堆积。

3) Get Token Duration

获取事务令牌（Token）的持续时间整体维持在较低水平，即使在高负载时段（09:41:00），平均值也未超过 0.4 毫秒，且第 99 百分位数延迟迅速下降至接近零。这反映出 PD（Placement Driver）组件在事务 ID 分配方面性能稳定，未成为系统延迟的瓶颈。尽管当前无明显问题，但仍建议在系统高并发场景中持续监控，以防资源争用影响事务调度。

4) Parse Duration

SQL 解析阶段的平均持续时间在高负载时段约为 600 微秒，属于较低水平，说明语法解析过程高效、无明显阻塞。此外，第 99 百分位的延迟也迅速下降至接近零，进一步印证了解析环节的性能稳定性。这表明当前 SQL 请求结构较为规范，未出现由于语法复杂性引发的性能问题。如后续出现解析瓶颈，可检查是否存在动态拼接的复杂 SQL。

5) Compile Duration

编译阶段的平均持续时间在 09:41:00 时达到 2.5 毫秒左右，属于正常范围，未见异常延迟波动。这意味着 SQL 执行计划的生成速度尚可，TiDB 编译器整体运行良好。虽然无需立即优化，但若未来出现复杂语句编译时间增长的情况，可考虑使用参数化查询以提高计划缓存命中率，减少重复编译开销。

6) Execute Duration

SQL 执行阶段是当前整体查询耗时的主要来源，平均执行时间在 09:41:00 时达到约 40 毫秒，与 Select 查询延迟高度一致。这表明系统在执行阶段（如数据扫描、聚合、排序等）耗时明显，可能由未命中索引、数据量大或计算密集操作导致。建议重点关注慢查询日志，结合执行计划识别瓶颈点，并在可能范围内调整 SQL 结构或优化表结构以提升执行效率。

综合分析：

分析显示，数据库在 09:41:00 左右出现了明显的查询延迟问题，主要表现为 Select 查询持续时间显著升高，从而带动整体查询延迟上升。同时，执行阶段的耗时（Execute Duration）也较高，表明 SQL 实际执行过程中的数据扫描或计算开销较大。此外，非事务中的空闲连接持续时间偏高，可能与连接池管理不当或连接释放不及时有关。综合来看，系统在该时段面临主要由查询执行效率低和连接管理不足导致的性能瓶颈，建议从 SQL 优化与连接池配置两方面入手进行调整。

3.7.5 事务



1) Transaction Per Second

图表中可以看出，09:40:00 左右乐观事务的提交数量（commit-optimistic）达到峰值，接近 5000 TPS，表明系统在该时段处理了大量并发写入操作。而其他类型事务（如回滚和中止）数量相对较少，说明大多数事务顺利提交。随后在 09:41:00 后，TPS 急剧下降至接近 0，可能与系统负载减轻或业务操作减少有关。整体来看，系统能有效处理高并发事务，但仍需关注潜在的事务冲突和资源分配情况。

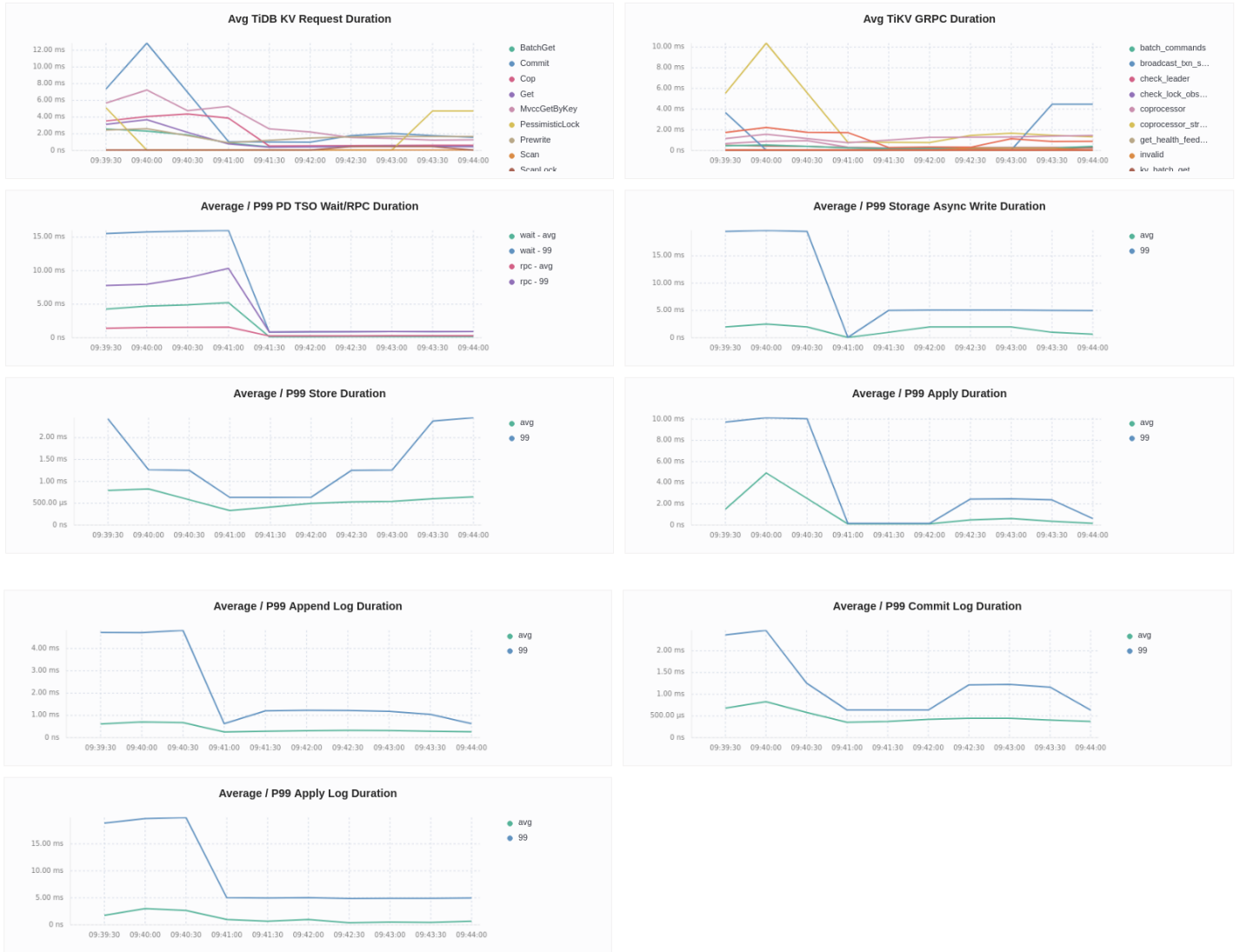
2) Transaction Duration

图表中，乐观事务的第 99 百分位持续时间在 09:40:00 左右达到了约 50 毫秒的峰值，反映出少部分事务存在较高延迟的情况，可能源于事务冲突、锁等待或网络抖动。相比之下，乐观与悲观事务的平均持续时间分别约为 10 毫秒和 20 毫秒，整体保持在可接受范围内。事务持续时间在 09:41:00 后快速下降，说明高延迟是短时波动。为优化事务性能，应持续关注高百分位事务的执行情况，优化查询逻辑、事务粒度以及资源配置。

综合分析：

在 09:40:00 左右，数据库系统出现了明显的事务高峰，主要表现为大量乐观事务成功提交，commit-optimistic 的事务数达到峰值，反映出该时段存在密集的并发写入操作。同时，乐观事务的第 99 百分位持续时间较高，提示部分事务可能因冲突或网络延迟导致处理时间延长。此外，事务数在 09:41:00 后迅速下降，表明事务负载存在较大波动。整体来看，该时段事务处理压力显著，需关注事务冲突、性能瓶颈以及系统资源的动态调度。

3.7.6 核心流程耗时



1) Avg TiDB KV Request Duration

在 09:40:00 左右，TiDB 的 KV 请求中 `Commit` 和 `Scan` 操作持续时间分别达到约 12 ms 和 6 ms，远高于其他操作，如 `Get`、`Prewrite` 等。说明该时段事务提交和大范围扫描操作成为主要性能瓶颈。建议优化事务大小、控制扫描范围，并确认相关 SQL 是否合理使用索引，以降低关键路径延迟。

2) Avg TiKV GRPC Duration

09:40:00 时，`batch_commands` 和 `kv_batch_net` 操作的平均持续时间分别达到 10 ms 和 5 ms，明显高于其他 RPC 操作。这可能与批量请求数据量大或网络拥塞有关。需关注 TiKV 节点间的网络状况和 batch 操作是否合理配置，必要时可调整 batch size 或优化网络拓扑。

3) Average / P99 PD TSO Wait/RPC Duration

该时间段 PD 的 TSO（时间戳）分配出现明显拥堵，`wait - avg` 和 `wait - 99` 延迟分别达到 15 ms 和 10 ms，而 RPC 时间则较低。这表明 PD 本身处理请求的能力可能是瓶颈，建议监控 PD 节点负载并视情况扩容或优化网络传输效率，确保 TSO 分配顺畅。

4) Average / P99 Storage Async Write Duration

异步写入操作在 09:40:00 达到性能高峰，平均和第 99 百分位写入延迟分别为 15 ms 和 10 ms，显示出存储 I/O 压力较大。需关注磁盘写入性能和 TiKV 的存储压力，建议检查磁盘队列深度和延迟，必要时升级存储设备或分散写入压力。

5) Average / P99 Store Duration

Store 层的操作耗时在高峰期达到平均 2 ms，第 99 百分位为 1.5 ms。尽管单位为微秒级，但相较平稳状态仍显著偏高。说明底层存储设备可能面临一定负载压力，应持续关注磁盘 I/O 和 CPU 使用率，并评估是否需要扩容或重新平衡 Region 分布。

6) Average / P99 Apply Duration

Apply 操作（Raft 日志应用）在 09:40:00 时平均和第 99 百分位时延分别达到 10 ms 和 5 ms，表明有大量事务数据需要应用到存储引擎。高延迟可能来自事务复杂性或数据量过大，建议从 SQL 层审查事务大小和写入模式，必要时优化写入逻辑。

7) Average / P99 Append Log Duration

日志追加操作延迟在 09:40:00 达到平均 4 ms、第 99 百分位 2 ms，显示日志写入成为性能瓶颈。可能由于磁盘带宽或写入吞吐限制，建议查看 RocksDB WAL 写入和磁盘性能指标，必要时可考虑 SSD 升级或 WAL 压缩配置调整。

8) Average / P99 Commit Log Duration

Commit Log 持续时间在高峰期分别为 1.5 ms（avg）和 1 ms（P99），尽管单位为微秒，但相较平时偏高。提交阶段延迟增加说明日志刷盘耗时上升，可能影响事务完成时间。应综合考虑 TiKV 写路径延迟、磁盘写能力和后台线程处理效率。

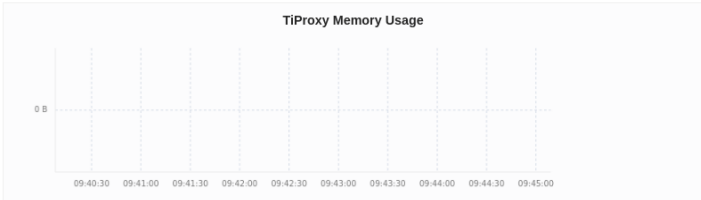
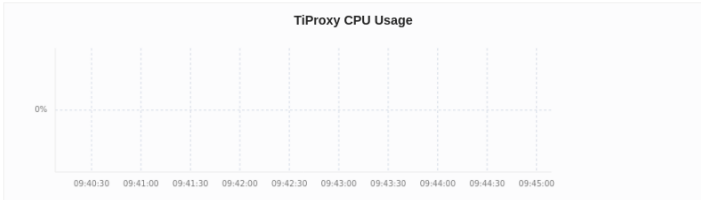
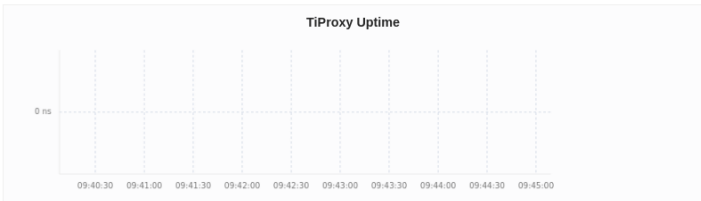
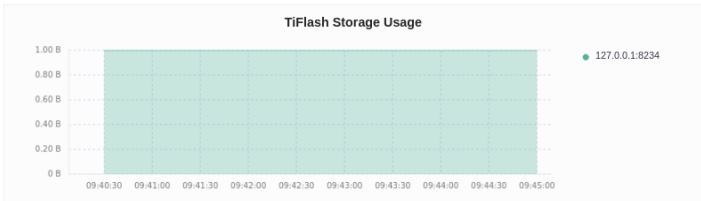
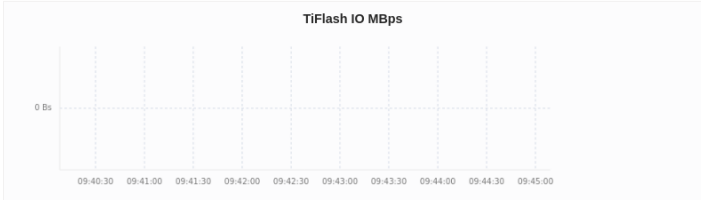
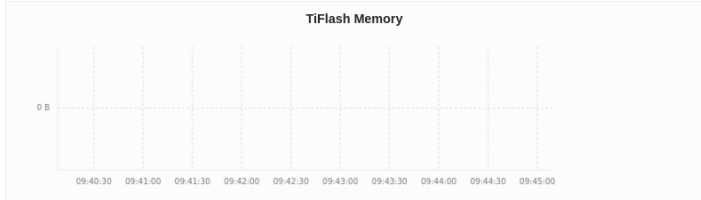
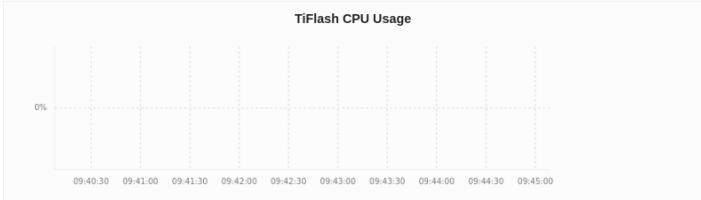
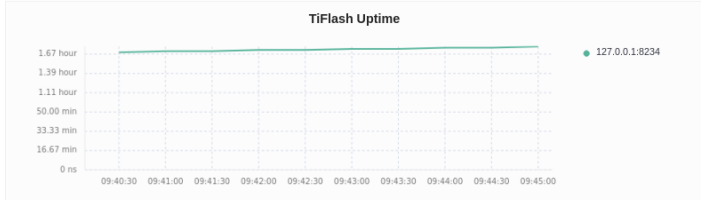
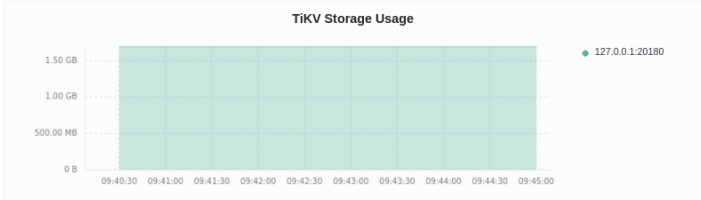
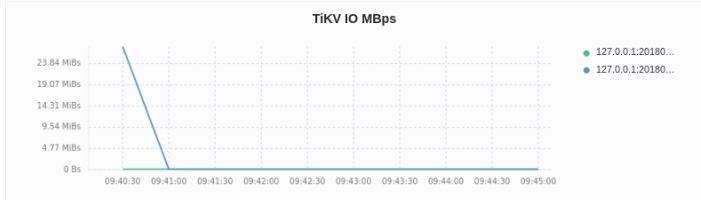
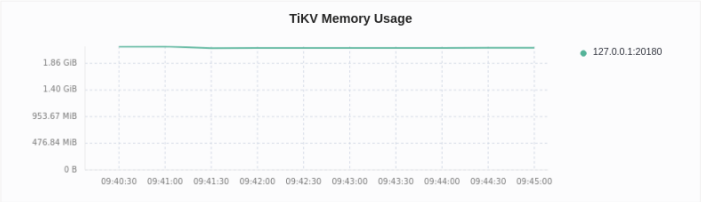
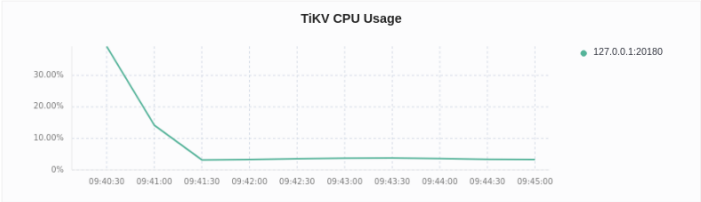
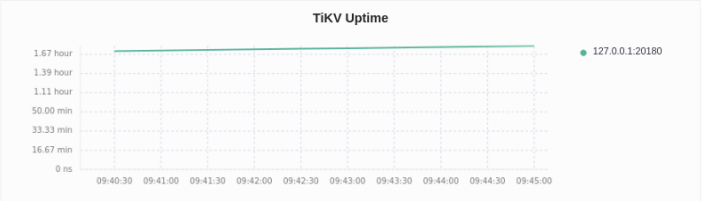
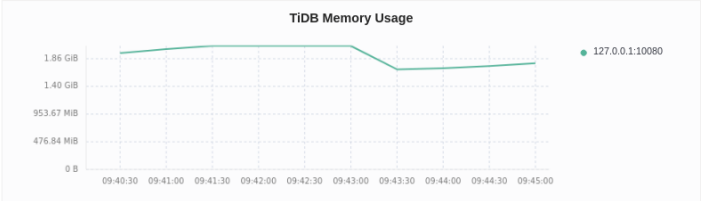
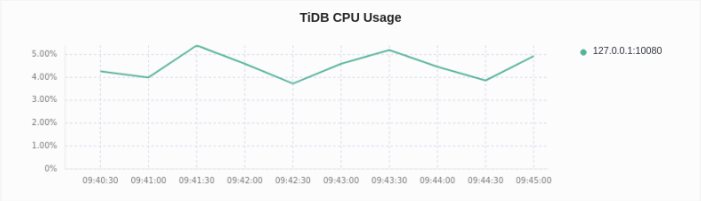
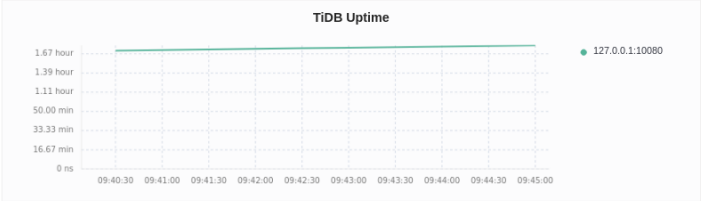
9) Average / P99 Apply Log Duration

日志应用过程在 09:40:00 达到峰值，平均耗时为 16 ms，P99 为 8 ms，远高于正常水平。这反映出事务日志被应用到状态机过程中的压力剧增。建议分析事务复杂度、Apply Task 排队情况，并结合写入放大指标判断存储引擎是否需优化。

综合分析：

09:40:00 时段 TiDB 系统经历了一次显著的性能抖动，多个组件（TiDB、TiKV、PD）同时表现出延迟上升，尤其在事务提交路径（包括 KV 请求、TSO 分配、日志追加与应用、存储写入等）集中表现出瓶颈。事务并发高峰导致系统多个层级资源压力上升。建议综合分析业务高峰时段特征，优化慢 SQL、事务规模与逻辑，同时评估是否需要扩容 PD、TiKV 节点或提升网络与存储资源配置，以实现更好的稳定性与吞吐能力。

3.7.7 服务



当前 TiDB 集群的核心组件运行状况整体良好，系统保持稳定状态。TiDB 和 TiKV 服务的运行时长均为约 1.67 小时，表明自启动以来未发生重启或异常中断。其中，TiDB 的 CPU 使用率维持在 3% 至 5% 之间，内存使用量稳定在 1.4 GB 至 1.86 GB，显示当前负载较轻，资源使用充足。TiKV 的 CPU 使用率在某一时间点出现短暂上升（达 30%），可能因批量操作导致，但随后迅速恢复正常；其内存使

用量维持在 1.86 GB 左右，I/O 带宽在初始阶段短暂达到高峰（约 23.84 MB/s），随后趋近于 0，表明 TiKV 在系统初始化或短时任务后稳定运行，整体性能良好。

另一方面，TiFlash 和 TiProxy 均未显示活跃状态。TiFlash 的各项指标（包括 CPU、内存、I/O 带宽和存储使用）均为 0，推测该组件当前未启用或未配置使用。TiProxy 同样未显示运行信息，可能尚未启动或未纳入监控系统。尽管这两个组件暂未启用，但并不会影响现有 TiDB 和 TiKV 构成的核心系统运行。

综合来看，当前集群的健康状况良好，TiDB 和 TiKV 稳定运行、资源使用合理，适合当前业务负载。建议根据未来需求决定是否启用 TiFlash 和 TiProxy，并持续关注 TiKV 在高负载情况下的表现，防止资源瓶颈影响系统稳定性。同时，定期检查存储使用情况，防止磁盘空间不足带来的风险。

3.7.8 监控指标



当前 TiDB 集群整体运行正常，但在特定时间段（如 09:35 和 09:40 左右）出现了明显的间歇性高负载，反映在资源单位（RU）消耗、TiDB 与 TiKV 的 CPU 使用率以及 TiKV 的 I/O 带宽上。RU 总消耗在上述时段达到了约 3000 RU，同时 TiDB CPU 使用率峰值达到约 600%，TiKV CPU 使用率约为 100%，I/O 带宽峰值则达到 23.84 MB/s。这些数据共同表明在该时间段内发生了较大规模的操作，如批量写入、复杂查询或事务处理。

此外，当前仅启用了 default 资源组，其 RU 使用趋势与总 RU 完全一致，说明系统所有负载集中在该资源组中。此种结构适用于单一业务环境，但若未来存在多业务或多用户需求，建议引入更多资源组以实现资源隔离和管理。

TiDB 在高负载时展现出良好的处理能力，但需关注其长时间高 CPU 占用可能引发的性能瓶颈；TiKV 的资源使用上升虽相对平缓，但其 I/O 带宽峰值显示其在高负载时也承受了一定压力。

综上，当前 TiDB 集群在大部分时间内负载较轻，系统稳定性和资源使用状况良好。建议针对间歇性高负载场景进行优化，例如分批处理数据、优化 SQL 查询、合理配置索引等；同时引入监控告警机

制，关注 RU、CPU、I/O 等关键指标；并预留资源扩展和资源组隔离能力，以支持未来业务增长与多用户环境下的稳定运行。

第四章 总结与展望

4.1 项目成果总结

本系统作为面向大规模CVE漏洞数据的综合管理平台，成功实现了从数据采集、清洗、规范化存储到可视化展示、统计分析以及安全访问控制等全流程功能。通过引入TiDB分布式数据库和Go语言构建后端服务，结合Bootstrap前端框架，系统在性能、扩展性和用户体验上均表现出色，为安全研究人员、运维团队及企业管理人员提供了高效、稳定、可扩展的漏洞治理工具。

首先，在高效的数据处理能力方面，系统支持从NIST公开的NVD数据库中批量导入CVE漏洞信息，并完成多维度字段的提取、校验和结构化存储。通过Python脚本实现自动化的数据采集与解析流程，结合GORM进行高效入库操作，显著提升了数据导入效率。同时，利用批处理机制减少单次事务压力，确保海量数据的快速导入与更新，为后续查询与分析奠定了坚实基础。

其次，在漏洞管理功能方面，系统具备完善的漏洞风险评估与查询能力。通过集成CVSS评分体系，系统能够提供基于baseScore、impactScore、exploitabilityScore等多个维度的风险评估，并支持自动化排序优先级，帮助用户快速识别高危漏洞。此外，系统支持厂商、产品、版本三级关联查询，精准定位影响自身系统的漏洞条目，提升开发人员与运维工程师的安全响应效率。同时，CPE（Common Platform Enumeration）标准的集成，使得系统可以实现组件级漏洞匹配，便于资产依赖管理和供应链风险分析，满足企业对软件供应链安全的管理需求。

在可视化分析能力方面，系统提供了丰富的图表展示，包括CVSS评分分布、年度漏洞趋势图、漏洞类型占比饼图等，支撑宏观风险态势感知。系统采用ECharts等前端可视化库，结合后端聚合视图（如vuln_type_summary_view、vuln_impact_stats_view等），为管理层提供可量化的决策依据，有助于企业制定更科学的安全策略与修复计划。

在安全与权限控制方面，系统采用了基于JWT的无状态认证机制，适用于分布式部署场景，保障身份验证的安全性。通过中间件方式实现接口级别的权限控制，系统支持按角色分配SELECT/INSERT/UPDATE/DELETE操作，确保敏感数据仅能被授权用户访问。该机制不仅提高了系统的安全性，也为未来多租户架构的扩展提供了良好基础。

在系统扩展性与灵活性方面，系统采用模块化设计，各功能模块之间松耦合，便于独立开发、测试与部署。这种设计使得系统能够灵活对接其他平台（如DevSecOps流水线、资产管理工具等），具备广泛的适用场景。同时，数据结构中预留了嵌入向量字段（vectorEmbedding），为未来语义搜索或AI辅助分类打下基础，增强了系统的智能化潜力。

最后，在系统稳定性与性能优化方面，监控数据显示，尽管在特定时间段内出现了间歇性高负载（如RU消耗增加、CPU使用率上升），但整体运行稳定，未出现严重性能瓶颈。通过合理的资源组划分、索引优化及分批处理策略，有效提升了系统的并发处理能力和响应效率。系统能够在高并发读写场景下保持良好的性能表现，展现出较强的鲁棒性与适应性。

综上所述，本系统在数据处理、漏洞管理、可视化展示、权限控制、扩展能力及性能优化等方面均取得了显著成果，为企业和组织提供了一个全面、智能、高效的CVE漏洞管理解决方案，具备良好的应用前景与推广价值。

4.2 存在的问题与改进方向

尽管本系统在功能和性能方面表现良好，但在实际应用过程中仍存在一些可以优化和改进的空间。首先，部分高级模块尚未启用，例如TiFlash列式存储引擎和TiProxy代理服务等。这些模块的缺失限制了系统的扩展性和高并发场景下的性能表现。此外，在高负载情况下，由于动态SQL语句较多，导致计划缓存利用率较低，影响了查询效率。其次，当前的数据更新机制依赖本地文件扫描，缺乏自动化远程同步能力，无法实时获取最新的CVE漏洞数据，可能影响漏洞响应的时效性。再者，可视化分析功能虽然提供了基础图表展示，但对漏洞分类和修复建议的支持较为有限，缺乏基于机器学习或NLP技术的智能识别与推荐能力，难以满足不同环境下的定制化需求。最后，权限管理界面复杂，普通用户需要一定的培训才能熟练操作，同时前端页面在移动端适配性较差，影响移动设备用户的使用体验。针对这些问题，未来可以从多个方面进行改进：一是启用TiFlash和TiProxy等高级功能，提升系统在复杂查询和负载均衡方面的性能；二是优化SQL语句结构，减少动态SQL的使用，提高计划缓存命中率；三是引入网络爬虫或REST API调用方式，实现CVE数据的自动抓取和增量更新；四是结合自然语言处理技术，增强漏洞分类和修复建议的智能化水平；五是简化权限管理流程，提供图形化配置工具，并优化前端响应式设计，提升移动端兼容性；六是加强系统监控与告警机制，设置关键指标阈值，定期生成健康报告，为运维人员提供更全面的性能分析和优化建议。

4.3 未来展望

随着网络安全形势日益严峻，漏洞管理已成为企业安全运营的核心环节。未来，本系统将进一步拓展其应用场景和技术能力，朝着更加智能化、开放化、合规化和云原生化的方向发展。首先，构建智能化漏洞治理平台将成为核心目标之一。通过引入人工智能技术，系统将具备漏洞优先级自动排序、修复建议生成等功能，显著提升漏洞响应效率。同时，该平台还将与威胁情报系统集成，结合外部攻击行为数据，形成闭环的漏洞治理体系，实现从发现到修复的全流程自动化。其次，打造开放型漏洞数据共享平台将是推动行业生态建设的重要举措。系统将提供标准化的API接口，支持第三方平台调用CVE数据，促进漏洞信息的开放共享。此外，构建开源社区，鼓励开发者贡献漏洞分类规则、修复建议等内容，也将进一步丰富平台知识库，提升整体服务能力。第三，强化行业合规支持能力将成为系统的重要发展方向。针对《等级保护2.0》《关键信息基础设施保护条例》等相关法规要求，系统将增加合规性检查模块，自动生成审计报告，帮助企业在面对监管审查时快速响应。同时，系统将支持多租户架构，满足政府、金融等行业对数据隔离和权限管控的特殊需求，确保敏感数据的安全性。最后，探索云原生架构下的漏洞管理新模式将成为提升系统可用性和灵活性的关键。未来，系统可部署于Kubernetes集群，利用容器编排技术实现弹性伸缩、自动恢复等功能，从而更好地适应大规模分布式环境。同时，推动与云服务商合作，打造SaaS模式的漏洞管理服务，将有助于降低中小企业在安全投入方面的门槛，使更多组织能够便捷地接入漏洞管理能力。综上所述，本系统已初步形成了一个集漏洞管理、数据分析、可视化展示于一体的安全平台，未来将继续深耕技术创新与功能完善，助力企业构建更加高效、智能的漏洞治理体系，推动网络安全防护能力的整体提升。